

DoPE10

Software Manual

DOLI Elektronik GmbH

Rudolf-Diesel-Str. 3
72525 Münsingen, Germany

☎ Headquarter	+49 7381 767320 0
✉ Info	info@doli.de
🌐 Web	www.doli.de
☎ Support	+49 7381 767320 200
✉ Support	support@doli.de

Index

DoPE10 Software Manual.....	1
1 Overview	8
1.1 Tasks of DoPE	9
1.2 Initializing.....	9
1.3 Supported programming languages.....	10
2 Communication Principles.....	11
2.1 Open a connection	11
2.2 Using Eventhandler	11
2.3 Transaction Number (TAN)	12
3 Establish communication to EDC	13
3.1 DoPEOpenDeviceID / DoPEOpenFunctionID.....	13
3.2 DoPECloseLink	13
3.3 DoPEwOpenAll.....	14
3.4 DoPEwCloseAll	14
3.5 DoPEwPortInfo / DoPEwCurrentPortInfo	15
4 Message and Data handling.....	16
4.1 DoPECurrentData	16
4.2 DoPEGetData.....	16
4.3 DoPEClearReceiver	16
4.4 DoPEClearTransmitter	16
4.5 DoPEGetErrors	17
4.6 DoPEClearErrors.....	17
4.7 DoPEGetState	17
5 Movement commands	18
5.1 Simple movement commands.....	18
5.1.1 DoPEFMove, DoPEFMove_A.....	18
5.1.2 DoPEPos, DoPEPos_A	19
5.1.3 DoPEPosExt, DoPEPosExt_A.....	20
5.2 Halt commands	23
5.2.1 DoPEHalt, DoPEHalt_A	23
5.2.2 DoPESHalt	24
5.2.3 DoPEHaltW, DoPEHaltW_A	25
5.2.4 DoPEXpCont.....	26
5.2.5 DoPETrig, DoPETrig_A	27
5.3 Combined movement commands	29
5.3.1 DoPEBlockHeader, DoPEBlockExecute.....	29
5.4 Complex movement commands.....	30
5.4.1 DoPECycle.....	30
5.4.2 DoPEExt2Ctrl.....	31
5.4.3 DoPEFDPoti.....	32
5.4.4 DoPEOffsC.....	33
5.5 PC Command	34
5.5.1 DoPEPcCmd	34
5.5.2 DoPEPcCmdFromFile.....	36
5.5.2.1 Scale and Offsets.....	38
5.5.2.2 Start/End Position, SpeedToStart and Fade In/Out.....	39
5.5.2.3 Random Values	41
5.5.2.4 Cosine with one Cycle big Amplitude	42
5.5.2.5 Triangle with one Cycle big Amplitude.....	43

5.5.3	DoPERdPcCmdInfo	44
5.5.4	DoPECalculatePcCmd	44
5.5.5	DoPECalculatePcCmdFromFile	45
5.6	DoPEDynCycles	46
5.6.1	Basic Waveforms	48
5.6.1.1	Cosine/Sine	48
5.6.1.2	Triangle	50
5.6.1.3	Rectangle	50
5.6.1.4	Sawtooth	51
5.6.1.5	Sawtooth inverse	51
5.6.1.6	Pulse	52
5.6.2	Modify Parameter of an active test	52
5.6.3	Relative Destinations	53
5.6.4	Peak Control	54
5.6.5	Sweeps	55
5.6.6	Superposition Mode	57
5.6.7	Bimodal Control Mode	58
5.6.7.1	Control arithmetic mean value	58
5.6.7.2	Control mean value	58
5.6.7.3	Control minimum value	59
5.6.7.4	Control maximum value	59
5.6.7.5	Control minimum AND maximum values	60
5.6.7.6	Control minimum AND maximum values	60
5.6.8	Restrictions:	61
5.6.9	DoPESetPeakCtrl	62
5.6.10	DoPESetPeakCtrlValue	62
5.6.11	DoPEGetPeakCtrlValue	62
6	Synchronized data and movement	63
6.1	DoPESynchronizeData	64
6.2	DoPESetOnSynchronizeDataHdlr	64
6.3	DoPESetOnSynchronizeDataOverflowHdlr	64
6.4	DoPESynchronizeSystemMode	65
6.5	DoPESynchronizeSystemStart	65
7	Miscellaneous Control Commands	68
7.1	DoPEOn	68
7.2	DoPEOff	68
7.3	DoPEDefaultAcc	68
7.4	DoPESpeedLimit	69
7.5	DoPESetCtrl	69
7.6	DoPEEmergencyMove	70
7.7	DoPEEmergencyOff	70
7.8	DoPESetOpenLoopCommand	71
8	Event Handler	72
8.1	Using DoPE Event handler	72
8.2	Overview of Events:	72
8.3	DoPESetOnLineHdlr	73
8.4	DoPESetOnDataBlockHdlr	73
8.5	DoPESetOnDataBlockSize	74
8.6	DoPEGetOnDataBlockSize	74
8.7	DoPESetOnCommandErrorHdlr	74
8.8	DoPESetOnPosMsgHdlr	75
8.9	DoPESetOnTPosMsgHdlr	76
8.10	DoPESetOnLPosMsgHdlr	77
8.11	DoPESetOnSftMsgHdlr	77

8.12	DoPESetOnOffsCMsgHdlr	78
8.13	DoPESetOnCheckMsgHdlr	78
8.14	DoPESetOnRefSignalMsgHdlr	79
8.15	DoPESetOnSensorMsgHdlr	79
8.16	DoPESetOnIoSHaltMsgHdlr	80
8.17	DoPESetOnGuardMsgHandler	80
8.18	DoPESetOnKeyMsgHdlr	81
8.19	DoPESetOnRmcEventHdlr	81
8.20	DoPESetOnRuntimeErrorHdlr	82
8.20.1	List of Runtime errors	82
8.21	DoPESetOnOverflowHdlr	83
8.22	DoPESetOnSystemMsgHdlr	83
8.23	DoPESetOnDebugMsgHdlr	84
8.24	DoPEThreadPollHdlr	85
9	Configuration	86
9.1	DoPEwRdVersion	86
9.2	DoPEwRdLanguageInfo	86
9.3	Data Acquisition commands	87
9.3.1	DoPEsetDataTransmissionRate	87
9.3.2	DoPEsetTime	87
9.3.3	DoPETransmitData	87
9.3.4	DoPEFilterTime	88
9.4	Channel supervision	89
9.4.1	DoPEsetCheck	89
9.4.2	DoPEClrCheck	90
9.4.3	DoPEsetCheckLimit	90
9.4.4	DoPEClrCheckLimit	90
9.4.5	DoPEsetCheckLimitIO	90
9.5	Controller Parameter	91
9.5.1	DoPEPosPID	91
9.5.2	DoPERdPosPID	91
9.5.3	DoPEWrPosPID	91
9.5.4	DoPEspeedPID	92
9.5.5	DoPERdspeedPID	92
9.5.6	DoPEWrspeedPID	92
9.5.7	DoPEFeedForward	93
9.5.8	DoPERdFeedForward	93
9.5.9	DoPEWrFeedForward	93
9.5.10	DoPEOptimizeFeedForward	94
9.5.11	DoPECurrentPID	94
9.5.12	DoPEDestWnd	95
9.5.13	DoPESft	96
9.5.14	DoPECtrlSpeedTimeBase	96
9.5.15	DoPEDeadbandCtrl	97
9.5.16	DoPERdCtrlParameter	98
9.5.17	DoPEsetNominalAccSpeed	99
9.6	Tare functions	100
9.6.1	DoPE(Basic)TareSetValue	100
9.6.2	DoPE(Basic)TareSetDisplay	100
9.6.3	DoPE(Basic)Tare	101
9.6.4	DoPE(Basic)TareGetValue	101
9.7	Reference signal handling of incremental sensors	102
9.7.1	DoPEsetRefSignalMode	102
9.7.2	DoPEsetRefSignalTare	102
9.8	Calculated measuring channels	103

9.8.1	DoPEConfPeakValue.....	103
9.8.2	DoPEMc2Output	104
9.9	Sensor Correction	105
9.9.1	DoPESetSensorCorrection	105
9.9.2	DoPESetStiffnessCorrection.....	105
9.10	Serial Sensors	106
9.10.1	DoPEWrSensorMsg	106
9.10.2	DoPESerialSensorDef	106
9.10.3	DoPESetSerialSensor.....	106
9.10.4	DoPESetSerialSensorTransparent	107
9.11	Debug Messages	108
9.11.1	DoPEDebugMsgEnable	108
9.11.2	DoPESendDebugCommand	108
10	Input / Output-Commands.....	109
10.1	Analogue output	109
10.1.1	DoPESetOutput	109
10.1.2	DoPESetOutChannelOffset	109
10.1.3	DoPESetDither.....	109
10.1.4	DoPEOfflineActionOutput	110
10.2	Bit I/O-Commands	110
10.2.1	DoPESetBit	110
10.2.2	DoPECalOut.....	111
10.2.3	DoPEBeep	111
10.2.4	DoPEBypass	111
10.2.5	DoPERdBitInput.....	112
10.2.6	DoPEWrBitOutput	112
10.2.7	DoPEOfflineActionBitOutput	112
10.3	IOSignals.....	113
10.3.1	DoPEIOGripEnable	113
10.3.2	DoPEIOGripSet.....	113
10.3.3	DoPEIOExtEnable	114
10.3.4	DoPEIOExtSet	114
10.3.5	DoPEIOFixedXHeadEnable	114
10.3.6	DoPEIOFixedXHeadSet.....	114
10.3.7	DoPEIOHighPressureEnable	115
10.3.8	DoPEIOHighPressureSet.....	115
10.3.9	DoPEIOGuardSetLimitedMode	115
10.3.10	DoPEIOGuardSetUnlockRequest	115
10.3.11	DoPEIOGuardSetRangeLimit	115
10.3.12	DoPEIOGuardGetRangeLimit.....	116
10.3.13	DoPEIOGuardSetMuteGuard	116
10.3.14	DoPEIOGuardGetMuteGuard	116
10.3.15	DoPEIOGuardSetMuteSpeedLimit	116
10.3.16	DoPEIOGuardGetMuteSpeedLimit	117
10.3.17	DoPEIOGuardSetMuteRangeLimit	117
10.3.18	DoPEIOGuardGetMuteRangeLimit.....	117
10.4	Display Commands	118
10.4.1	DoPEDspClear.....	118
10.4.2	DoPEwDspHeadLine	118
10.4.3	DoPEwDspFKeys	118
10.4.4	DoPEwDspMValue	119
11	EDC keyboard interface	120
11.1	DoPERdNumberOfKeyboards	120
11.2	DoPESetKeyShiftState	120
11.3	DoPERdKeyShiftState.....	120

11.4	DoPESetLed	120
11.5	DoPELedMask	121
11.6	DoPECurrentKeys	121
11.7	DoPEKeyPressed	121
11.8	DoPEKeyGet	121
11.9	Definition of keys	122
11.10	Keyboard Lock Commands	123
11.10.1	DoPESetKeyboardLockState	123
11.10.2	DoPEGetKeyboardLockState	124
11.11	Definition of LEDs	125
11.12	RMC Default Handling	127
11.12.1	DoPERmcEnable	128
11.12.2	DoPERmcKeys	129
11.12.3	DoPERmcState	129
11.12.4	DoPESetOnRmcEventHdlr	129
12	Setup Commands	130
12.1	GeneralData	130
12.1.1	DoPEGeneralData Structure	130
12.1.2	DoPERdGeneralData	130
12.1.3	DoPEWrGeneralData	131
12.2	Machine Data	132
12.2.1	DoPEMachine Structure	132
12.2.1.1	DoPEMachineDef	132
12.2.1.2	DoPESenDef	133
12.2.1.3	DoPECtrlSenDef	134
12.2.1.4	DoPEOutChaDef	135
12.2.1.5	DoPEBitDef	135
12.2.1.6	DoPEIOSignals	136
12.2.1.6.1	DoPEIOGrip	136
12.2.1.6.2	DoPEIOExt	137
12.2.1.6.3	DoPEIOFixedXHead	137
12.2.1.6.4	DoPEIOHighPressure	138
12.2.1.6.5	DoPEIOMisc	138
12.2.1.6.6	DoPEIOSHalt	139
12.2.1.6.7	DoPEIOKey	139
12.2.1.6.8	DoPEIOLimitLoad	139
12.2.1.6.9	DoPEIOLimitLoad	140
12.2.1.6.10	DoPEIOGuard	141
12.2.1.7	DoPEMainMenu:	142
12.2.2	DoPERdMachine	142
12.2.3	DoPEWrMachine	142
12.2.4	DoPEMachineScale	143
12.2.5	DoPERdMachineNumber	143
12.2.6	DoPESelMachine	143
12.3	DoPEInitialize	144
12.4	DoPEInitializeResetXHead	144
12.5	DoPERdSensorInfo	144
12.6	DoPERdSensorUserData	145
12.7	DoPEWrSensorUserData	145
12.8	DoPEwRdModuleInfo	145
12.9	DoPEwRdDriveInfo	146
13	Sensor EEPROM Handling	147
13.1	DoPERdSensorConKey	147
13.2	DoPERdSensorHeaderData	147
13.3	DoPEWrSensorHeaderData	148

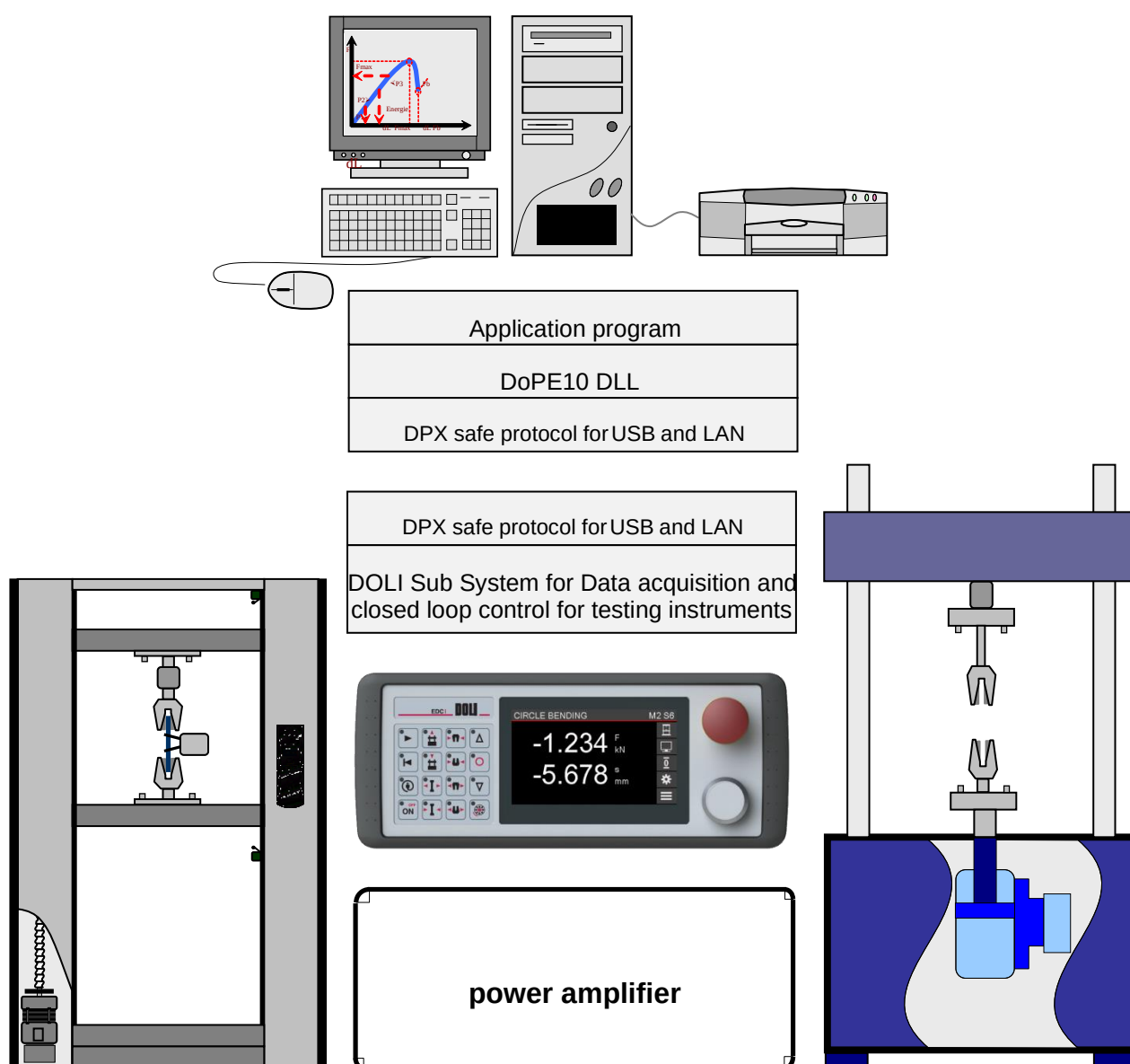
13.4	DoPERdSensorAnalogueData	149
13.5	DoPEWrSensorAnalogueData	149
13.6	DoPERdSensorIncData	150
13.7	DoPEWrSensorIncData	150
13.8	DoPERdSensorAbsData	151
13.9	DoPEWrSensorAbsData	151
13.10	DoPECalcSensorAbsGenericSignalType	152
13.11	DoPECalcSensorAbsGenericData	152
14	Reestablishing a Connection.....	153
14.1	DoPEReInitializeEnable	153
14.2	DoPEReInitialize	153
15	Default Measuring Data Record.....	155
15.1	Logical Input Signals	156
15.2	Logical Output Signals	157
15.3	Controller State WORD 1	158
15.4	Controller State WORD 2	158
15.5	Drive Interface State Input 1.....	159
15.6	Drive Interface State Input 2.....	160
15.7	Drive Interface State Output.....	161
15.8	Detecting Cycles	162
16	DoPE System Message	163
17	DoPE Error Constants	166
18	For Experts	167
18.1	Realtime Version of DoPE Event Handler.....	167
19	.NET Wrapper	168
19.1	Establish Communication	168
19.2	Event Handler	168
19.3	.NET API function groups.....	169
19.4	IntelliSense	170
20	Versions	171

1 Overview

DOLI has very powerful software for data acquisition and closed loop control for testing instruments like tensile testers. This software runs on the DOLI EDCi hardware platform like EDCi50 (support for older hardware (EDC5 to EDC220V/580V) is discontinued). It is configurable within a wide range to satisfy all requirements at a testing instrument.

Communication between EDC and PC is handled by a communication layer, called DPX. DPX is a secured protocol and currently supports USB, and Ethernet (LAN) as transportation layer.

DoPE10 represents the interface to access all EDC-functions from any standard PC with Windows®.



1.1 Tasks of DoPE

- DoPE will initialize the EDC according to predefined set-up data, stored in EDC memory.
- DoPE provides the application program with measuring data in SI-units (N, m etc.)
- DoPE offers a wide range of machine control commands
Simple commands like:
„move Up in position control with 0.01 m/s“
Complex commands like:
„move in position control with 0.02 m/s to 500 N and keep 500 N in load control“
Cycling commands like:
„do 100 sinusoidal load cycles with an amplitude of 100N with 3 Hz“
- DoPE reports events like limit switch etc.

DoPE is available as a

- 32/64 Bit DLL for Windows 7/8/8.1/10/11
- 64 Bit Shared Object for x64 Linux

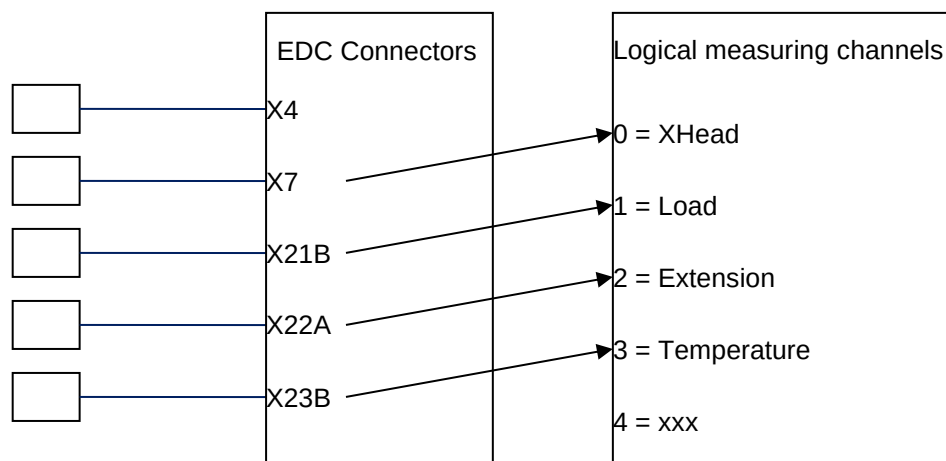
1.2 Initializing

The EDC firmware can handle up to:

- sixteen logical measuring channels
- sixteen logical analogue output channels
- ten logical Bit input devices
- ten logical Bit output devices

The above logical channels and devices must be assigned to physical interfaces. For assignment you just have to specify the connector number, the logical measuring channel and some transducer specific data.

Use DOLI Installation Center to edit all necessary set-up data.



All Set-up data will be stored in the EDC.

1.3 Supported programming languages

DoPE is delivered as a DLL running under Windows ® operating system. Each programming language that supports the use of DLLs can be used for the application program.

However, DOLI supplies only header files for “C”, “Delphi/Pascal”, and a .NET wrapper.

For these programming languages, simple demo programs are available.

Two DLLs are needed to use DoPE: “DoPE10.DLL” and “DoDPX10.DLL”. The .NET wrapper is placed in the “DoPE10Net.DLL”.

For the 64bit Version, “DoPE10x64.DLL” and “DoDPX10x64.DLL” is needed. And the 64 bit .NET wrapper is placed in the “DoPE10x64Net.DLL”.

2 Communication Principles

DoPE represents a library that uses functions inside the remote EDC controller. Physically EDC and PC are connected via a standard communication port, like USB or LAN. A safe protocol is used for communication. This protocol, named DPX, detects ON/OFF-line transitions, transmits messages and measured data packets. The user of DoPE must always be aware of the remote controller and due to this an asynchronous behavior of DoPE.

2.1 Open a connection

The application program establishes a connection by calling one of the **DoPEOpen...** functions. This function creates a thread with a priority depending on the priority of the application thread.

Normally the DoPE thread will use the priority `THREAD_PRIORITY_TIME_CRITICAL`. If the application thread has `REALTIME_PRIORITY_CLASS`, the DoPE thread will use the priority `THREAD_PRIORITY_ABOVE_NORMAL`.

The application thread should not start another thread with a higher priority than its own. Otherwise the communication with the EDC might be delayed. This will cause unpredictable problems.

By this mechanism it is safeguarded that DoPE and hence the communication with the EDC has always a higher priority than the application program.

2.2 Using Eventhandler

Whenever necessary, EDC will send a message, possibly with parameter to PC. There are many different reasons for such messages. E.g. if a positioning command has reached the destination, EDC will send a message to PC, reporting the destination has been reached.

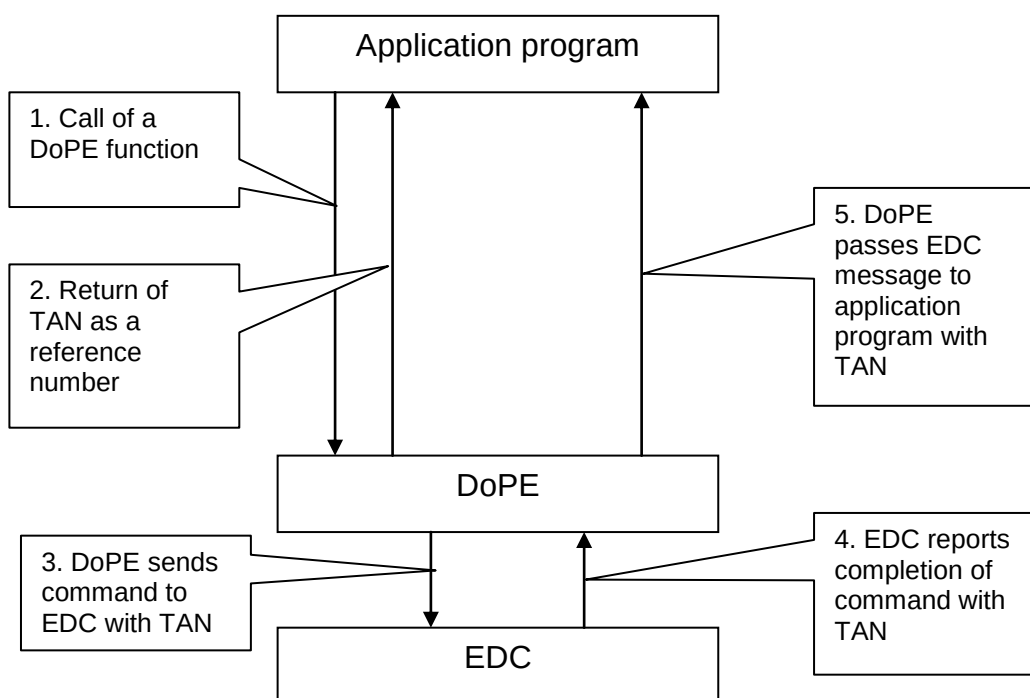
To make it easy for the application program to get these messages, DoPE offers an event handler interface. For each event, the application program wants to process, an event handler must be installed. DoPE will call these event handlers, whenever the event occurs.

2.3 Transaction Number (TAN)

Some of the following commands have pointers to a transaction number (lpusTAN) as a parameter. This TAN may be used to identify messages, which are received asynchronously, to the command.

Example: A positioning command is sent to the EDC. After 30 seconds the desired position is reached and a message "position reached" is received. This message comes with the TAN of the positioning command.

The TAN is generated by DoPE for all commands, which may send a message later on. It's optional to use the TAN. If NULL pointers are passed, no TANs are returned.



3 Establish communication to EDC

3.1 DoPEOpenDeviceID / DoPEOpenFunctionID

Open the given DoPE link with a matching device or function ID. All USB and LAN communication ports are scanned, starting with USB. All LAN ports of the PC are included to the scan. The link parameters are set and the link is established. If DoPEOpenDeviceID / DoPEOpenFunctionID returns DoPERR_TIMEOUT, connection to the EDC did not go online. You must connect the EDC, switch it on and try the open function again. If the return code is DoPERR_VERSION an EDC went online but either the DoPE API version or the EDC type didn't match.

If the parameter DeviceID in DoPEOpenDeviceID is set to ZERO, DoPE will open communication to the first EDC that answers. This method is recommended for systems with only one EDC.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description
extern unsigned DLLAPI	DoPEOpenDeviceID / DoPEOpenFunctionID (	Function returns Error constant (DoPERR_XXXX)
unsigned	DeviceID/ FunctionID	EDC Device ID / Function ID
unsigned	RcvBuffers	Number of requested receiver buffers for messages. This number of messages can be stored inside DoPE until they are read with DoPEGetMsg function.
unsigned	XmitBuffers	Number of requested transmitter buffers for messages. This number of messages can be stored inside DoPE. They will be transmitted by DoPE to EDC.
unsigned	DataBuffers	Number of requested data buffers. The measuring data record will be stored inside DoPE in a circular buffer. If data are not read with DoPEGetData the oldest record will be overwritten!
unsigned	APIVersion	DoPE API version used by the DoPE user.
void	*Reserved	Reserved for future use
DoPE_HANDLE	*DoPEHdl	Pointer to memory for DoPE link handle
		This handle has to be used for all further DoPE commands as a reference for this link.
.NET <Edc object> = new Edc(DoPE.OpenBy.DeviceID/FunctionID, 0);		

3.2 DoPECloseLink

Close the link and free all allocated memory.

After this call DoPEHdl is invalid and all further calls with this DoPEHdl will return with an error.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description
extern unsigned DLLAPI	DoPECloseLink (	Function returns Error constant (DoPERR_XXXX)
DoPE_HANDLE	*DoPEHdl)	Pointer to DoPE link handle (is set to NULL after call)
.NET <Edc object>.Dispose()		

3.3 DoPEwOpenAll

Open all available DoPE links and fill the open link info table. All USB and LAN communication ports are scanned, starting with DoPEPORT_USB. All LAN ports of the PC are included to the scan.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI unsigned unsigned unsigned unsigned void unsigned unsigned DoPEwOpenLinkInfo	DoPEwOpenAll (RcvBuffers XmitBuffers DataBuffers APIVersion *Reserved InfoTableMaxEntries *InfoTableValidEntries InfoTable[])
Function returns Error constant (DoPERR_XXXX) Number of requested receiver buffers for messages. This number of messages can be stored inside DoPE until they are read with DoPEGetMsg function. Number of requested transmitter buffers for messages. This number of messages can be stored inside DoPE. They will be transmitted by DoPE to EDC. Number of requested data buffers. The measuring data record will be stored inside DoPE in a circular buffer. If data are not read with DoPEGetData the oldest record will be overwritten! DoPE API version used by the DoPE user. Reserved for future use Number of entries that can be stored to the info table. Pointer to storage for the number of valid entries in the info table. Info table will be filled by with all detected EDC's	

```
typedef struct
{
    DoPE_HANDLE    DoPEHdl;
    unsigned       PortType;                /* DoPEPORT_USB or DoPEPORT_LAN */
    DoPE_wPORTINFO PortInfo;
    DoPEwModuleInfo ModuleInfo;
} DoPEwOpenLinkInfo;
```

.NET <EdcList object> = new EdcList(int MaximumNumberOfEdcs)

3.4 DoPEwCloseAll

Close all DoPE links previously opened by DoPEwOpenAll.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI unsigned DoPEOpenLinkInfo	DoPECloseAll (InfoTableValidEntries InfoTable[])
extern unsigned DLLAPI unsigned DoPEwOpenLinkInfo	DoPEwCloseAll (InfoTableValidEntries InfoTable[])
Function returns Error constant (DoPERR_XXXX) Number of valid entries in the info table. Info table containing valid DoPE handles for all links to close. All DoPE handles in the InfoTable are set to NULL.	
Function returns Error constant (DoPERR_XXXX) Number of valid entries in the info table. Info table containing valid DoPE handles for all links to close. All DoPE handles in the InfoTable are set to NULL.	

.NET <EdcList object>.Dispose()

3.5 DoPEwPortInfo / DoPEwCurrentPortInfo

Get the port information for PC-communication ports (USB, LAN)		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPEwPortInfo (unsigned Port, unsigned First, DoPE_wPORTINFO *pPortInfo)	Function returns Error constant (DoPERR_xxxx) (DPXERR_BADPORT if no more port info is available) Communication Port Type. (DoPEPORT_USB, DoPEPORT_LAN) != 0: get the first port information == 0: get the next port information pointer to port info structure	
.NET RdPortInfo(DoPE.Port Port, bool First, ref DoPE.PortInfo PortInfo)		
Get the current port information of an established link		
Function declaration	Description	
extern unsigned DLLAPI DoPECurrentPortInfo (DoPE_HANDLE DoPEHdl, DoPE_wPORTINFO *pPortInfo)	Function returns Error constant (DoPERR_xxxx) DoPE link handle pointer to port info structure	
extern unsigned DLLAPI DoPEwCurrentPortInfo (DoPE_HANDLE DoPEHdl, DoPE_wPORTINFO *pPortInfo)	Function returns Error constant (DoPERR_xxxx) DoPE link handle pointer to port info structure	
<pre>typedef struct { unsigned Ix; /* Device index */ wchar_t Name[DoPE_PORTNAMELEN]; /* Device name */ unsigned PortType; /* DoPEPORT_USB or DoPEPORT_LAN */ MAC NicMAC; /* NIC address */ } DoPE_wPORTINFO;</pre>		
.NET <EdcList object>.PortInfo		

4 Message and Data handling

4.1 DoPECURRENTData

Get current samples from receiver buffer.
DoPE receives measuring data in an adjustable time scale. The data record is stored inside DoPE in a circular buffer. You can read the latest data record with this function.
For definition of DoPEData record refer to chapter 15.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEData DoPECURRENTData(DoPEHdl, *Sample)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to storage for data record.
.NET <Edc object>.Data.CurrentData(ref DoPE.Data Sample)	

4.2 DoPEGetData

Get oldest sample from receiver buffer.
DoPE receives measuring data in an adjustable time scale. The data record is stored inside DoPE in a circular buffer. You can read the oldest data record in the buffer with this function.
For definition of DoPEData record refer to chapter 15.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEData DoPEGetData(DoPEHdl, *Sample)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to storage for data record.
.NET <Edc object>.Data.GetData(ref DoPE.Data Sample)	

4.3 DoPEClearReceiver

Discard all received messages. If you want to get rid of old messages, that are of no interest anymore, use this command.

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEClearReceiver (DoPEHdl)	Function returns Error constant (DoPERR_XXXX) DoPE link handle
.NET not available	

4.4 DoPEClearTransmitter

Discard all unsent transmitter buffers

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEClearTransmitter (DoPEHdl)	Function returns Error constant (DoPERR_XXXX) DoPE link handle
.NET not available	

4.5 DoPEGetErrors

Get current error counter values. DoPE counts all sort of communication errors. You may read these errors by using this command.

The DoPEError-structure contains several communication error counters. Whenever an error is detected, the appropriate counter is increased by one. This error counters are useful to analyze communication problems.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEError *	DoPEGetErrors(DoPEHdl, Error)
Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to buffer for error counters	
typedef struct { unsigned Overrun; unsigned InvACK; unsigned NoBuffer; unsigned BccErr; unsigned MwError; } DoPEError;	Error counters DoPE link handle LAN Packet Overrun errors Invalid ACKs received No receiver buffer available Checksum error Invalid sample encoding
.NET not available	

4.6 DoPEClearErrors

Clear current error counter values.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE	DoPEClearErrors(DoPEHdl)
Function returns Error constant (DoPERR_XXXX) DoPE link handle	
.NET not available	

4.7 DoPEGetState

Get state information structure.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEState	DoPEGetState (DoPEHdl *Status)
Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to buffer for state information	
typedef struct { unsigned ComState; unsigned RcvBuffer; unsigned XmitBuffer; } DoPEState; Constants for ComState COM_STATE_OFF COM_STATE_OFFLINE COM_STATE_INITCYCLE COM_STATE_ONLINE	Communication state Number messages in receive queue Number of messages to be transmitted Link is disabled Link is offline Link is initializing Link is established and Online
.NET not available	

5 Movement commands

DoPE provides several commands for moving the crosshead of a machine. The movement may be in different control modes, such as position, load or extension control. There are simple commands like move up, or down with a certain speed without explicit destination, or commands with a given destination. Furthermore DoPE supplies commands with a destination in another control channel such as go in position control to a load destination, and cyclic commands.

For most of the movement commands DoPE provides a command that uses default acceleration e.g. DoPEPos and another where acceleration and deceleration are specified in the command e.g. DoPEPos_A.

After a movement has been finished, the event OnPosMsg will be activated.

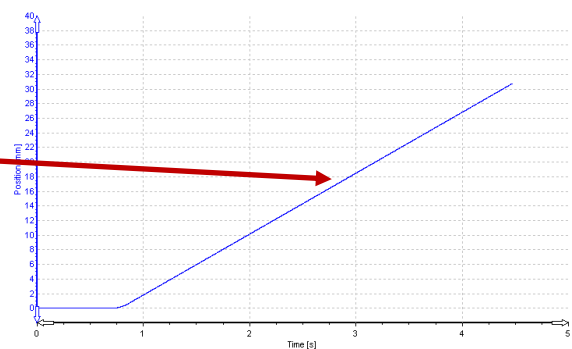
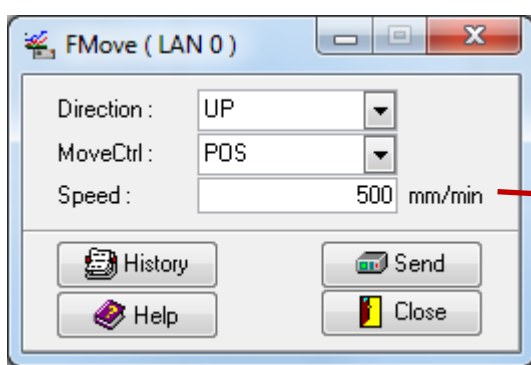
A new movement command terminates an active movement command.

5.1 Simple movement commands

5.1.1 DoPEFMove, DoPEFMove_A

Move crosshead in the specified control mode and speed UP or DOWN. As an implicit limit of this command, softends are used, if active.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

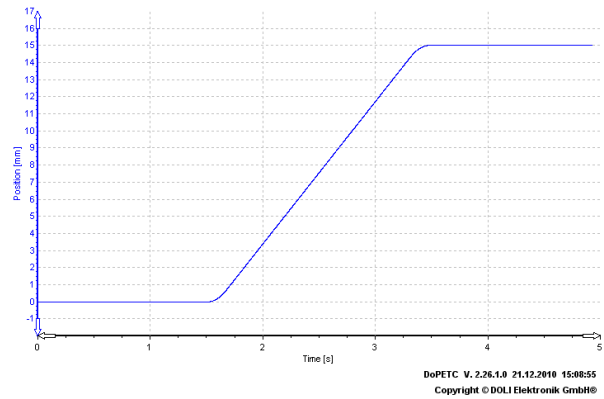
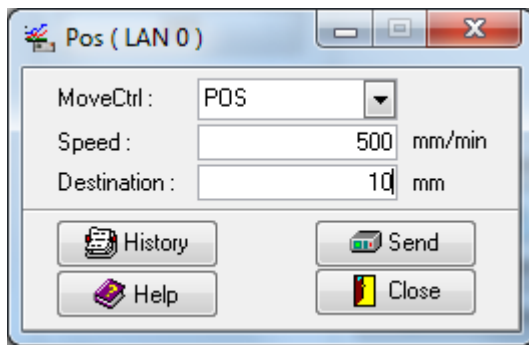


Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short double unsigned short *	DoPEFMove(DoPEHdl, Direction, MoveCtrl, Speed, lpusTAN);	Function returns Error constant (DoPERR_XXXX) DoPE link handle Direction of movement Control mode of movement (CTRL_XXX) Speed for movement Pointer to transaction number.	Unit/s
.NET <Edc object>.Move.FMove(DoPE.MOVE Direction, DoPE.CTRL MoveCtrl, Double Speed, ref Int16 Tan)			
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short double double unsigned short *	DoPEFMove_A(DoPEHdl, Direction, MoveCtrl, Acc Speed, lpusTAN);	Function returns Error constant (DoPERR_XXXX) DoPE link handle Direction of movement Control mode of movement(CTRL_XXX) Acceleration Speed for movement Pointer to transaction number.	Unit/s ² Unit/s
.NET <Edc object>.Move.FMove_A(DoPE.MOVE Direction, DoPE.CTRL MoveCtrl, Double Acc, Double Speed, ref Int16 Tan)			

5.1.2 DoPEPos, DoPEPos_A

Move crosshead in the specified control mode and speed to the given destination.
DoPEPos will use the default acceleration while DoPEPos_A uses the given acceleration.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



Function declaration

extern unsigned DLLAPI
DoPE_HANDLE
unsigned short
double
double
unsigned short *

DoPEPos (
DoPEHdl,
MoveCtrl,
Speed,
Destination
IpusTAN);

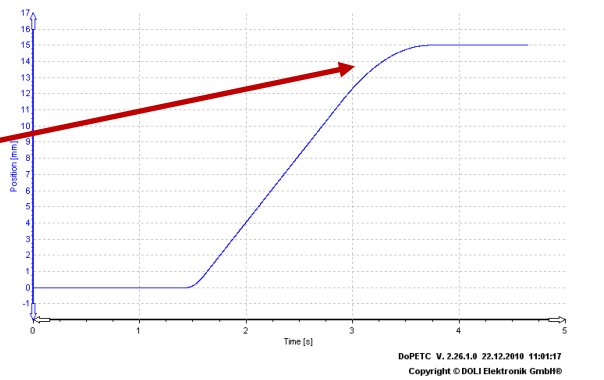
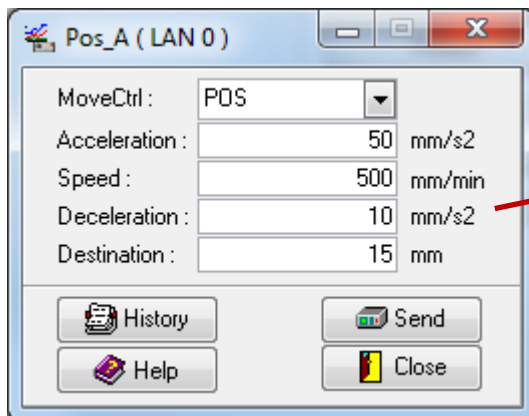
Description

Function returns Error constant (DoPERR_xxxx)
DoPE link handle
Control mode (CTRL_xxx)
Speed for movement
Final destination
Pointer to transaction number.

Unit

Unit/s
Unit

.NET <Edc object>.Move.Pos(DoPE.CTRL MoveCtrl, Double Speed, Double Destination, ref Int16 Tan)



Function declaration

extern unsigned DLLAPI
DoPE_HANDLE
unsigned short
double
double
double
double
unsigned short *

DoPEPos_A (
DoPEHdl,
MoveCtrl,
Acc
Speed,
Dec
Destination
IpusTAN);

Description

Function returns Error constant (DoPERR_xxxx)
DoPE link handle
Control mode (CTRL_xxx)
Acceleration
Speed for movement
Deceleration
Final destination
Pointer to transaction number.

Unit

Unit/s²
Unit/s
Unit/s²
Unit

.NET <Edc object>.Move.Pos_A(DoPE.CTRL MoveCtrl, Double Acc, Double Speed, Double Dec, Double Destination, ref Int16 Tan)

5.1.3 DoPEPosExt, DoPEPosExt_A

Move crosshead in the specified control mode 'MoveCtrl' and 'Speed' to the given 'Destination'. 'DestinationCtrl' may be different to 'MoveCtrl'. Default acceleration and deceleration will be used. The Parameter 'DestinationMode' specifies how to reach destination position and the action after reaching it. In case the limit position is reached before destination position, crosshead will be halted in 'MoveCtrl' at limit position.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

PosExt (LAN 0)

MoveCtrl: POS

Speed: 5 mm/min

Limit mode: NOT ACTIVE

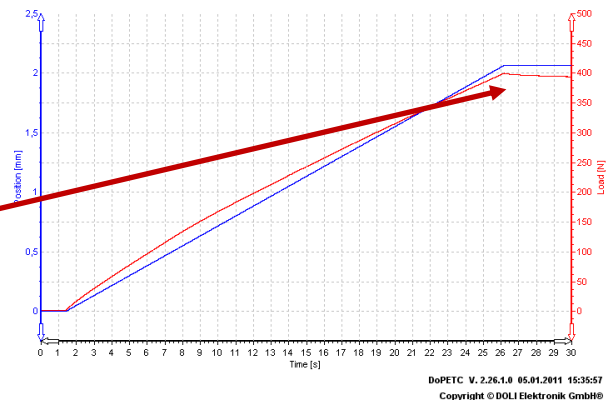
Limit: 0 mm

DestCtrl: LOAD

Destination: 400 N

Dest. mode: APPROACH

History Send Help Close



Halt in position control after load destination is reached. Due to specimen relaxation, load will decrease!

PosExt (LAN 0)

MoveCtrl: POS

Speed: 5 mm/min

Limit mode: NOT ACTIVE

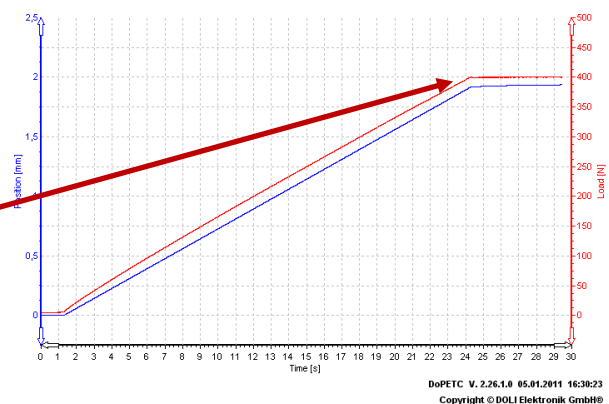
Limit: 0 mm

DestCtrl: LOAD

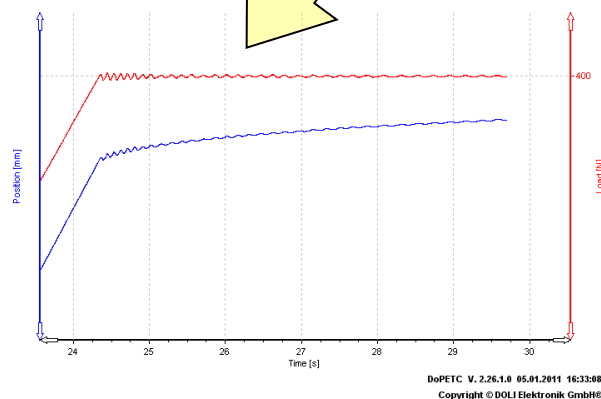
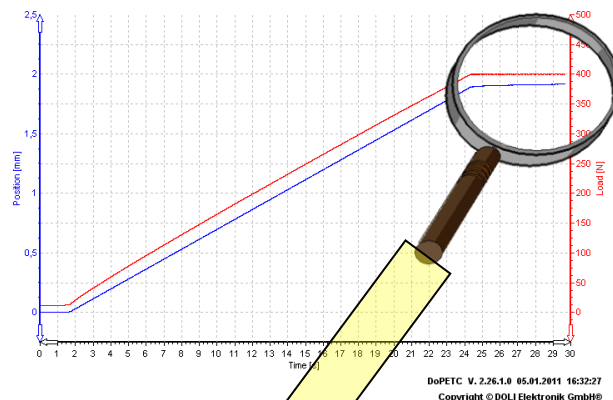
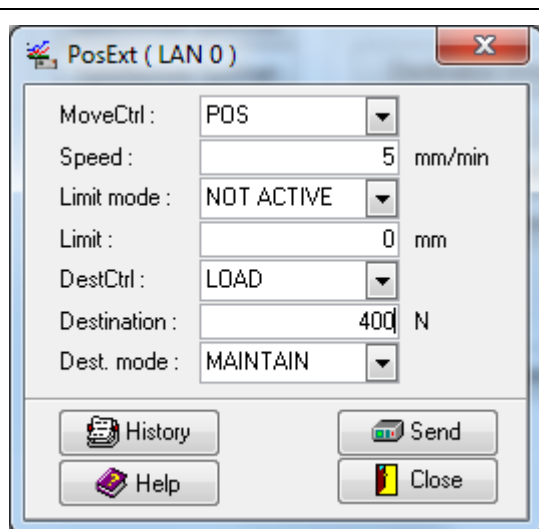
Destination: 400 N

Dest. mode: POSITION

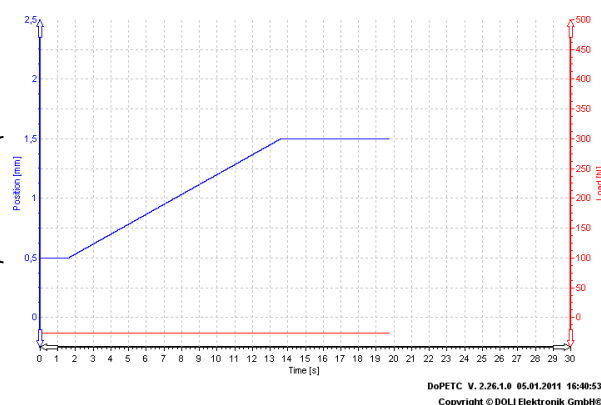
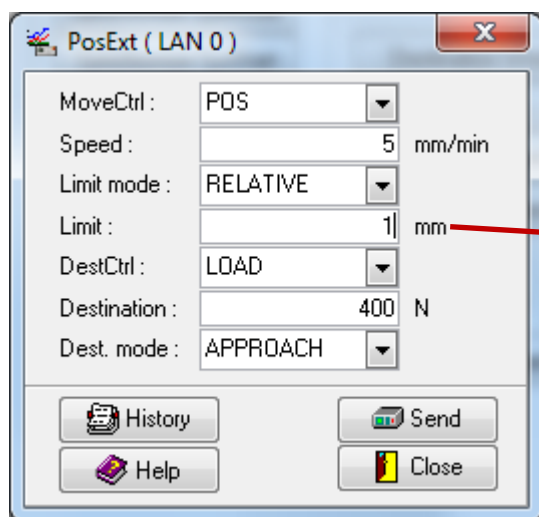
History Send Help Close



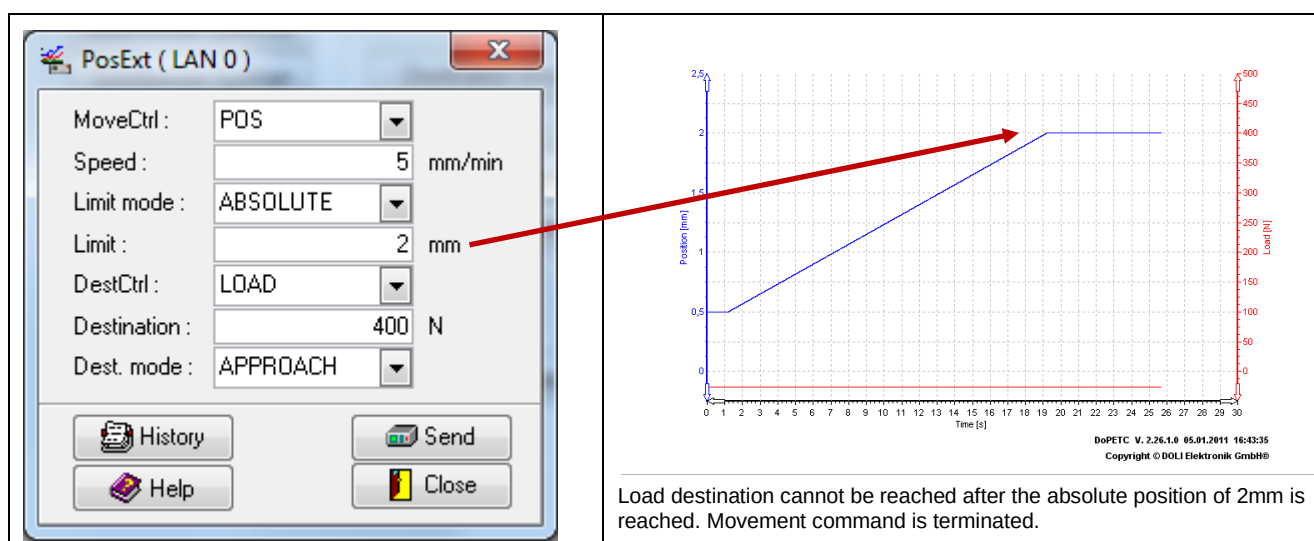
Switch to load control just before load destination is reached. Load is kept constant, position is increasing, due to specimen relaxation.



Halt in position control after load destination is reached and maintain this load destination in position control.



Load destination cannot be reached after 1 mm relative movement. Movement command is terminated.



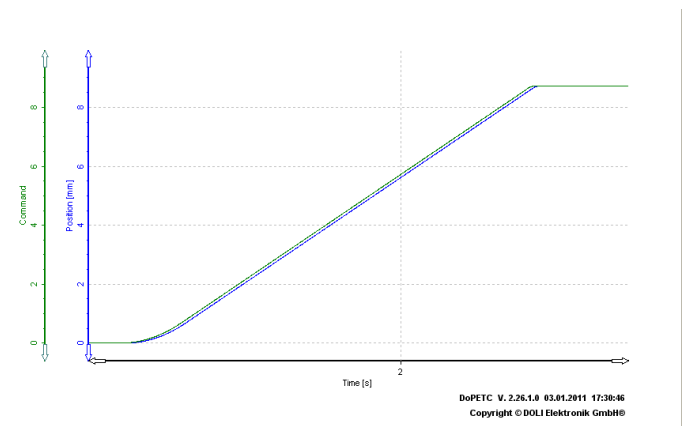
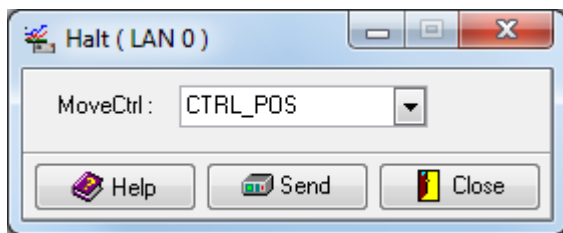
Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double unsigned short	DoPEPosExt (DoPEHdl, MoveCtrl, Speed, LimitMode,	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_xxx) Speed for movement LIMIT_ABSOLUTE: Limit is a absolute position LIMIT_RELATIVE: Limit is a distance (relative) position LIMIT_NOT_ACTIVE: No Limit is active Limit position in case Destination cannot be reached Channel definition for destination (CTRL_xxx) Final destination DEST_APPROACH: Halt after reaching destination, do not change control mode. DEST_POSITION: Switch to 'DestinationCtrl' just before 'Destination' is reached. Then move to 'Destination'. DEST_MAINTAIN: No change of control mode at destination but maintain destination in 'MoveCtrl' mode. Pointer to transaction number.	Unit/s
double unsigned short double unsigned short	Limit, DestinationCtrl, Destination DestinationMode,		Unit
unsigned short *	lpustAN);		
.NET <Edc object>.Move.PosExt(DoPE.CTRL MoveCtrl, Double Speed, DoPE.LIMITMODE LimitMode, Double Limit, DoPE.CTRL DestinationCtrl, Double Destination, DoPE.DESTMODE DestinationMode, ref Int16 Tan)			
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double double unsigned short	DoPEPosExt_A (DoPEHdl, MoveCtrl, Acc, Speed, DecLimit, LimitMode,	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_xxx) Acceleration Speed for movement Deceleration at Limit LIMIT_ABSOLUTE: Limit is a absolute position LIMIT_RELATIVE: Limit is a distance (relative) position LIMIT_NOT_ACTIVE: No Limit is active Limit position in case Destination cannot be reached Channel definition for destination (CTRL_xxx) Deceleration at Limit Final destination DEST_APPROACH: Halt after reaching destination, do not change control mode. DEST_POSITION: Switch to 'DestinationCtrl' just before 'Destination' is reached. Then move to 'Destination'. DEST_MAINTAIN: No change of control mode at destination but maintain destination in 'MoveCtrl'mode. Pointer to transaction number.	Unit/s² Unit/s Unit/s²
double unsigned short double double unsigned short	Limit, DestinationCtrl, DecDestination, Destination DestinationMode,		Unit
unsigned short *	lpustAN);		
.NET <Edc object>.Move.PosExt_A(DoPE.CTRL MoveCtrl, Double Acc, Double Speed, Double DecLimit, DoPE.LIMITMODE LimitMode, Double Limit, DoPE.CTRL DestinationCtrl, Double DecDestination, Double Destination, DoPE.DESTMODE DestinationMode, ref Int16 Tan)			

5.2 Halt commands

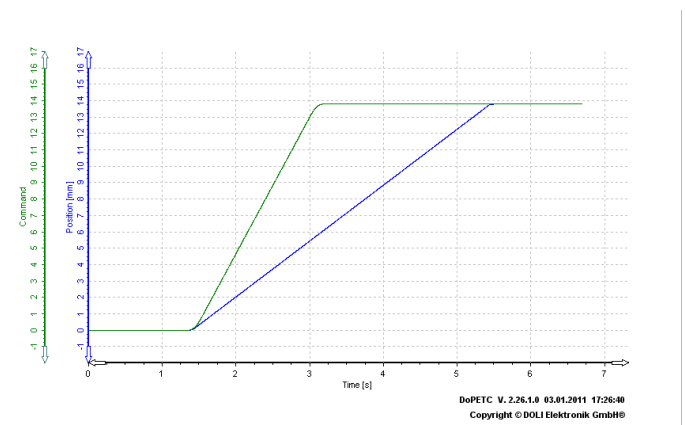
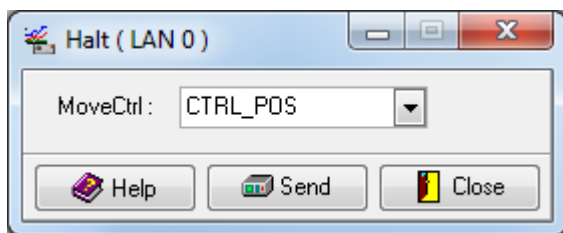
5.2.1 DoPEHalt, DoPEHalt_A

HALT is understood as a deceleration from current speed to speed zero. This is done by the build in position generator. If not specified by the halt command default deceleration will be used. The final destination position cannot be specified, it depends on current speed and deceleration.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



DoPEHalt will terminate a current movement command. The Command will be decelerated to ZERO speed.



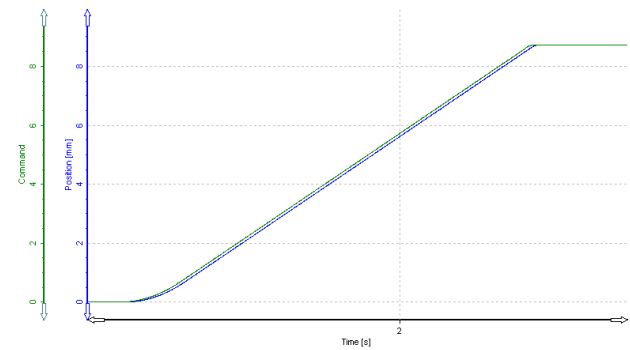
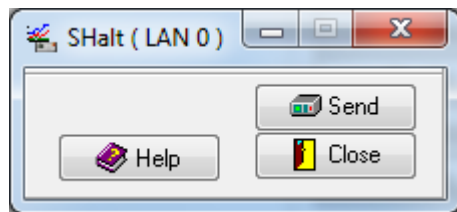
The machine cannot follow the desired speed.
(nominal speed in set-up is too high)
Command will be decelerated to ZERO speed.
After the command has decreased speed to zero, the machine is still moving, until the command position has been reached.

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short *	DoPEHalt(DoPEHdl, MoveCtrl, lpusTAN);	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_XXX) Pointer to transaction number.	
.NET <Edc object>.Move.Halt(DoPE.CTRL MoveCtrl, ref Int16 Tan)			
extern unsigned DLLAPI DoPE_HANDLE unsigned short double unsigned short *	DoPEHalt_A(DoPEHdl, MoveCtrl, Dec lpusTAN);	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode of movement (CTRL_XXX) Deceleration Pointer to transaction number.	Unit/s ²
.NET <Edc object>.Move.Halt_A(DoPE.CTRL MoveCtrl, Double Dec, ref Int16 Tan)			

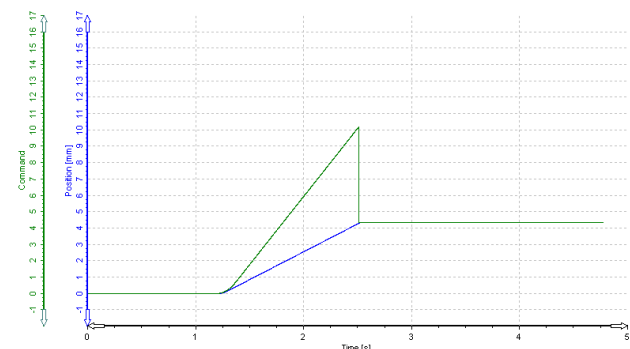
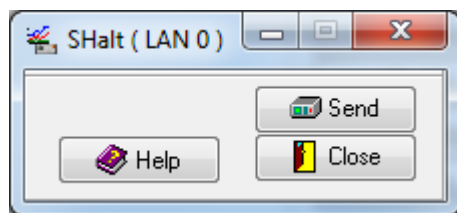
5.2.2 DoPEShAlt

Instant Halt of crosshead in position control mode. If needed, the control mode is changed to position control. Before decelerating with default deceleration, the command is set to the current position. This is the fastest method to halt the machine in position control.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



DoPEShAlt will terminate a current movement command. If nominal speed is set correctly, there is no difference to DoPEHalt command.



The machine cannot follow the desired speed. (nominal speed in set-up is too high)
Command will be set to current position and then decelerated to ZERO speed.

Function declaration	Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short * DoPEShAlt(DoPEHdl, lpusTAN);	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Pointer to transaction number.	

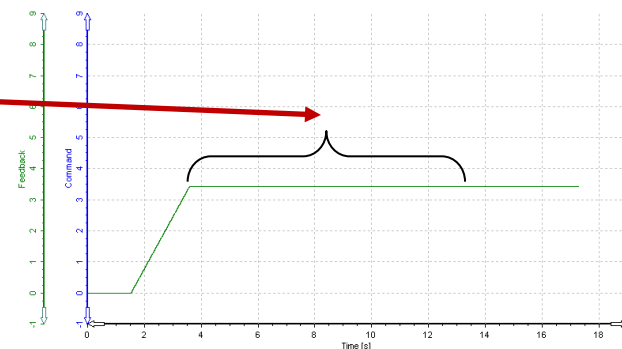
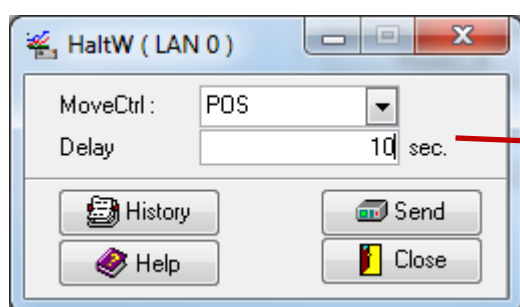
.NET <Edc object>Move.Shalt(ref Int16 Tan)

5.2.3 DoPEHaltW, DoPEHaltW_A

Decelerate from current speed to speed zero and wait delay time. After delay time has expired, the command has finished and a position reached message is generated.

This command is only useful inside a combined movement command.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



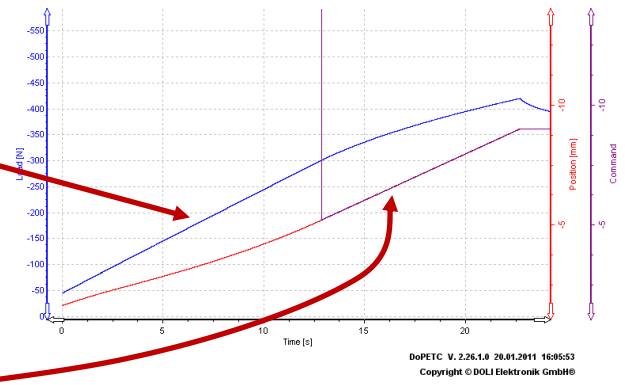
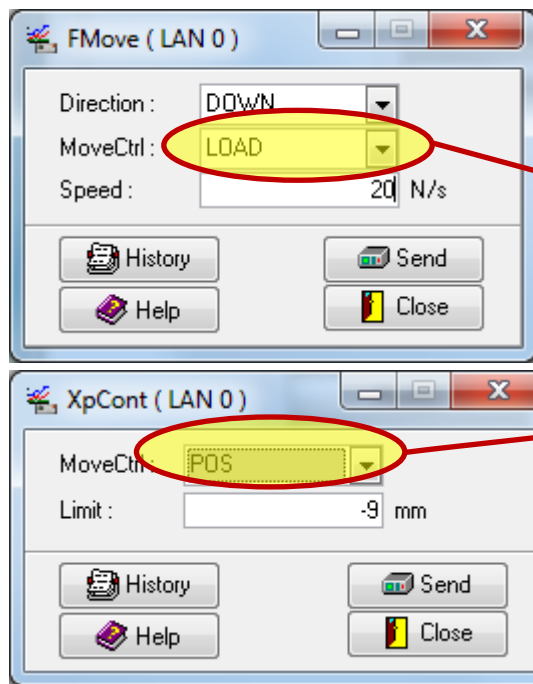
DoPEHalt will terminate a current movement command. The message, indicating the end of this command will be delayed. (here 10 seconds)

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double unsigned short *	DoPEHaltW(DoPEHdl, MoveCtrl, Delay lpusTAN);	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_XXX) Delay time Pointer to transaction number.	s
.NET <Edc object>.Move.HaltW(DoPE.CTRL MoveCtrl, Double Delay, ref Int16 Tan)			
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double unsigned short *	DoPEHaltW_A(DoPEHdl, MoveCtrl, Dec Delay lpusTAN);	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode of movement (CTRL_XXX) Deceleration Delay time Pointer to transaction number.	Unit/s ² s
.NET <Edc object>.Move.HaltW_A(DoPE.CTRL MoveCtrl, Double Dec, Double Delay, ref Int16 Tan)			

5.2.4 DoPEXpCont

Change control mode and continue movement in the new control mode with the current speed.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



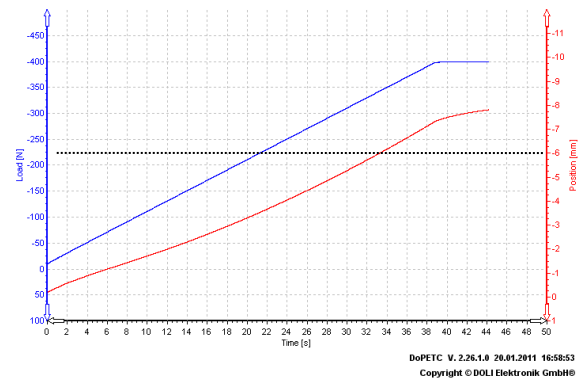
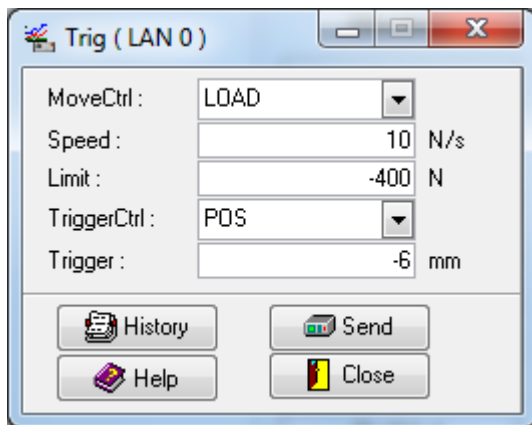
Machine is moving in load control with 20 N/s. At about 17 s, the DoPEXpCont command switches to position control, and continues with the current speed in position. After reaching the Limit position, the command is terminated.

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double unsigned short *	DoPEXpCont(DoPEHdl, MoveCtrl, Limit lpusTAN);	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) Limit position Pointer to transaction number.	Unit
.NET <Edc object>.Move.XpCont(DoPE.CTRL MoveCtrl, Double Limit, ref Int16 Tan)			

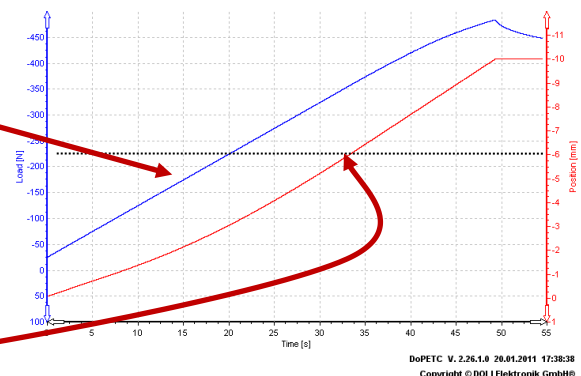
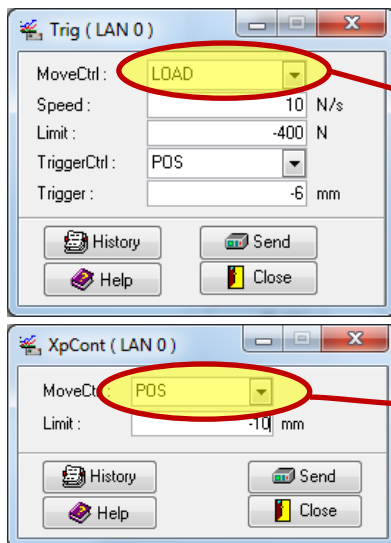
5.2.5 DoPETrig, DoPETrig_A

Move crosshead with the specified speed to the limit position. If the trigger position is reached a message will be transmitted and if used inside a combined movement sequence, the next command is activated.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



Move in load control with 10 N/s to -400N. A trigger message is generated at -6mm position. Upon this, the PC program may send another e.g. movement command.



The DoPETrig command can be used inside a combined movement command. Upon reaching -6mm, the next command will be automatically started. Here go with the current speed in position to -10mm.

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double unsigned short double unsigned short *	DoPETrig(DoPEHdl, MoveCtrl, Speed, Limit, TriggerCtrl, Trigger, lpusTAN);	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Control mode (CTRL_ xxx) Speed for movement Limit position in case Destination cannot be reached Control mode for trigger channel(CTRL_ xxx) Trigger level Pointer to transaction number.	Unit/s
.NET <Edc object>.Move.Trig(DoPE.CTRL MoveCtrl, Double Speed, Double Limit, DoPE.CTRL TriggerCtrl, Double Trigger, ref Int16 Tan)			

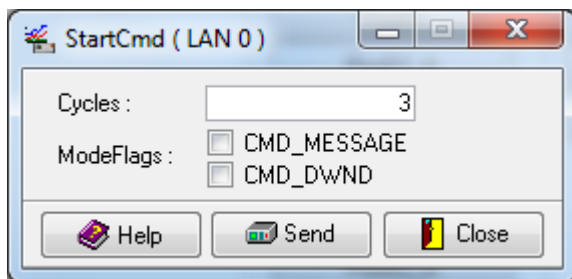
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double double double unsigned short double unsigned short *	DoPETrig_A(DoPEHdl, MoveCtrl, Acc, Speed, Dec, Limit, TriggerCtrl, Trigger, lpusTAN);	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) Acceleration Speed for movement Deceleration Limit position in case Destination cannot be reached Control mode for trigger channel (CTRL_xxx) Trigger level Pointer to transaction number.	Units/s ² Unit/s Units/s ²
.NET <Edc object>.Move.Trig_A(DoPE.CTRL MoveCtrl, Double Acc, Double Speed, Double Dec, Double Limit, DoPE.CTRL TriggerCtrl, Double Trigger, ref Int16 Tan)			

5.3 Combined movement commands

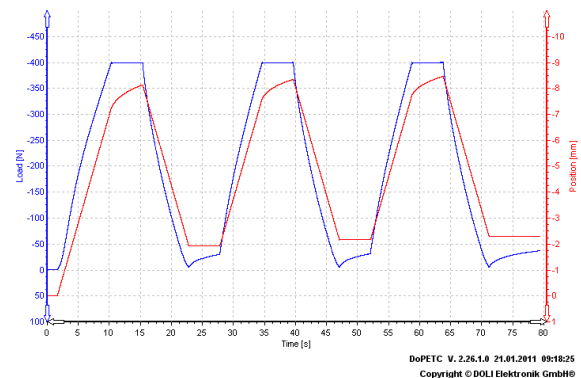
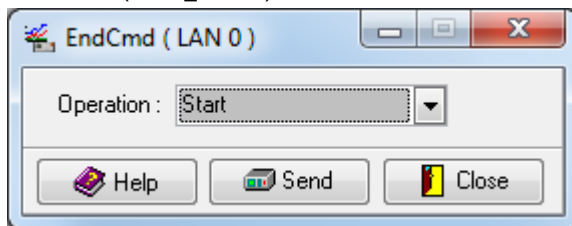
5.3.1 DoPEBlockHeader, DoPEBlockExecute

All simple movement commands can be used inside a combined movement command. The combined movement command starts with DoPEBlockHeader and ends with DoPEBlockExecute.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



```
DoPEPosExt(CTRL_POS, 0.833, LIMIT_RELATIVE, 10,
CTRL_LOAD, -400, DEST_POSITION);
DoPEHaltW(CTRL_LOAD, 5);
DoPEPosExt(CTRL_POS, 0.833, LIMIT_RELATIVE, 10,
CTRL_LOAD, -5, DEST_APPROACH)
DoPEHaltW(CTRL_POS, 5);
```



Repeat following sequence 3 times:

1. Go in position control with 8.33 mm/s (50 mm/min) to -400N and switch to load control.
2. Keep load for 5 seconds.
3. Go in position control with 8.33 mm/s (50 mm/min) to -5N and halt in position control.
4. Keep position for 5 seconds.

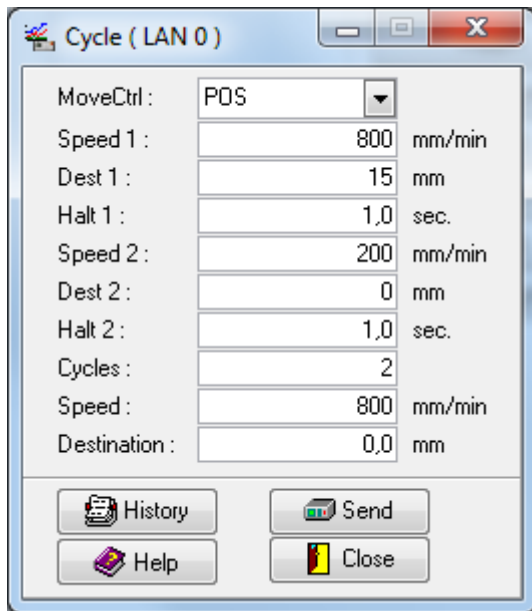
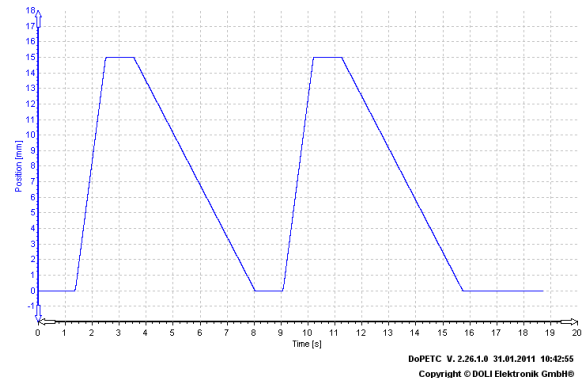
Function declaration	Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned long unsigned short	DoPEBlockHeader (DoPEHdl, Cycles, ModeFlags);	
	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Number of cycles to be done Flags. Both flags may be ignored. CMD_DWND : Supervise destination window. Next command is started after the current reaches its destination window. Otherwise the next command is started, after the command value reaches its destination CMD_MESSAGE : Report intermediate destinations. For each single command a message (OnTPos) is sent after it was finished. After the last command in the sequence is finished a message (OnPos) is always sent.	
.NET <Edc object>.Block. Header(Int32 Cycles, DoPE.CMD_	MODE ModeFlags)	
extern unsignedDLLAPI DoPE_HANDLE unsigned short unsigned short *	DoPEBlockExecute(DoPEHdl, Operation, lpusTAN);	
	Function returns Error constant (DoPERR_ xxxx) DoPE link handle CMD_START: Start the command sequence. CMD_DISCARD: Discard the command sequence. Pointer to transaction number.	
.NET <Edc object>.Block.Execute(DoPE.CMD_OPERATION	Operation, ref Int16 Tan)	

5.4 Complex movement commands

5.4.1 DoPECycle

Cycle movement using ramps.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Go with 800mm/min to 15mm, Halt 15mm for 1 second, go with 200mm/min to 0mm and halt 0mm for 1 second.

Repeat this sequence two times.

Function declaration		Description	Unit
extern unsigned DLLAPI	DoPECycle(	Function returns Error constant (DoPERR_XXXX)	
DoPE_HANDLE	DoPEHdl	DoPE link handle	
unsigned short	MoveCtrl	Control mode (CTRL_XXX)	
double	Speed1	Speed to reach destination 1	Unit/s
double	Dest1	Destination 1	Unit
double	Halt1	Halt time at destination 1	s
double	Speed2	Speed to reach destination 2	Unit/s
double	Dest2	Destination 2	Unit
double	Halt2	Halt time at destination 2	s
unsigned long	Cycles	Number of cycles	
double	Speed	Speed to final destination	Unit/s
double	Destination	Final destination	Unit
unsigned short *	lpusTAN	Pointer to transaction number.	

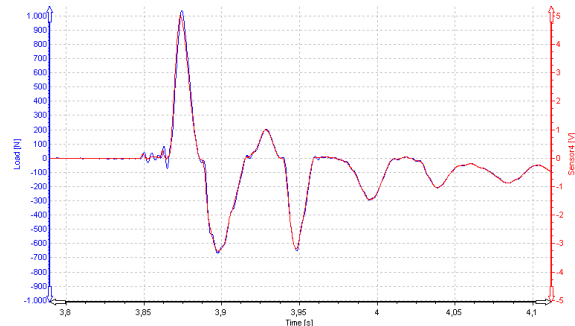
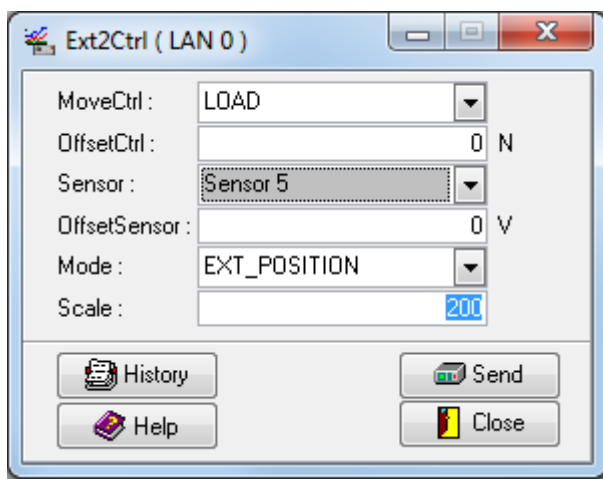
.NET <Edc object>.Move.Cycles(DoPE.CTRL MoveCtrl, Double Speed1, Double Dest1, Double Halt1, Double Speed2, Double Dest2, Double Halt2, Int32 Cycles, Double Speed, Double Destination, ref Int16 Tan)

5.4.2 DoPEExt2Ctrl

Move crosshead according to an external command signal. You may supply a random function to an A/D converter and specify movement according to this random function. Scaling of the command- and sensor-system is done by offsets and scale. Following formula is used:

$$\text{Output} = (\text{ExternalCommandValue} - \text{OffsetSensor}) * \text{Scale} + \text{OffsetCtrl}$$

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



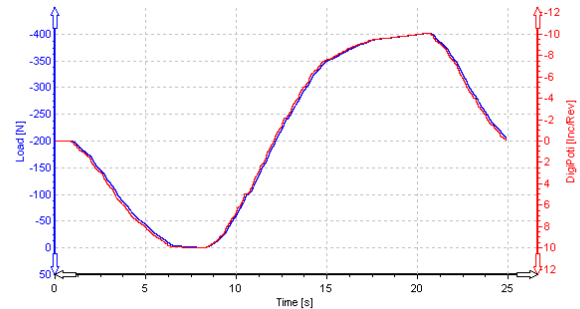
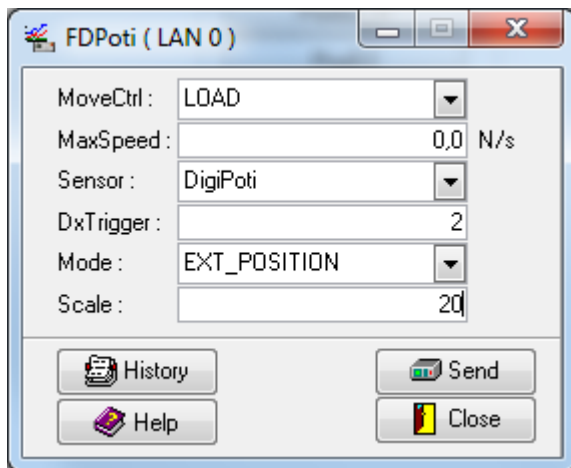
Follow external signal at Sensor4 in load control.
0V at Sensor4 will be 0N.
10V at Sensor4 will be 2000N.

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double unsigned short double unsigned short double	DoPEExt2Ctrl (DoPEHdl MoveCtrl OffsetCtrl SensorNo OffsetSensor Mode Scale	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_XXX) Offset for position, load or extension Sensor number for the external command signal Offset for external command signal various position or speed control modes (see next page) Scaling factor for external command signal Scale = 1 for POSITION mode: Nominal value of external command (e.g. 10V, or 1 round of a encoder) represents one unit of the control channel (e.g. 1mm, or 1N, or 1kN) Scale = 1 for SPEED mode: Nominal value of external command (e.g. 10V, or 1 round of a encoder) represents nominal speed of the control channel (e.g. 20mm/s, or 100kN/s) Scale = 1 for OPENLOOP mode: Nominal value of external command (e.g. 10V, or 1 round of a encoder) represents 100% output Pointer to transaction number.	Unit
unsigned short *	lpusTAN)		
.NET <Edc object>.Move.Ext2Ctrl(DoPE.CTRL MoveCtrl, Double OffsetCtrl, DoPE.SENSOR SensorNo, Double OffsetSensor, DoPE.EXT Mode, Double Scale, ref Int16 Tan)			

5.4.3 DoPEFDPoti

Move crosshead according to an external command signal generated by a digital encoder (DigiPoti). This is a special version of the DoPEExt2Ctrl command. Offsets and limits are handled inside this function.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



DoPETC V. 2.26.1.0 31.01.2011 10:27:48
Copyright © DOLI Elektronik GmbH®

Follow DigiPoti signal in load control.
One revolution of the DigiPoti will cause 20N in load.

Function declaration		Description	Unit
extern unsigned DLLAPI	DoPEFDPoti (	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) Start speed for speed controlled modes Sensor number for the external command input. Use sensor SENSOR_DP for Digital Encoder (DigiPoti) on EDC front panel. Dead area of encoder. The Encoder has to change the specified number of digits before the command is active. For EDC front panel DigiPoti 2 or 3 is a good value. various position or speed control modes (see below) For EXT_POSITION Number of Units per revolution e.g. Scale = 1 -> 1 mm per revolution Scale = 10 -> 10 N per revolution. For EXT_SPEED_xx number of revolutions to nominal speed e.g. Scale = 2 -> after 2 revolutions to nominal speed. Pointer to transaction number.	Unit/s
DoPE_HANDLE	DoPEHdl		
unsigned short	MoveCtrl		
double	MaxSpeed		
unsigned short	SensorNo		
unsigned short	DxTrigger		
unsigned short	Mode		
double	Scale		
unsigned short *	lpusTAN		

Various position or speed control modes:

The first four modes refer to measuring values. E.g. EXT_SPEED_POSITIVE moves to increasing measuring values.

EXT_POSITION	0	Position, moves to increasing and decreasing positions
EXT_SPEED_BIPOLAR	1	Speed bipolar (positive and negative speed)
EXT_SPEED_POSITIVE	2	Speed positive direction
EXT_SPEED_NEGATIVE	3	Speed negative direction

The next four modes refer to movement UP/DOWN. E.g. EXT_SPEED_UP moves Cross-head UP. The relation between measured values and movement Up/Down is defined in the set-up data.

EXT_POS_UP_DOWN	4	Position Up / Down
EXT_SPEED_UP_DOWN	5	Speed bipolar (Up and Down)
EXT_SPEED_UP	6	Speed UP
EXT_SPEED_DOWN	7	Speed DOWN
EXT_POS_OPENLOOP	8	Position mode for openloop configurations.

For use with DoPEFDPoti command we recommend only the following modes:

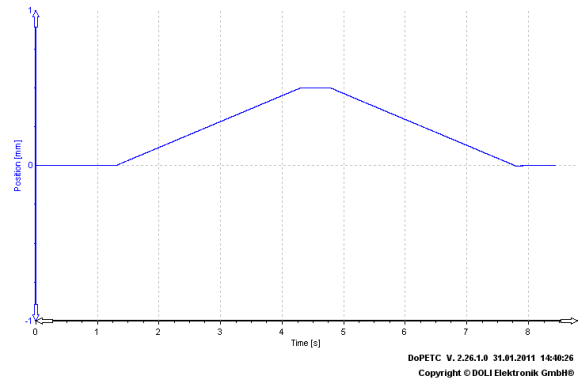
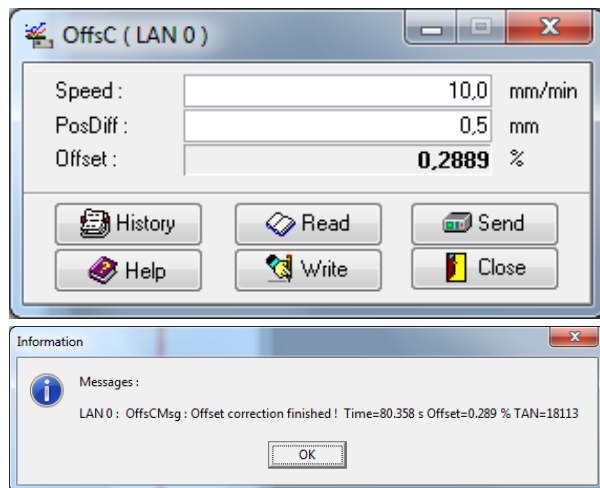
EXT_POS_UP_DOWN, EXT_SPEED_UP, EXT_SPEED_DOWN, EXT_POS_OPENLOOP

.NET <Edc object>.Move.FDPoti(DoPE.CTRL MoveCtrl, Double MaxSpeed, DoPE.SENSOR Sensor, Int16 DxTrigger, DoPE.EXT Mode, Double Scale, ref Int16 Tan)

5.4.4 DoPEOffsC

Special movement command to measure the offset of an external, analogue speed controller. This offset will be used for the speed output signal and compensates the offset of the external speed controller.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



Move 0.5 mm up and down and calculate the offset.
The offset is transmitted to PC (see DoPEOnOffsCMsgHdlr)

Function declaration		Description	Unit
extern unsigned DLLAPI	DoPEOffsC (	Function returns Error constant (DoPERR_XXXX)	Unit/s Unit
DoPE_HANDLE	DoPEHdl	DoPE link handle	
double	Speed	Speed	
double	PosDiff	Distance to move cross-head	
unsigned short *	lpusTAN	Pointer to transaction number.	
.NET <Edc object>.Move.OffsC(Double Speed, Double PosDiff, ref Int16 Tan)			

5.5 PC Command

With the PC Command a digital command signal can be used to move the crosshead.
PC Command is not supported by EDCi10

5.5.1 DoPEPcCmd

Move crosshead according to a digital command signal.
Scaling of the command- and sensor-system is done by offsets and scale. Following formula is used:
Output = (Value – Offset) * Scale + OffsetCtrl

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00
(not supported by EDCi10)

Function declaration	Description	Unit
extern unsigned DLLAPI DoPEPcCmd (DoPE_HANDLE DoPEHdl, unsigned short ExecutionMode, unsigned short MoveCtrl, unsigned short CmdMode, double SpeedToStart, double Offset, double Scale, double OffsetCtrl, double FadeInTime, double FadeOutTime, unsigned Cycles, unsigned Count, DoPEPcCmdData *Data, unsigned short * lpusTAN);	Function returns Error constant (DoPERR_ xxxx) DoPE link handle (see description below) Control mode for movement (Position, Load, Extension) Interpolation mode between the values DoPE_PC_CMD_MODE_POS: Command data are position values for each control loop cycle. DoPE_PC_CMD_MODE_POS_LINEAR: Command data are time and position. A linear function is interpolated between the position values DoPE_PC_CMD_MODE_POS_COSINUS: Command data are time and position. A cosine function is interpolated between the position values Speed to StartPosition Offset for stimulation signal Scaling factor for stimulation signal Offset for move control channel Fade in time Fade out time Number of cycles (ignored in append mode) Number of PcCmdData items in Data array Pointer to PcCmdData array In DoPE_PC_CMD_MODE_POS mode the time values will be ignored Pointer to transaction number.	Unit/s stimUnit Unit s s
typedef struct { double Time; double StimulationSignal; } DoPEPcCmdData;		s stimUnit
.NET <Edc object>.Ilc.PcCmd(DoPE.PC_CMD_EXECUTE ExecutionMode, DoPE.CTRL MoveCtrl, DoPE.PC_CMD_MODE CmdMode, Double SpeedToStart, Double Offset, Double Scale, Double OffsetCtrl, Double FadeInTime, Double FadeOutTime, Int32 Cycles, Int32 Count, DoPE.PcCmdData[] Data, ref Int16 Tan)		

DoPEPcCmd can be used in two different ways. A single call can include all information to perform the movement command. In this case ExecutionMode DoPE_PC_CMD_EXECUTE_LAST must be used and all parameters are considered. Appending data to the running movement command isn't possible.

If the Data can't be stored on the EDC the command will be discarded and the return code is set to DoPERR_PARAMETER.

In the second case ExecutionMode is set to DoPE_PC_CMD_EXECUTE_APPEND in the first call. The movement command starts immediately after the transmission of the data. Now any number of calls can follow to append data to the running movement command (DoPE_PC_CMD_EXECUTE_APPEND). With the last call ExecutionMode must be set to DoPE_PC_CMD_EXECUTE_LAST. No more data can be appended.

In the append mode buffer overruns and underruns can occur. At a buffer overrun PC Command data is provided to fast and can't be buffered in the EDC. In this case the data block will be discarded and the return code is set to DoPERR_PARAMETER.

At a buffer underrun PC Command data isn't provided fast enough. In this case the last point of the latest PC Command data block is used as stimulation signal until a new data block is available.

The number of buffer underruns can be retrieved with DoPERdPcCmdInfo.

Parameter	First call	Following calls	Last call
ExecutionMode	EXECUTE_APPEND	EXECUTE_APPEND	EXECUTE_LAST
MoveCtrl	valid	ignored	ignored
SpeedToStart	valid	ignored	ignored
CmdMode, Offset, Scale, OffsetCtrl, Count, Data	valid	valid	valid
FadeInTime	valid	ignored	ignored
FadeOutTime	ignored	ignored	valid
Cycles	ignored	ignored	valid
IpustAN	valid	ignored	ignored

To interrupt, continue or terminate a running movement command the ExecutionModes DoPE_PC_CMD_EXECUTE_PAUSE, DoPE_PC_CMD_EXECUTE_CONTINUE and DoPE_PC_CMD_EXECUTE_TERMINATE can be used.

ExecutionMode	EXECUTE_PAUSE	EXECUTE_CONTINUE	EXECUTE_TERMINATE
MoveCtrl	ignored	ignored	ignored
SpeedToStart	ignored	ignored	ignored
CmdMode, Offset, Scale, OffsetCtrl, Count, Data	ignored	ignored	ignored
FadeInTime	ignored	valid	ignored
FadeOutTime	valid	ignored	valid
Cycles	ignored	ignored	ignored
IpustAN	ignored	ignored	ignored

OffsetCtrl will be used as start/end point of the movement command if fade in/out will be used. Otherwise the first/last Data point will be used.

The Cycles count of the measuring data record is set to zero at the PcCmd call. Cycles count will be incremented with the execution of the first point of each PC Command data block.

For a detailed description of the SpeedToStart, Scale and Offset parameters and the fade times, please refer to the DoPEPcCmdFromFile documentation.

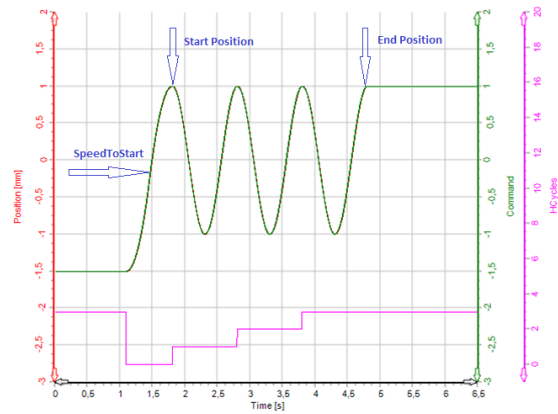
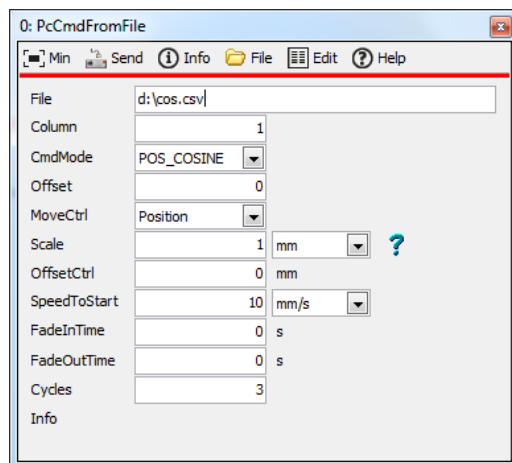
Move crosshead according to a digital command signal stored in a file.
DoPEPCmdFromFile is similar to DoPEPCmd except the stimulation data is passed thru a file and no data can be appended. So, all parameters will be considered.

$$\text{Output} = (\text{Value} - \text{Offset}) * \text{Scale} + \text{OffsetCtrl}$$

Function declaration	Description	Unit
extern unsigned DLLAPI DoPEPcCmdFromFile (DoPE_HANDLE DoPEHdl, unsigned short MoveCtrl, unsigned short CmdMode,	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode for movement (Position, Load, Extension) Interpolation mode between the values DoPE_PC_CMD_MODE_POS: Command data are position values for each control loop cycle. DoPE_PC_CMD_MODE_POS_LINEAR: Command data are time and position. A linear function is interpolated between the position values DoPE_PC_CMD_MODE_POS_COSINUS: Command data are time and position. A cosine function is interpolated between the position values	
double SpeedToStart,	Speed to StartPosition	Unit/s
double Offset,	Offset for stimulation signal	stimUnit
double Scale,	Scaling factor for stimulation signal	
double OffsetCtrl,	Offset for move control channel	Unit
double FadeInTime,	Fade in time	s
double FadeOutTime,	Fade out time	s
unsigned Cycles,	Number of cycles	
unsigned Column,	Column number of Stimulation Signal	
char *FileName,	In interpolation modes Time has always column number 0 Pointer to file name and path. Supported file types: csv "Comma Separated Values" text file: Value separator must be ',' Decimal point can be '.' or ',' Lines NOT starting with a number or a separator will be skipped bmv "DOLI Binary Measured Values" file: generated by DoPETestCenter version 2.27.1 (or above) In DoPE_PC_CMD_MODE_POS the time values will be ignored bmv2 "DOLI Binary Measured Values 2" file: generated by DoliInstallationCenter V10.13 (or above) In DoPE_PC_CMD_MODE_POS the time values will be ignored File access errors (e.g. wrong file extension) leads to an DoPERR_OPEN error Pointer to transaction number.	
unsigned short * lpusTAN);		
.NET	<Edc object>.llc.PcCmdFromFile(DoPE.CTRL MoveCtrl, DoPE.PC_CMD_MODE CmdMode, Double SpeedToStart, Double OffsetFile, Double Scale, Double OffsetCtrl, Double FadeInTime, Double FadeOutTime, Int32 Cycles, Int32 Column, String FileName, ref Int16 Tan)	

To illustrate the various parameters, we use a simple cos.csv file with cosinus interpolation and three cycles.

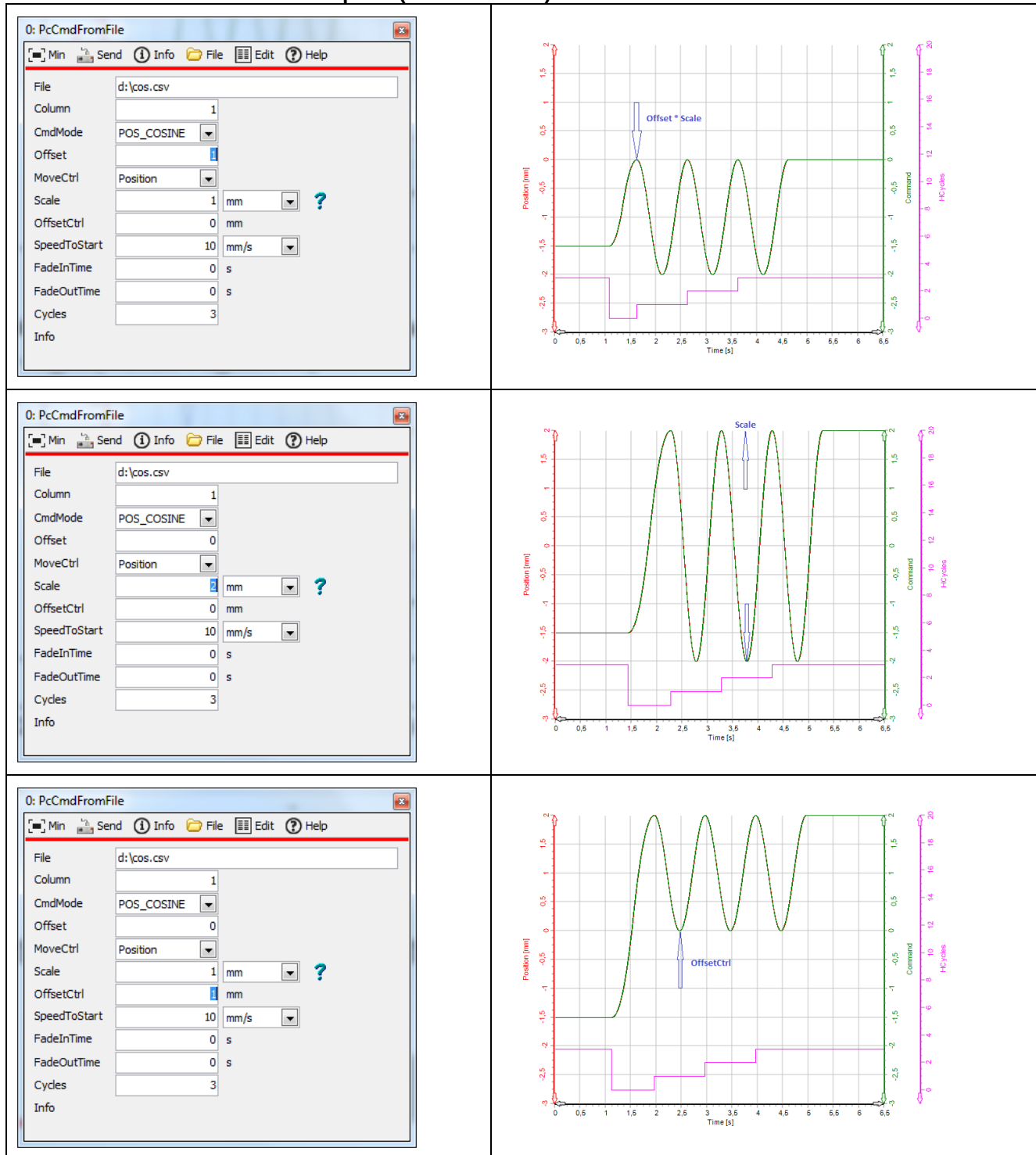
Time;	Position
Sec;	mm
0;	1
0.5;	-1
1.0;	1



5.5.2.1 Scale and Offsets

Scaling of the command- and sensor-system is done by offsets and scale. Following formula is used:

$$\text{Output} = (\text{Value} - \text{Offset}) * \text{Scale} + \text{OffsetCtrl}$$



To convert e.g. temperature from Fahrenheit to Celsius according the formula $^{\circ}\text{C} = (^{\circ}\text{F} - 30) / 2$ Offset should be 30 and Scale 0.5.

OffsetCtrl specifies the start/end position for fade in/out.

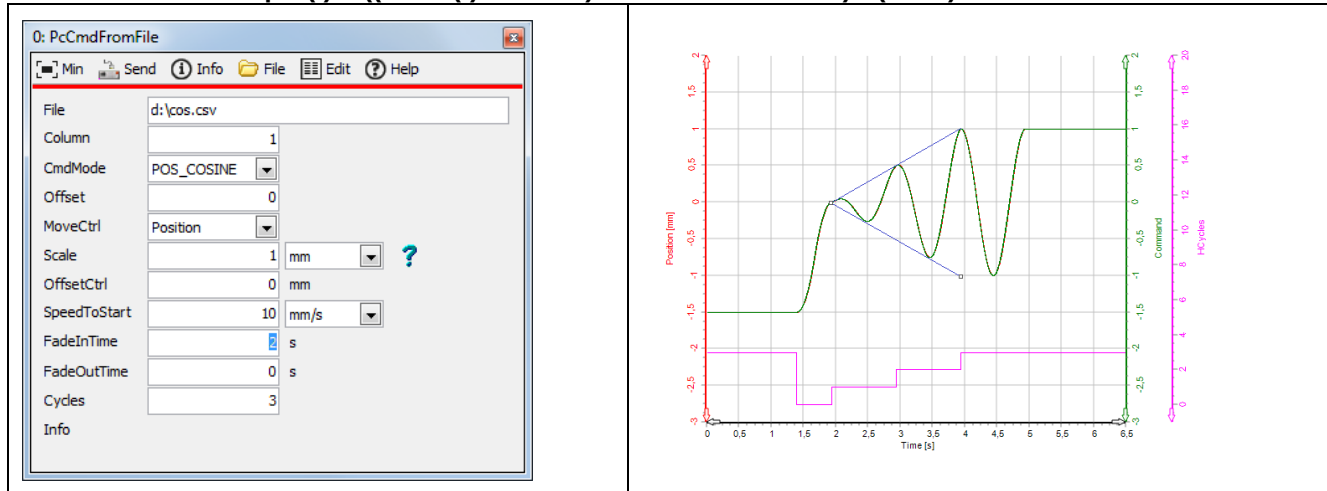
5.5.2.2 Start/End Position, SpeedToStart and Fade In/Out

If fade in/out time are zero the first point of the PcCmd data is used as the start/end position.

Otherwise OffsetCtrl is used as the start/end position.

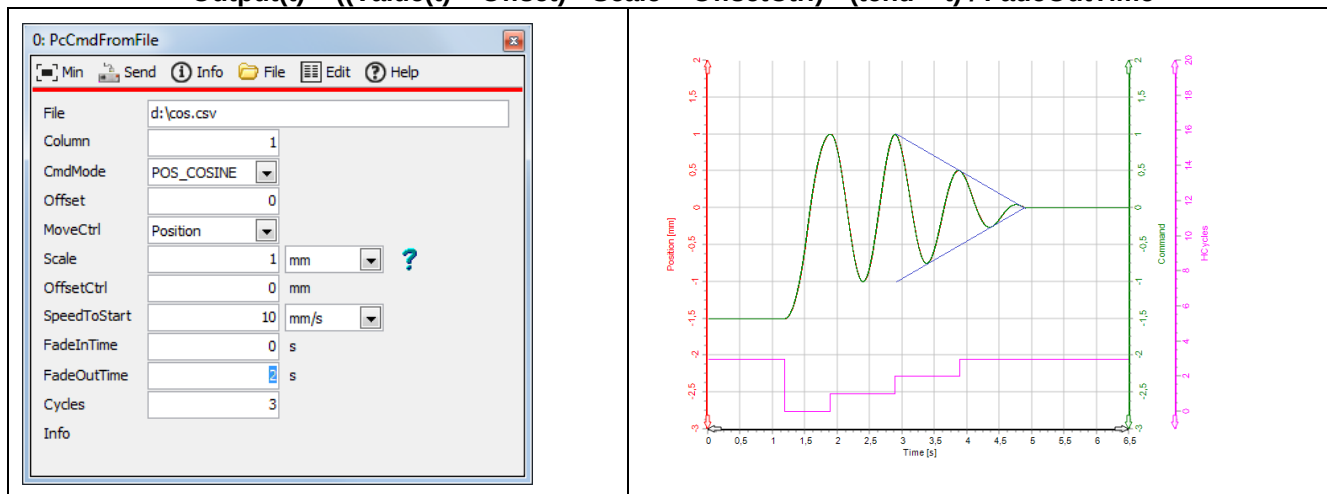
During the Fade In time the following formula is used:

$$\text{Output}(t) = ((\text{Value}(t) - \text{Offset}) * \text{Scale} + \text{OffsetCtrl}) * (t - t_0) / \text{FadeInTime}$$

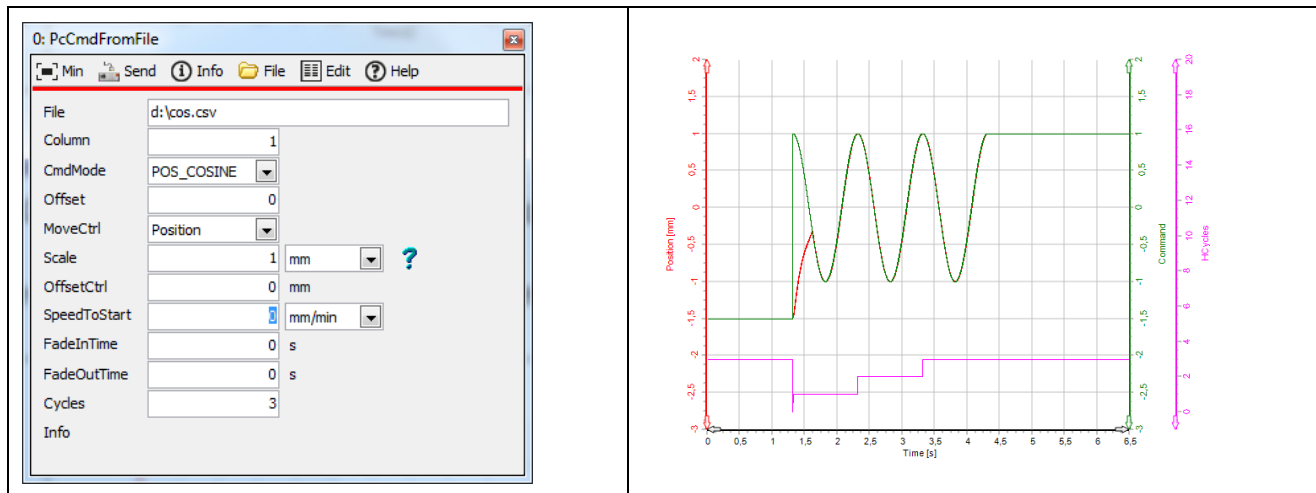


During the Fade Out time the following formula is used:

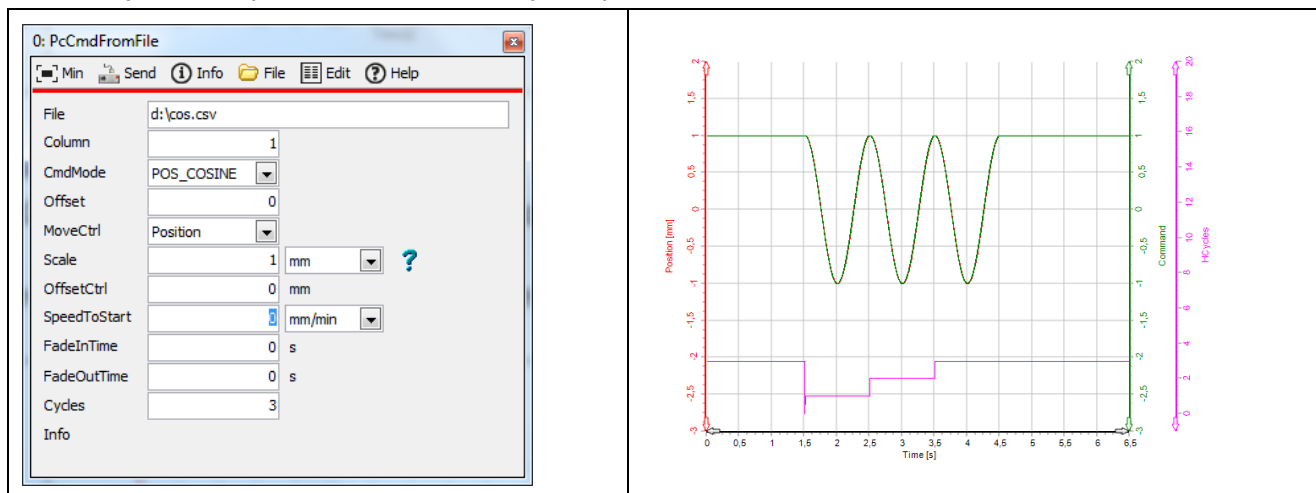
$$\text{Output}(t) = ((\text{Value}(t) - \text{Offset}) * \text{Scale} + \text{OffsetCtrl}) * (\text{tend} - t) / \text{FadeOutTime}$$



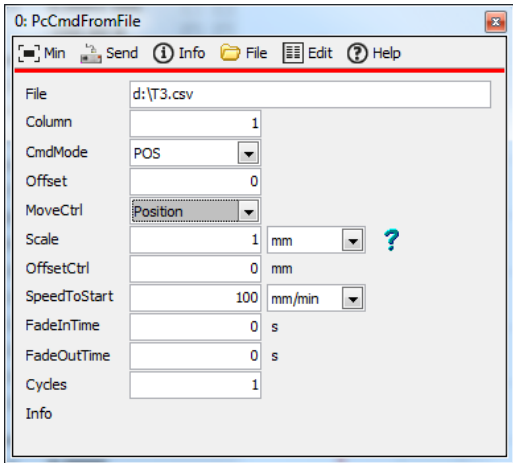
If **SpeedToStart** is set to zero the command of the closed loop controller 'jumps' to the start position within one closed loop controller cycle. This is a feature used for synchronized EDCs.

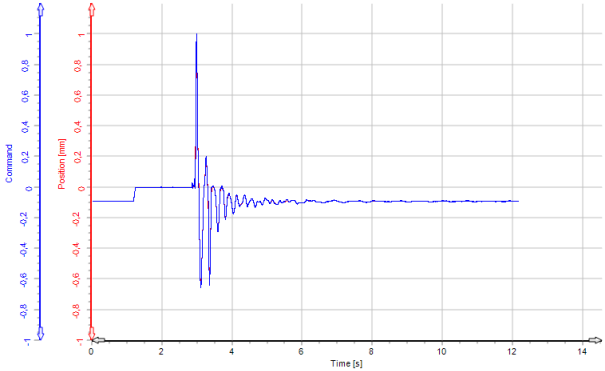


Obviously the start position should be set by a separate movement command.



5.5.2.3 Random Values





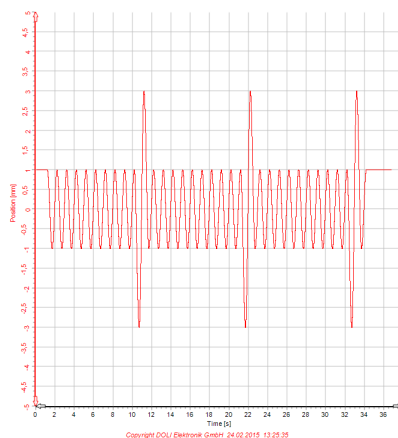
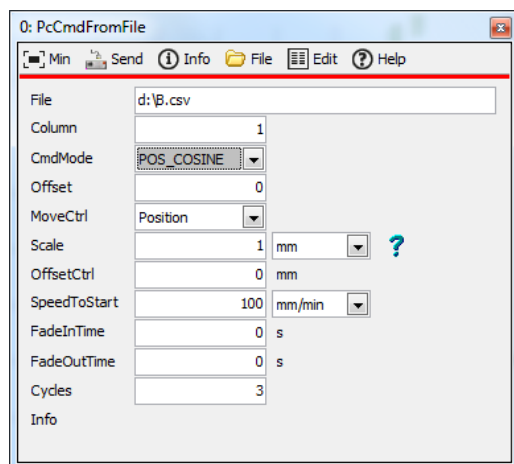
Copyright DOLI Elektronik GmbH 24.02.2015 15:11:33

Follow digital command from PC in position control. Don't use interpolation between the points in the file (ignore time column). Use scale 0.001 to convert μm to mm.

Sample of a csv file:

Time; Sec;	Position μm
3.819;	-0,8092685
3.8192;	-0,6743904
3.8194;	-0,6069514
3.8196;	-0,5395123
3.8198;	-0,6069514
3.820;	-0,6743904
3.8202;	-0,6743904
3.8204;	-0,7418295
3.8206;	-0,8767075
3.8208;	-1,011586
3.821;	-1,146464
3.8212;	-1,281342
3.8214;	-1,146464
3.8216;	-1,011586
3.8218;	-0,8767075
3.822;	-0,8092685
...	...

5.5.2.4 Cosine with one Cycle big Amplitude

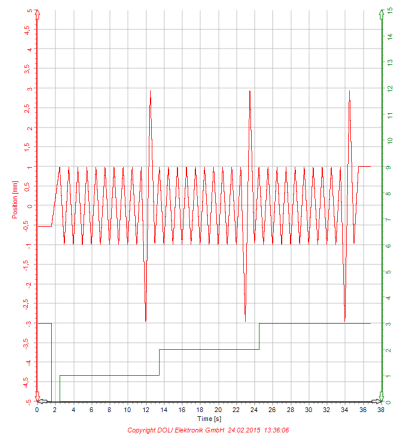
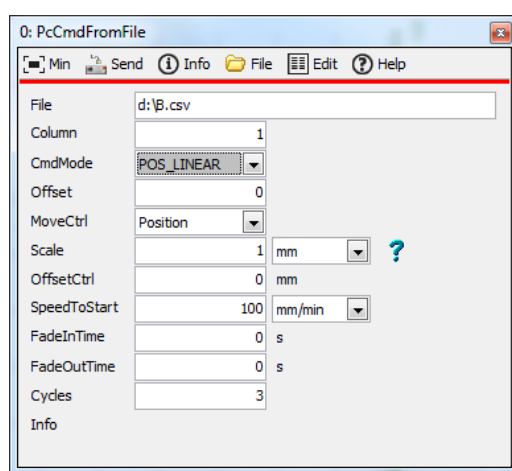


Follow digital command from PC in position control. Use a **cosine** interpolation between the points in the file.

Sample of a csv file:

Time;	Position
Sec;	mm
0;	1.0
0.5;	-1.0
1.0;	1.0
1.5;	-1.0
2.0;	1.0
2.5;	-1.0
3.0;	1.0
3.5;	-1.0
4.0;	1.0
4.5;	-1.0
5.0;	1.0
5.5;	-1.0
6.0;	1.0
6.5;	-1.0
7.0;	1.0
7.5;	-1.0
8.0;	1.0
8.5;	-1.0
9.0;	1.0
9.5;	-3.0
10.0;	3.0
10.5;	-1.0
11.0;	1.0

5.5.2.5 Triangle with one Cycle big Amplitude



Follow digital command from PC in position control. Use a **linear** interpolation between the points in the file.

At the beginning of the command, the HCycle counter is set to zero. At the beginning of each cycle, the counter is incremented.

Sample of a csv file:

Time;	Position
Sec;	mm
0;	1.0
0.5;	-1.0
1.0;	1.0
1.5;	-1.0
2.0;	1.0
2.5;	-1.0
3.0;	1.0
3.5;	-1.0
4.0;	1.0
4.5;	-1.0
5.0;	1.0
5.5;	-1.0
6.0;	1.0
6.5;	-1.0
7.0;	1.0
7.5;	-1.0
8.0;	1.0
8.5;	-1.0
9.0;	1.0
9.5;	-3.0
10.0;	3.0
10.5;	-1.0
11.0;	1.0

5.5.3 DoPERdPcCmdInfo

Read PC Command buffer and status information.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00
(not supported by EDCi10)

Function declaration	Description	Unit
extern unsigned DLLAPI DoPERdPcCmdInfo (DoPE_HANDLE DoPEHdl, DoPEPcCmdInfo *Info);	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer for DoPEPcCmdInfo structure.	
typedef struct { unsigned BuflItemsFree unsigned BuflItemsUsed; unsigned BufferUnderrun; double PosCtrlTime; unsigned Paused; } DoPEPcCmdInfo;	number of available PcCmdData items number of used PcCmdData items number of buffer underruns Position controller cycle time PC Command execution paused	s
.NET <Edc object>.Ilc.RdPcCmdInfo(ref DoPE.PcCmdInfo Info)		

5.5.4 DoPECalculatePcCmd

Calculate a digital command for each control loop cycle without execution.
If the OutData buffer size isn't sufficient the function returns DoPERR_NOMEM.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00
(not supported by EDCi10)

Function declaration	Description	Unit
extern unsigned DLLAPI DoPECalculatePcCmd (DoPE_HANDLE DoPEHdl, unsigned short ExecutionMode, unsigned short CmdMode, double Offset, double Scale, double OffsetCtrl, double FadeInTime, double FadeOutTime, unsigned Cycles, unsigned InCount, DoPEPcCmdData *InData, double SpeedT, unsigned OutCountMax, unsigned *OutCount, DoPEPcCmdData *OutData);	Function returns Error constant (DoPERR_XXXX) DoPE link handle (can be NULL for offline links) (see description at DoPEPcCmd, PAUSE, CONTINUE and TERMATE mode have no effect) Interpolation mode between the values DoPE_PC_CMD_MODE_POS: Command data are position values for each control loop cycle. DoPE_PC_CMD_MODE_POS_LINEAR: Command data are time and position. A linear function is interpolated between the position values DoPE_PC_CMD_MODE_POS_COSINUS: Command data are time and position. A cosine function is interpolated between the position values Offset for stimulation signal Scaling factor for stimulation signal Offset for move control channel Fade in time Fade out time Number of cycles (ignored in append mode) Number of PcCmdData items in InData array Pointer to PcCmdData InData array In DoPE_PC_CMD_MODE_POS mode the time values will be ignored Speed controller time (can be 0 to use the Speed Controller Time of an online connection) Maximum number of PcCmdData items in OutData array (typically (start time – end time) / SpeedT * Cycles + 1 , Can be more in append mode and due to numeric inaccuracy) Pointer to number of valid PcCmdData items in OutData array Pointer to PcCmdData OutData array	Unit/s stimUnit Unit s s
typedef struct { double Time; double Signal;		s Unit

	} DoPEPcCmdData;		
.NET	<Edc object>.Ilc.CalculatePcCmd(DoPE.PC_CMD_EXECUTE ExecutionMode, DoPE.PC_CMD_MODE CmdMode, Double Offset, Double Scale, Double OffsetCtrl, Double FadeInTime, Double FadeOutTime, Int32 Cycles, Int32 InCount, DoPE.PcCmdData[] InData, Double SystemTime, Int32 OutCountMax, ref Int32 OutCount, ref DoPE.PcCmdData[] OutData)		

5.5.5 DoPECalculatePcCmdFromFile

Calculate a digital command stored in a file for each control loop cycle without execution. The command creates a new csv output file. The size of this file is limited to 2 GB. If the limit is reached or temporarily allocated memory isn't sufficient the function returns DoPERR NOMEM.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00 (not supported by EDCi10)

Function declaration	Description	Unit
extern unsigned DLLAPI DoPECalculatePcCmdFromFile (DoPE_HANDLE DoPEHdl, unsigned short CmdMode,	Function returns Error constant (DoPERR_xxxx) DoPE link handle (can be NULL for offline links) Interpolation mode between the values DoPE_PC_CMD_MODE_POS: Command data are position values for each control loop cycle. DoPE_PC_CMD_MODE_POS_LINEAR: Command data are time and position. A linear function is interpolated between the position values DoPE_PC_CMD_MODE_POS_COSINUS: Command data are time and position. A cosine function is interpolated between the position values	
double Offset,	Offset for stimulation signal	Unit/s
double Scale,	Scaling factor for stimulation signal	stimUnit
double OffsetCtrl,	Offset for move control channel	
double FadeInTime,	Fade in time	
double FadeOutTime,	Fade out time	Unit
unsigned Cycles,	Number of cycles	s
unsigned Column,	Column number of Stimulation Signal In interpolation modes Time has always column number 0	s
char *InFileName,	Pointer to input file name and path. Supported file types: csv "Comma Separated Values" text file: Value separator must be ',' Decimal point can be '.' or ',' Lines NOT starting with a number or a separator will be skipped bmv "DOLI Binary Measured Values" file: generated by DoPETestCenter version 2.27.1 (or above) In DoPE_PC_CMD_MODE_POS the time values will be ignored bmv2 "DOLI Binary Measured Values 2" file: generated by DoliInstallationCenter V10.13 (or above) In DoPE_PC_CMD_MODE_POS the time values will be ignored	
double SpeedT,	File access errors (e.g. wrong file extension) leads to an DoPERR_OPEN error Speed controller time (can be 0 to use the current Speed Controller Time of an online connection)	
char *OutFileName);	Pointer to output file name and path. Each line contains the time and command value separated by ';' with '.' as decimal point. E.g: 0.0708;-9.62E-04	
.NET	<Edc object>.Ilc.CalculatePcCmdFromFile(DoPE.PC_CMD_MODE CmdMode, Double OffsetFile, Double Scale, Double OffsetCtrl, Double FadeInTime, Double FadeOutTime, Int32 Cycles, Int32 Column, String InFileName, Double SystemTime, String OutFileName)	

5.6 DoPEDynCycles

Generates the most common waveforms like **Cosine, Triangle, Rectangle, Saw tooth, inverted Saw tooth, Pulse and Sine.**

Additional functions can be activated like:

Peak Control, to correct amplitude errors and increase control fidelity.

Linear or logarithmic **Sweeps** for frequency, offset, amplitude or a combination of sweeps.

All waveforms can be **superimposed** with a cosine wave.

Bi-Modal modes, to perform a test in e.g. position control and keep load peak/mean values constant.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Waveform Sine requires EDCiApp 9149.026 and DoPE 10.22 or higher.

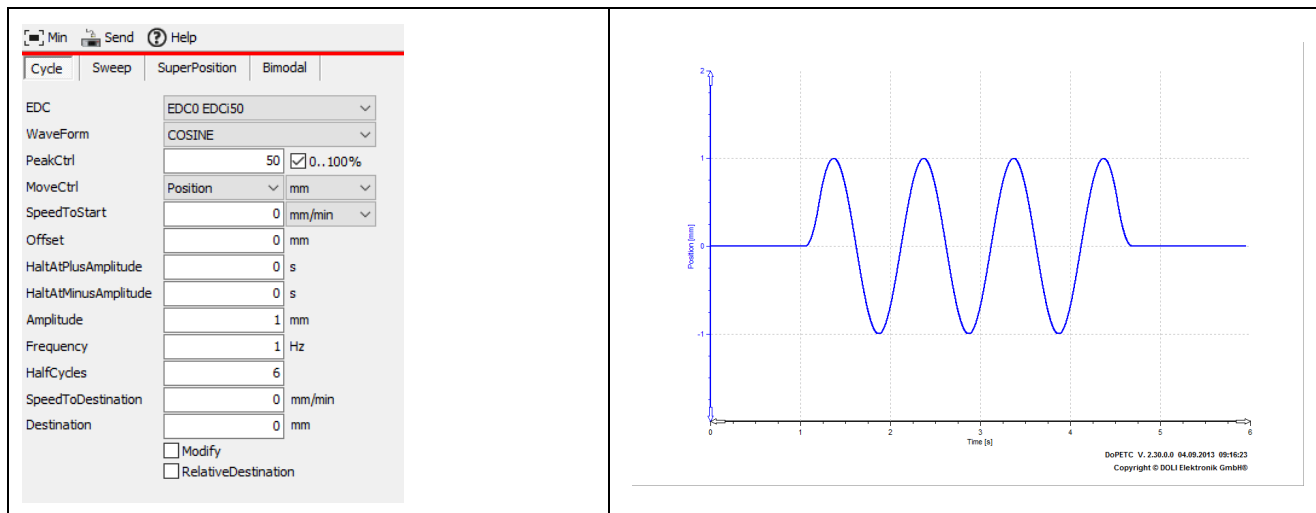
(Modify, Sweeps, Superposition and Bimodal Modes are not supported by EDCi10)

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short unsigned short	DoPEDynCycles (DoPEHdl WaveForm Modify PeakCtrl	Function returns Error constant (DoPERR_xxxx) DoPE link handle DYN_WAVEFORM_xxx (Cosine, Triangular,...) Modify Parameter of active cycles Peak Control Gain 0-100% if DYN_PEAK_CTRL_PERCENT is added. Or (0..16) see 5.6.4 Peak Control	
unsigned short unsigned short	MoveCtrl RelativeDestination	Control mode (CTRL_xxx) false, Offset, and Destination absolute true, Offset, and Destination relative	Unit/s
double double double double double double double double	SpeedToStart Offset Amplitude HaltAtPlusAmplitude HaltAtMinusAmplitude Frequency HalfCycles SpeedToDestination Destination	Speed to Offset + Amplitude Offset Amplitude Halt Time at Offset + Amplitude Halt Time at Offset – Amplitude Frequency Number of half cycles Speed to final destination (0=automatic speed calculation) Final destination	Unit Unit s s 1/s Unit/s Unit
unsigned short double double unsigned long	SweepFrequencyMode SweepEndFrequency SweepFrequencyTime SweepFrequencyCount	DYN_SWEEP_xxx (Off, Linear, Logarithmic,...) End frequency for frequency sweep Time for frequency sweep Frequency sweep counter (0=infinite)	1/s s
unsigned short double double unsigned long	SweepOffsetMode SweepEndOffset SweepOffsetTime SweepOffsetCount	DYN_SWEEP_xxx (Off, Linear, Logarithmic,...) Second offset for offset sweep Time for offset sweep Offset sweep counter (0=infinite)	Unit s
unsigned short double double unsigned long	SweepAmplitudeMode SweepEndAmplitude SweepAmplitudeTime SweepAmplitudeCount	DYN_SWEEP_xxx (Off, Linear, Logarithmic,...) Second offset for offset sweep Time for amplitude sweep Amplitude sweep counter (0=infinite)	Unit s Unit
unsigned short double double	SuperpositionMode SuperpositionFrequency SuperpositionAmplitude	Superposition Mode (DYN_SUPERPOS_ON / _OFF) Superposition Frequency Superposition Amplitude	1/s Unit
unsigned short unsigned short double double double	BimodalCtrlMode BimodalCtrlSensor BimodalValue1 BimodalValue2 BimodalScale	Mode for bimodal control Sensor for bimodal control Value1 to keep constant Value2 only for DYN_BIMODAL_AMPLITUDE_OFFSET_VAR Scale	UnitBi UnitBi Unit/UnitBi
unsigned short *	lpusTAN	Pointer to transaction number.	

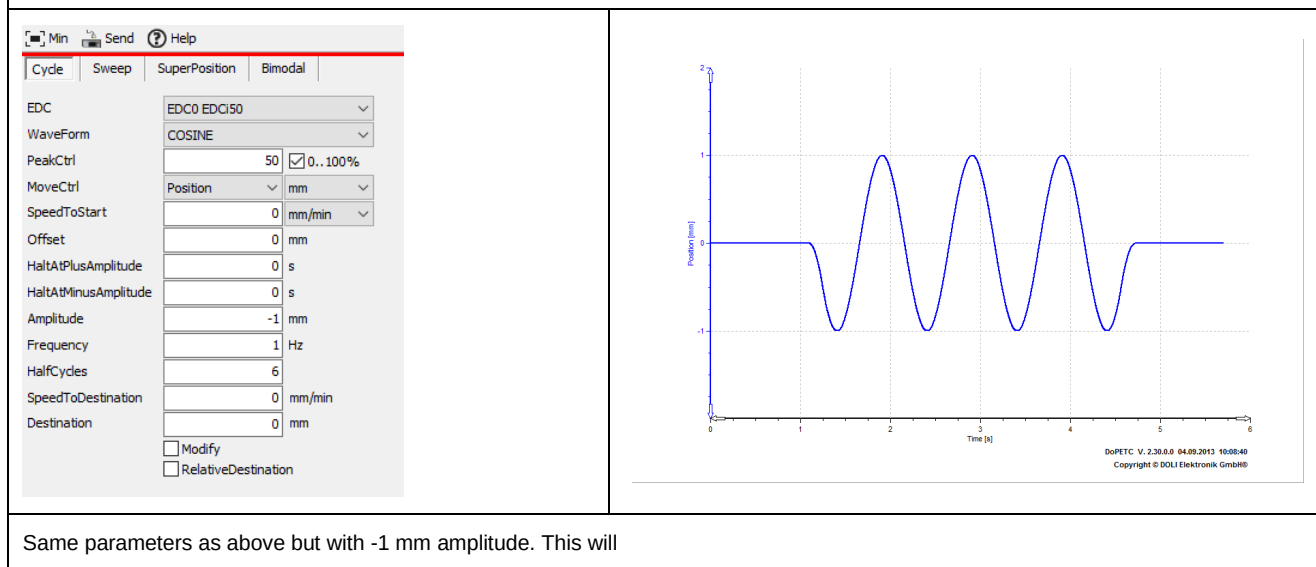
Constants for DoPEDynCycles: Constants for waveform: DYN_WAVEFORM_COSINE DYN_WAVEFORM_TRIANGLE DYN_WAVEFORM_RECTANGLE DYN_WAVEFORM_SAW_TOOTH DYN_WAVEFORM_SAW_TOOTH_INV DYN_WAVEFORM_PULSE DYN_WAVEFORM_SINE Constants for PeakCtrl: DYN_PEAK_CTRL_PERCENT Constants for SweepMode: DYN_SWEEP_OFF DYN_SWEEP_LINEAR DYN_SWEEP_LINEAR_START_END DYN_SWEEP_LOGARITHMIC DYN_SWEEP_LOGARITHMIC_START_END Constants for SuperpositionMode: DYN_SUPERPOS_OFF DYN_SUPERPOS_ON Constants for BimodalCtrlMode: DYN_BIMODAL_CTRL_OFF DYN_BIMODAL_ARITHMETIC_MEAN_VALUE DYN_BIMODAL_MEAN_VALUE DYN_BIMODAL_MIN_VALUE DYN_BIMODAL_MAX_VALUE DYN_BIMODAL_AMPLITUDE_OFFSET_VAR DYN_BIMODAL_AMPLITUDE_OFFSET_CONST	Cosine waveform Triangle waveform Rectangular waveform Sawtooth waveform Inverse sawtooth waveform Pulse waveform Sine waveform Control bit to set peak control value in percent No sweep active Linear sweep from e.g Frequency to SweepEndFrequency Linear sweep at Start and End of cycles Logarithmic sweep Logarithmic sweep at Start and End of cycles No superposition active Superposition active No bimodal control active Keep arithmetic mean value of second sensor constant by changing the offset of the controlled sensor Keep the mean value of second sensor constant by changing the offset of the controlled sensor Keep the minimum value of second sensor constant by changing the offset of the controlled sensor Keep the maximum value of second sensor constant by changing the offset of the controlled sensor Keep the amplitude of second sensor constant by changing the offset and amplitude of the controlled sensor Keep the amplitude of second sensor constant by changing the amplitude of the controlled sensor. The offset of the controlled sensor is constant	
.NET <Edc object>.Move.DynCycles(DoPE.DYN_WAVEFORM WaveForm, bool Modify, DoPE.DYN_PEAKCTRL PeakCtrl, DoPE.CTRL MoveCtrl, bool RelativeDestination, Double SpeedToStart, Double Offset, Double Amplitude, Double HaltAtPlusAmplitude, Double HaltAtMinusAmplitude, Double Frequency, Int32 HalfCycles, Double SpeedToDestination, Double Destination, DoPE.DYN_SWEEP SweepFrequencyMode, Double SweepEndFrequency, Double SweepFrequencyTime, Int32 SweepFrequencyCount, DoPE.DYN_SWEEP SweepOffsetMode, Double SweepEndOffset, Double SweepOffsetTime, Int32 SweepOffsetCount, DoPE.DYN_SWEEP SweepAmplitudeMode, Double SweepEndAmplitude, Double SweepAmplitudeTime, Int32 SweepAmplitudeCount, DoPE.DYN_SUPERPOS SuperpositionMode, Double SuperpositionFrequency, Double SuperpositionAmplitude, DoPE.DYN_BIMODAL BimodalCtrlMode, DoPE.SENSOR BimodalCtrlSensor, Double BimodalValue1, Double BimodalValue2, Double BimodalScale, ref Int16 Tan)		

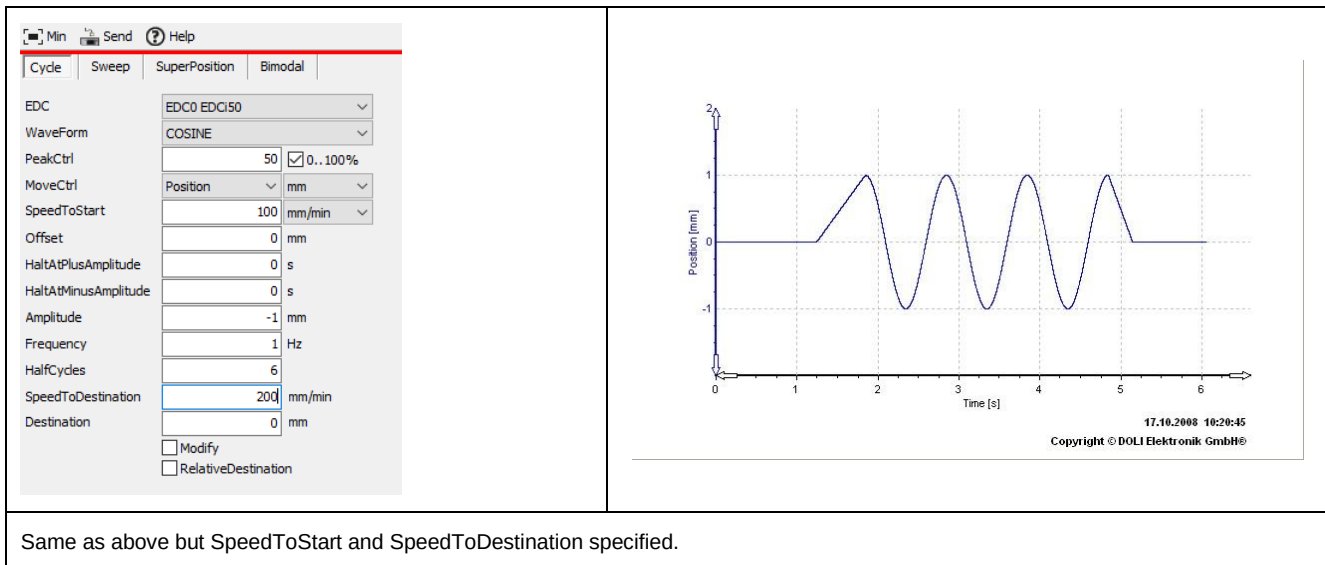
5.6.1 Basic Waveforms

5.6.1.1 Cosine/Sine

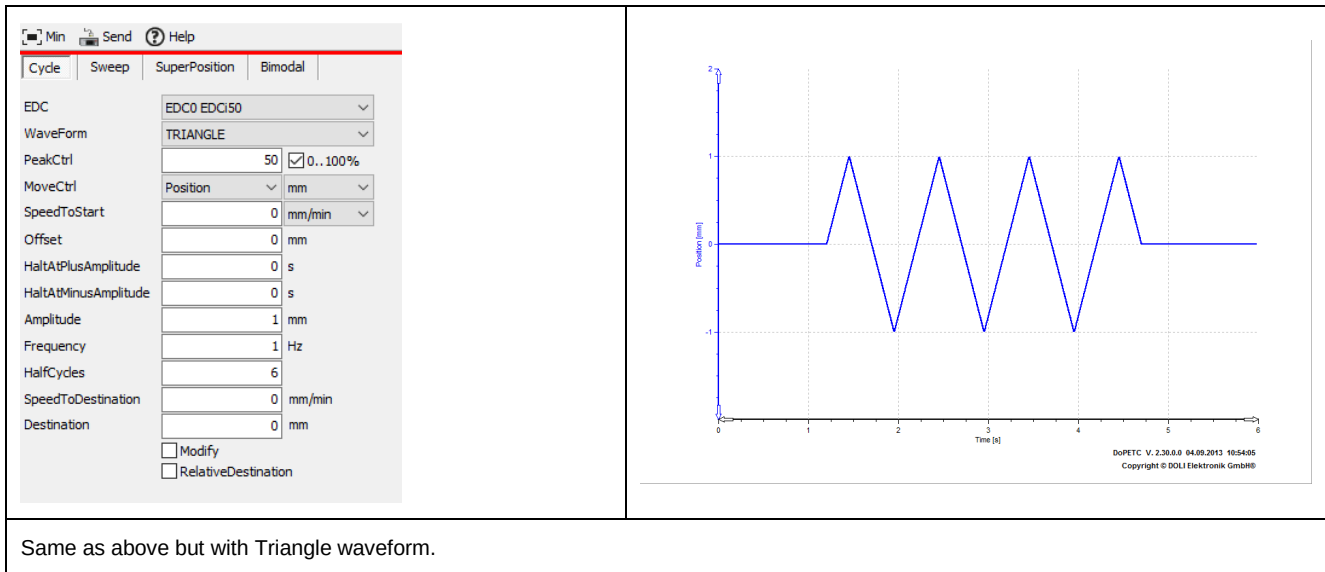


Cosine waveform with 0 mm offset, 1mm amplitude, frequency 1Hz and 6 half-cycles (3 cosine periods). SpeedToStart and SpeedToDestination has been set to 0 mm/min so the ramp to the starting point (1mm) and destination point (0mm) will be calculated automatically.

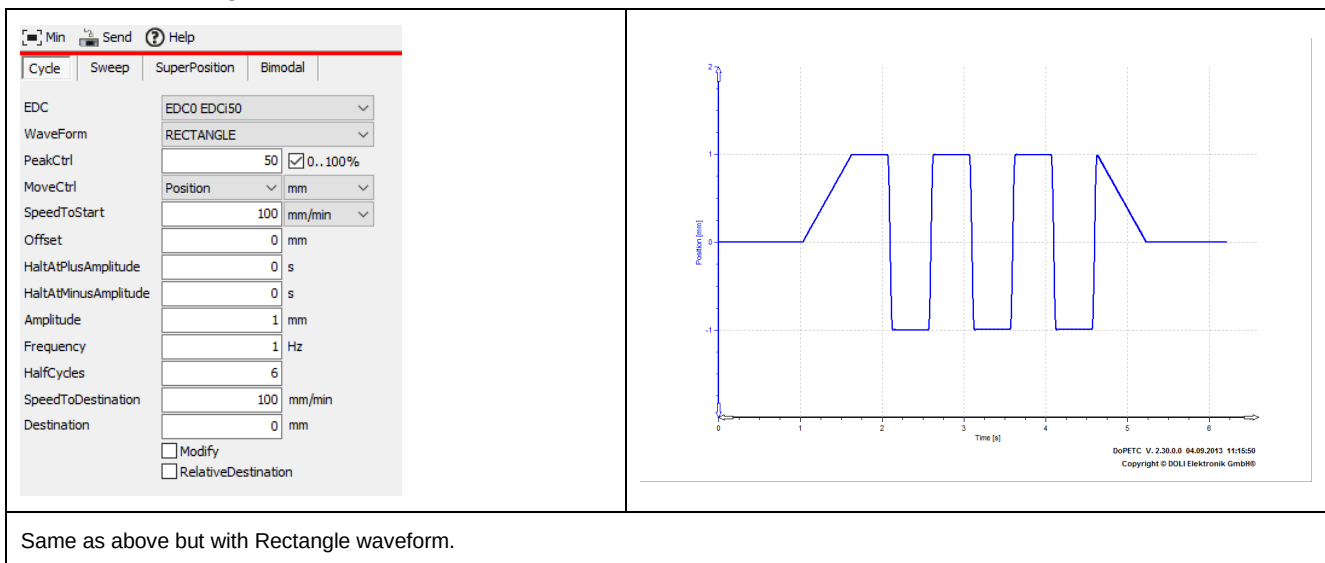




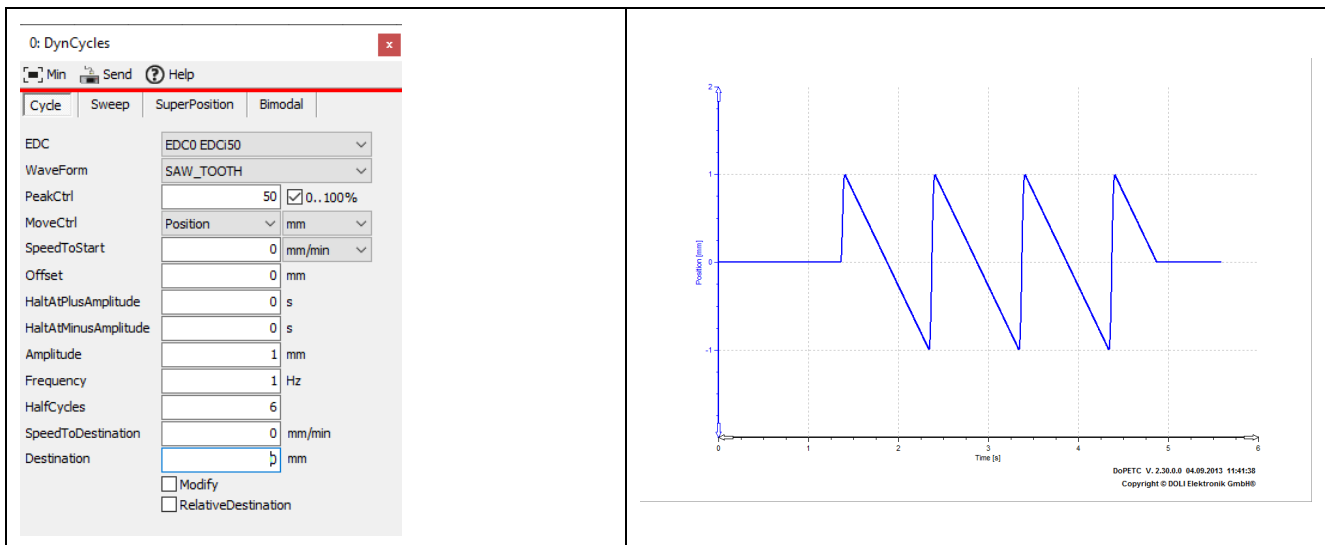
5.6.1.2 Triangle



5.6.1.3 Rectangle

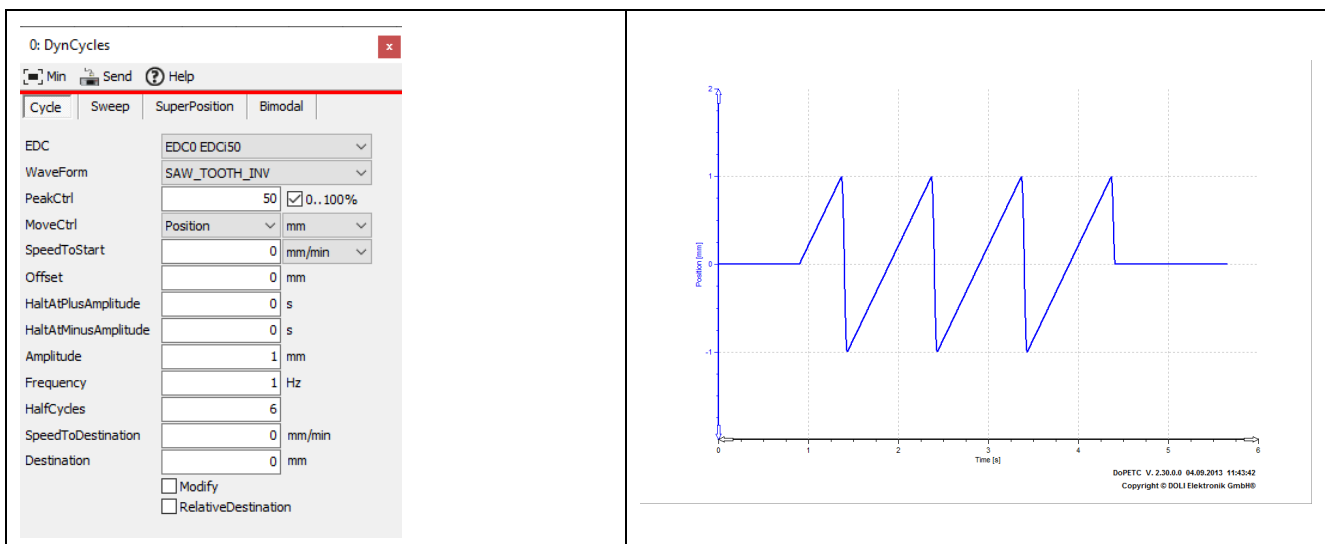


5.6.1.4 Sawtooth



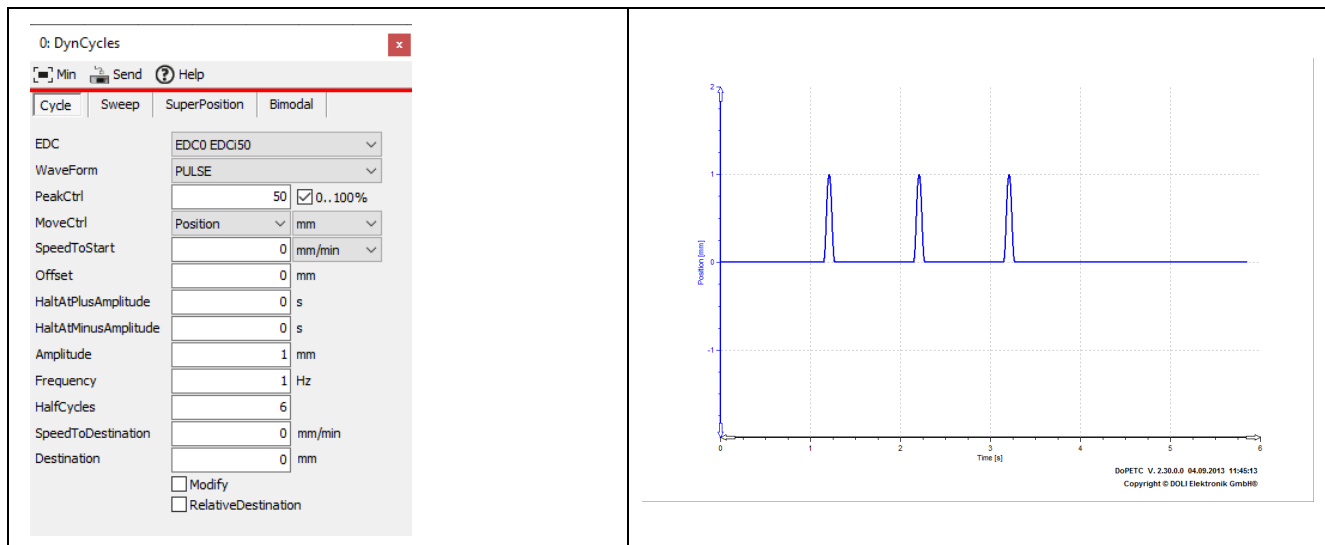
Same as above but with Sawtooth waveform.

5.6.1.5 Sawtooth inverse



Same as above but with an inverted Sawtooth waveform.

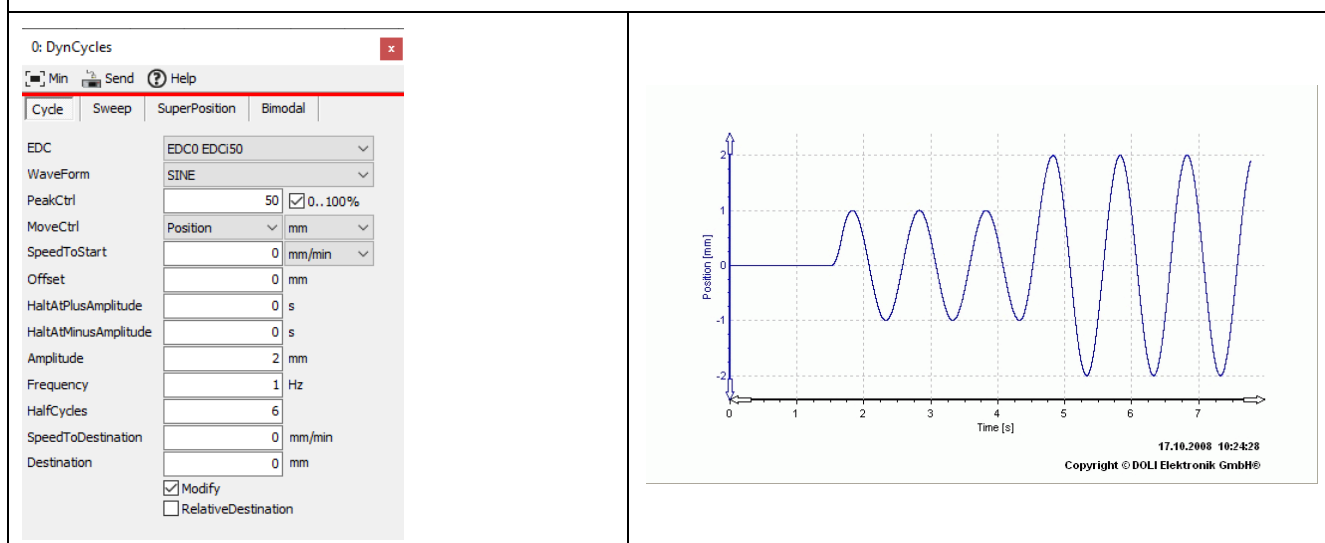
5.6.1.6 Pulse



Same as above but with Pulse waveform.

5.6.2 Modify Parameter of an active test

The Modify Parameter is not supported by EDCi10

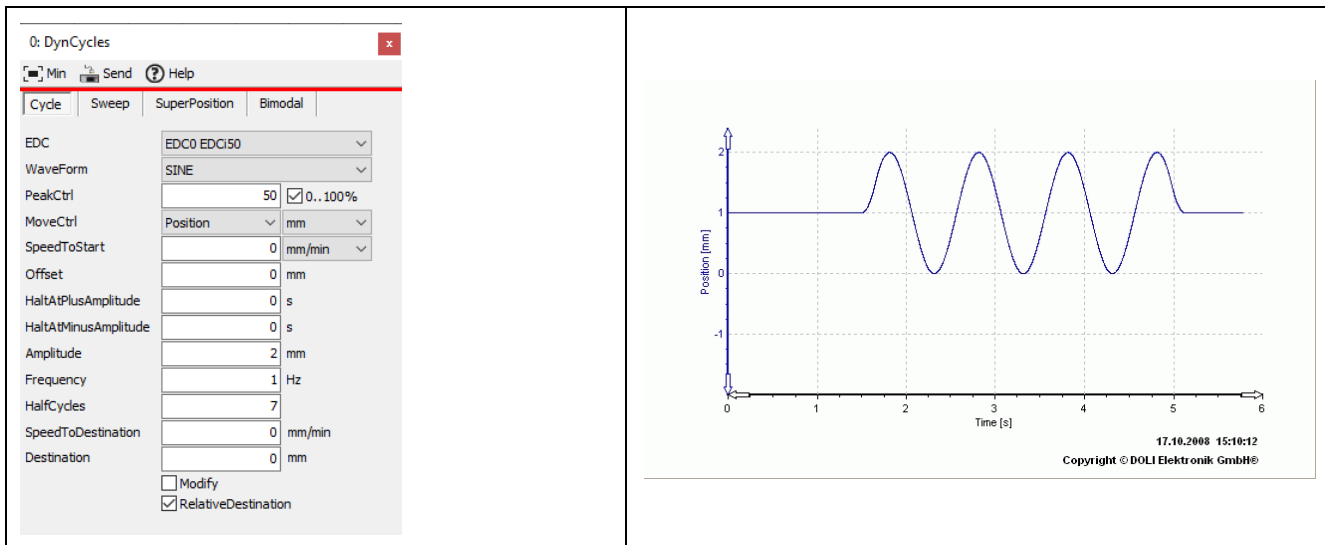


Modify amplitude of an active Sine.

Sine was started with 1mm amplitude. After some cycles, DynCycle command was sent again, with the Modify flag set and amplitude of 2mm.

Besides amplitude the offset and frequency may also be modified.

5.6.3 Relative Destinations



Sine command with relative Destinations.

Sine was started at a position of 1mm. This current position will be added to the offset, and Destination.

5.6.4 Peak Control

Peak Control can be activated for all waveforms by choosing any value for PeakCtrl. The value can be set in two different ways.

Control bit in PeakCtrl is set to 1 (DYN_PEAK_CTRL_PERCENT 0x8000) (EDCiApp 9149.027 or higher):

PeakCtrl can be set between 0% and 100%. The peaks will be corrected with the percentage of the detected error at every maximum and minimum.

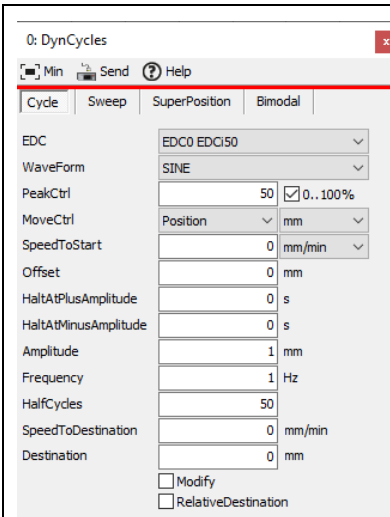
Control bit in PeakCtrl is set to 0:

PeakCtrl = 0 peak control is deactivated

PeakCtrl = 1 peak control is activated. Controls the peak value every cycle using 50% of the error

PeakCtrl = 2,4,8,16

- EDCiApp 9149.026 or higher :
Controls the peak value every cycle using the average error of 1,2,4,8,16 cycles.
- EDCiApp 9149.025 or lower:
Controls the error every 1st,2nd,4th,8th,16th peak and valley points, using the average error of 1,2,4,8 or 16 cycles.



0: DynCycles

Min Send Help

Cycle Sweep SuperPosition Bimodal

EDC: EDC0 EDCi50

WaveForm: SINE

PeakCtrl: 50 ☒ 0..100%

MoveCtrl: Position mm

SpeedToStart: 0 mm/min

Offset: 0 mm

HaltAtPlusAmplitude: 0 s

HaltAtMinusAmplitude: 0 s

Amplitude: 1 mm

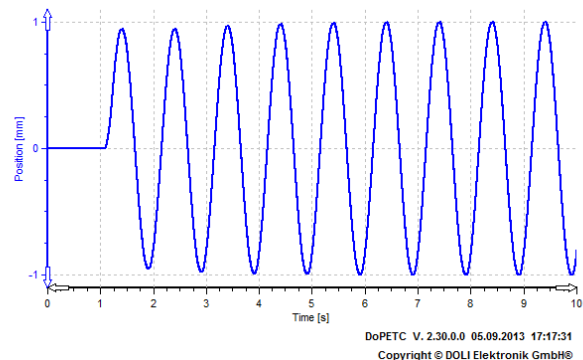
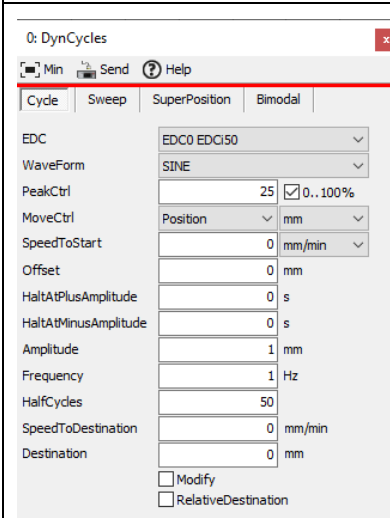
Frequency: 1 Hz

HalfCycles: 50

SpeedToDestination: 0 mm/min

Destination: 0 mm

☐ Modify ☐ RelativeDestination

0: DynCycles

Min Send Help

Cycle Sweep SuperPosition Bimodal

EDC: EDC0 EDCi50

WaveForm: SINE

PeakCtrl: 25 ☒ 0..100%

MoveCtrl: Position mm

SpeedToStart: 0 mm/min

Offset: 0 mm

HaltAtPlusAmplitude: 0 s

HaltAtMinusAmplitude: 0 s

Amplitude: 1 mm

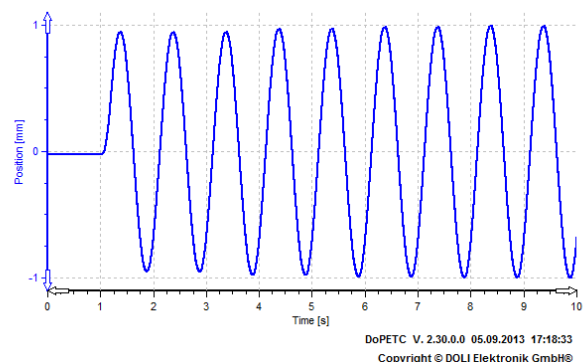
Frequency: 1 Hz

HalfCycles: 50

SpeedToDestination: 0 mm/min

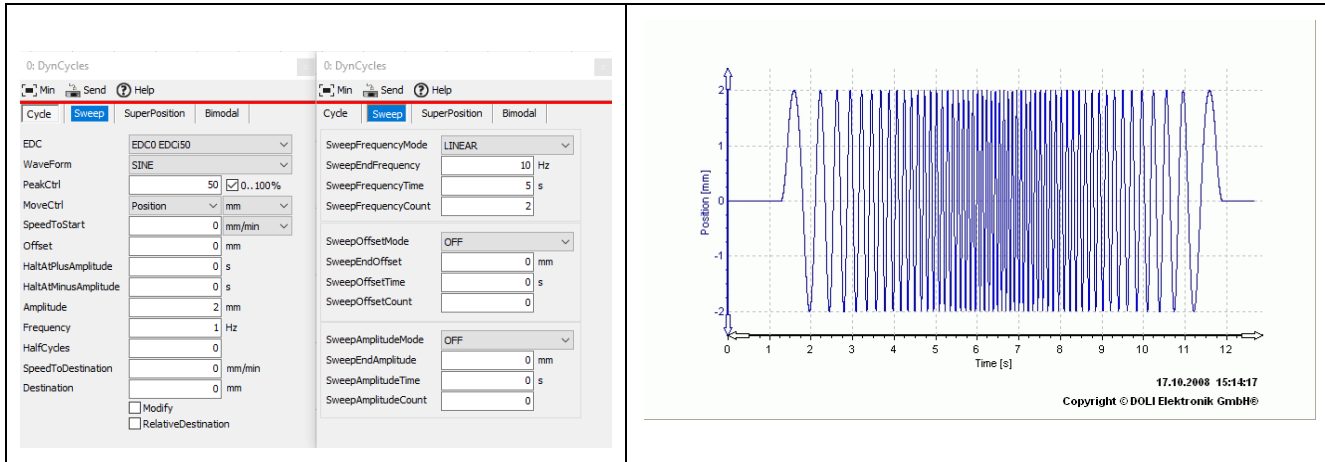
Destination: 0 mm

☐ Modify ☐ RelativeDestination



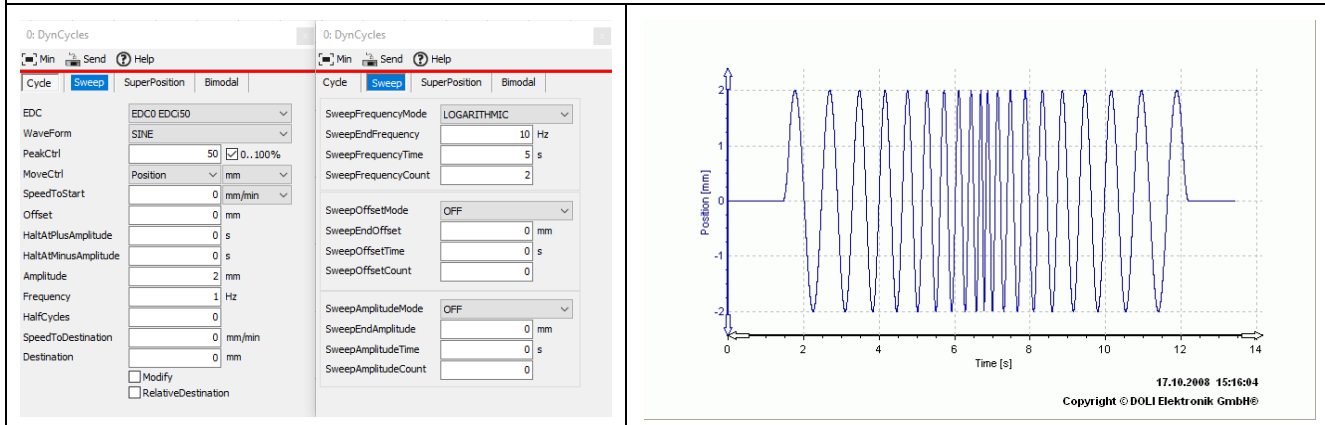
5.6.5 Sweeps

Sweeps are not supported by EDCi10



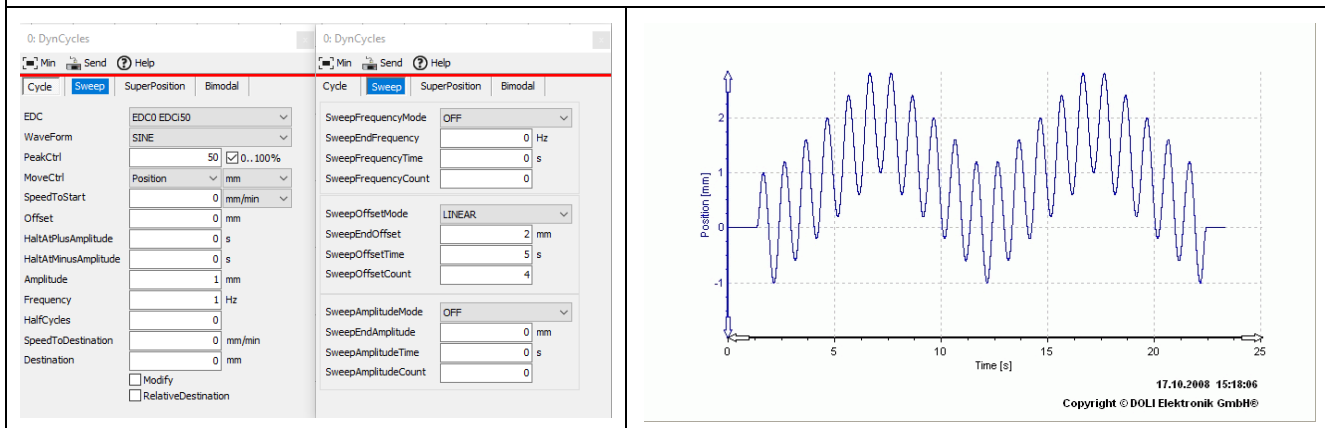
Sine command with a linear frequency sweep.

Sine starts with a frequency of 1Hz. The frequency will be continuously increased to 10Hz within 5 seconds. The parameter SweepFrequencyCount specifies how many sweeps should be done. In this case go up to 10Hz, and back to 1Hz.



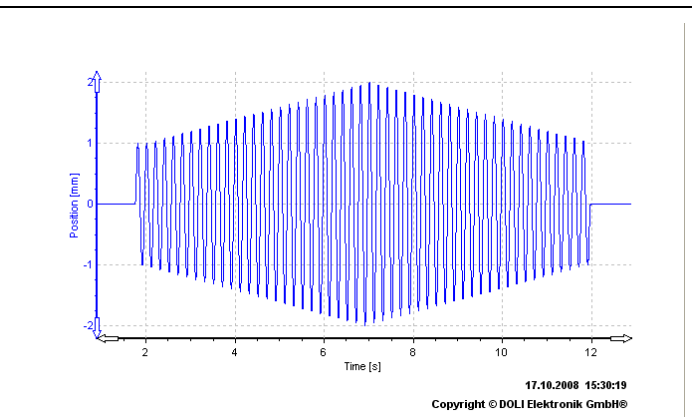
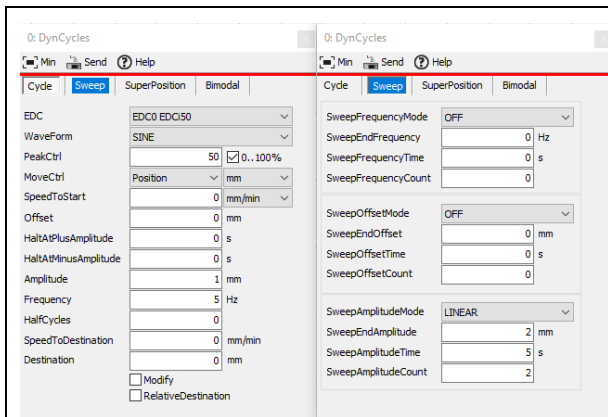
Sine command with a logarithmic frequency sweep.

Sine starts with a frequency of 1Hz. The frequency will be exponentially increased to 10Hz within 5 seconds. The parameter SweepFrequencyCount specifies how many sweeps should be done. In this case go up to 10Hz, and back to 1Hz.



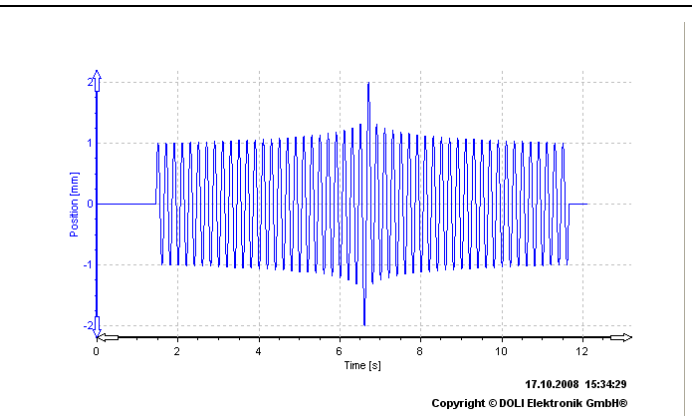
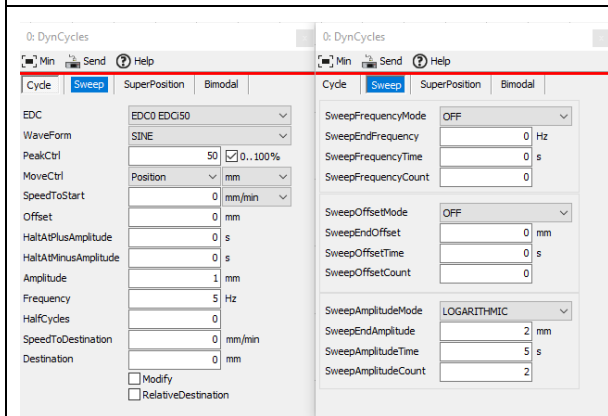
Sine command with a linear offset sweep.

Cosine starts with an offset of 0mm. The offset is linearly increased to 2mm within 5 seconds. The parameter SweepOffsetCount specifies how many sweeps should be done.



Sine command with a linear amplitude sweep.

Sine starts with amplitude of 1mm. The amplitude is linearly increased to 2mm within 5 seconds. The parameter SweepAmplitudeCount specifies how many sweeps should be done.



Sine command with a logarithmic amplitude sweep.

Sine starts with amplitude of 1mm. The amplitude is exponentially increased to 2mm within 5 seconds. The parameter SweepAmplitudeCount specifies how many sweeps should be done.

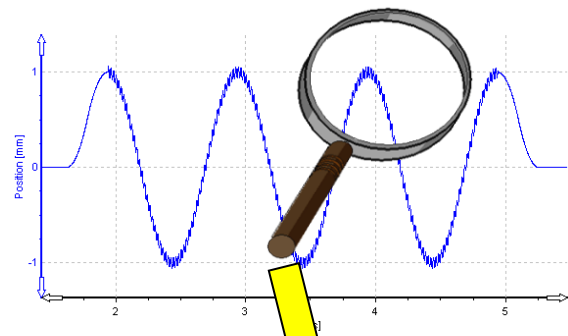
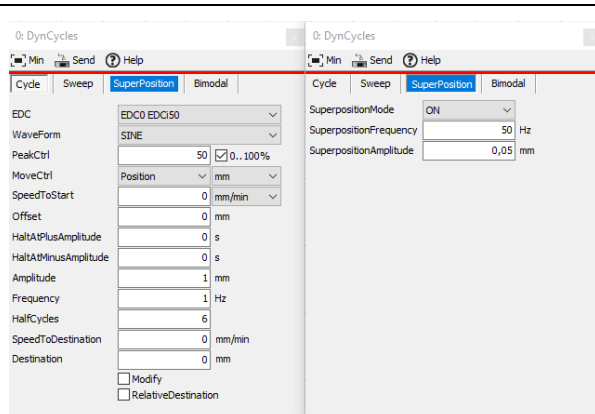
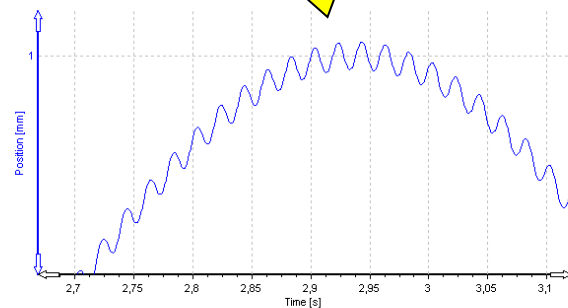
Please note: it is also possible to have different kind of sweeps at the same time. E.g. Frequency and amplitude sweep.

Criteria for detecting end of cycles:

First the parameter HalfCycles will be considered. If it's not equal to zero, the command is finished after the number of half-cycles are done, no matter if the sweep count has been reached.

If the parameter HalfCycles is set to zero, the command will be finished after all sweeps are completed.

5.6.6 Superposition Mode


 17.10.2008 15:37:21
 Copyright © DOLI Elektronik GmbH®

 17.10.2008 15:37:52
 Copyright © DOLI Elektronik GmbH®

Sine command superposition cosine.
 The basic frequency of the Cosine is 1Hz. A cosine with a frequency of 50Hz and amplitude of 0.05mm is superimposed.

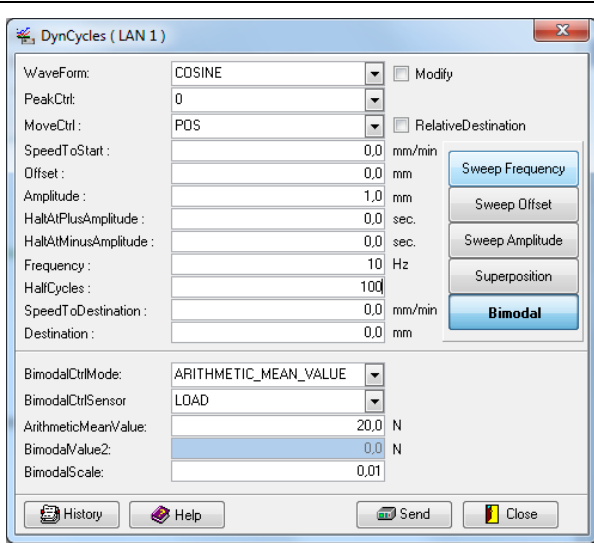
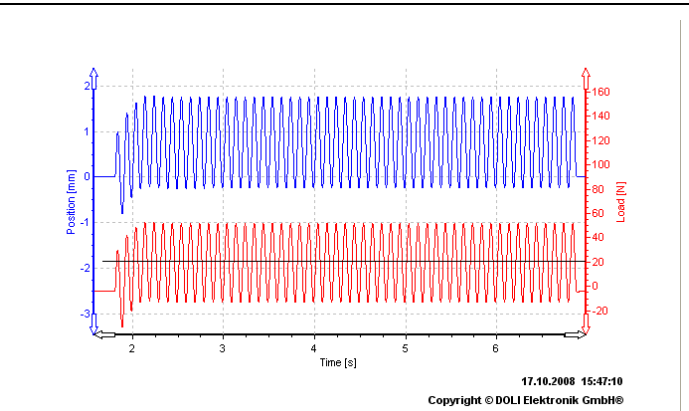
5.6.7 Bimodal Control Mode

Bimodal commands are not supported by EDCi10

A bimodal command runs the test in e.g. position control, but keep a certain e.g. load. The load is controlled by changing the offset of e.g. position. The parameter BimodalScale defines the scaling between the two sensors. You may calculate this parameter by determining the ratio between move control sensor / bimodal control sensor.

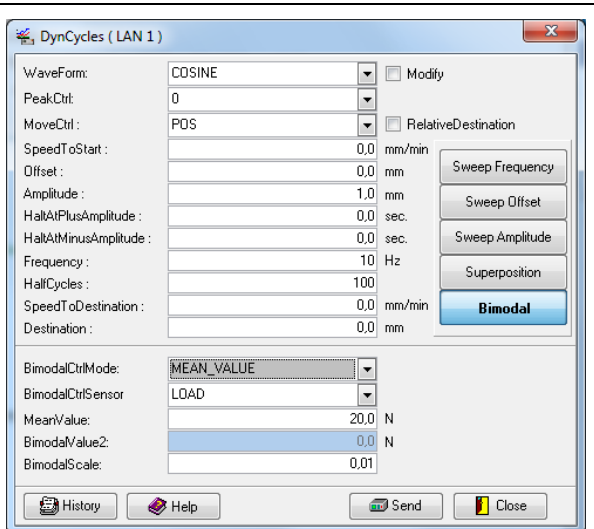
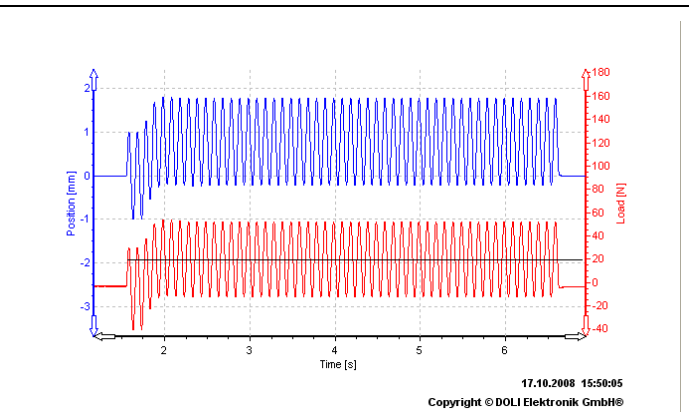
(For move control = Position, and bimodal control = Load, BimodalScale represents 1/stiffness of your specimen in mm/N.)

5.6.7.1 Control arithmetic mean value

Do cosine cycles in position control with amplitude of 1mm and starting offset of 0mm.
Control the arithmetic mean value of load at 20N, by modifying the offset of position.

5.6.7.2 Control mean value

Do cosine cycles in position control with an amplitude of 1mm and starting offset of 0mm.
Control the mean value of load at 20N, by modifying the offset of position.

5.6.7.3 Control minimum value

DynCycles (LAN 1)

WaveForm: COSINE ☐ Modify

PeakCtrl: 0

MoveCtrl: POS ☐ RelativeDestination

SpeedToStart: 0.0 mm/min

Offset: 0.0 mm

Amplitude: 1.0 mm

HaltAtPlusAmplitude: 0.0 sec.

HaltAtMinusAmplitude: 0.0 sec.

Frequency: 10 Hz

HalfCycles: 100

SpeedToDestination: 0.0 mm/min

Destination: 0.0 mm

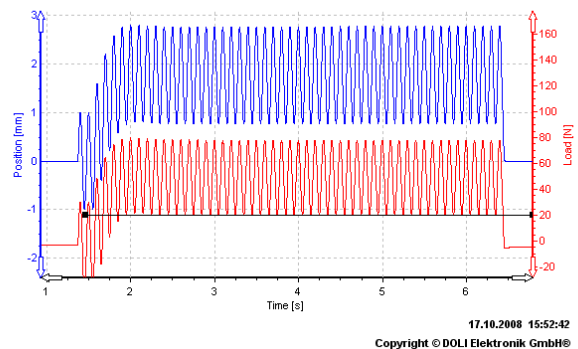
BimodalCtrlMode: MIN_VALUE

BimodalCtrlSensor: LOAD

MinValue: 20.0 N

BimodalValue2: 60.0 N

BimodalScale: 0.01



Do cosine cycles in position control with amplitude of 1mm and starting offset of 0mm.
Control the minimum value of load at 20N, by modifying the offset of position.

5.6.7.4 Control maximum value

DynCycles (LAN 1)

WaveForm: COSINE ☐ Modify

PeakCtrl: 0

MoveCtrl: POS ☐ RelativeDestination

SpeedToStart: 0.0 mm/min

Offset: 0.0 mm

Amplitude: 1.0 mm

HaltAtPlusAmplitude: 0.0 sec.

HaltAtMinusAmplitude: 0.0 sec.

Frequency: 10 Hz

HalfCycles: 100

SpeedToDestination: 0.0 mm/min

Destination: 0.0 mm

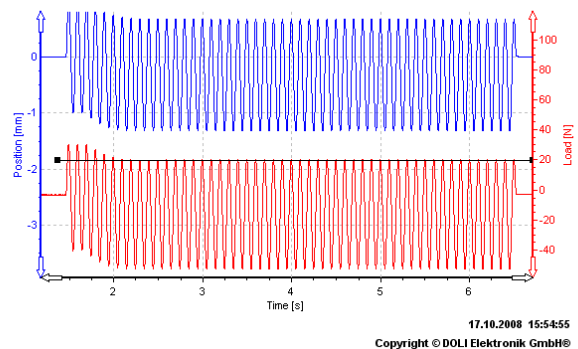
BimodalCtrlMode: MAX_VALUE

BimodalCtrlSensor: LOAD

MaxValue: 20.0 N

BimodalValue2: 0.0 N

BimodalScale: 0.01



Do cosine cycles in position control with amplitude of 1mm and starting offset of 0mm.
Control the maximum value of load at 20N, by modifying the offset of position.

5.6.7.5 Control minimum AND maximum values,

DynCycles (LAN 1)

WaveForm: COSINE ☐ Modify

PeakCtrl: 0

MoveCtrl: POS ☐ RelativeDestination

SpeedToStart: 0,0 mm/min

Offset: 0,0 mm

Amplitude: 1,0 mm

HaltAtPlusAmplitude: 0,0 sec.

HaltAtMinusAmplitude: 0,0 sec.

Frequency: 10 Hz

HalfCycles: 100

SpeedToDestination: 0,0 mm/min

Destination: 0,0 mm

BimodalCtrlMode: AMPLITUDE_OFFSET_VAR

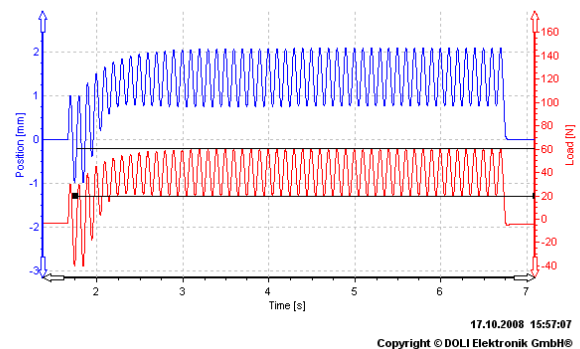
BimodalCtrlSensor: LOAD

Minimum: 20,0 N

Maximum: 60,0 N

BimodalScale: 0,01

History Help Send Close



Do cosine cycles in position control with amplitude of 1mm and starting offset of 0mm.
 Control the minimum and maximum value of the load at 20N, and 60N, by modifying the offset, and amplitude of position.

5.6.7.6 Control minimum AND maximum values

DynCycles (LAN 1)

WaveForm: COSINE ☐ Modify

PeakCtrl: 0

MoveCtrl: POS ☐ RelativeDestination

SpeedToStart: 0,0 mm/min

Offset: 0,0 mm

Amplitude: 1,0 mm

HaltAtPlusAmplitude: 0,0 sec.

HaltAtMinusAmplitude: 0,0 sec.

Frequency: 10 Hz

HalfCycles: 100

SpeedToDestination: 0,0 mm/min

Destination: 0,0 mm

BimodalCtrlMode: AMPLITUDE_OFFSET_CONST

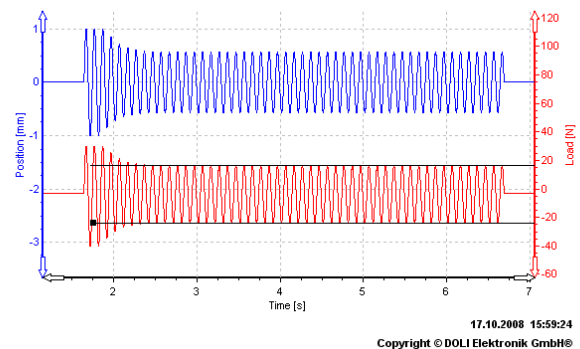
BimodalCtrlSensor: LOAD

Amplitude: 20,0 N

BimodalValue2: 60,0 N

BimodalScale: 0,01

History Help Send Close



Do cosine cycles in position control with amplitude of 1mm and starting offset of 0mm.
 Control the amplitude of load at 20N, by modifying the amplitude of position.

5.6.8 Restrictions:

- Frequency Sweeps are only possible for Cosine and Sine waveforms
- Amplitude or Offset sweep should not change faster than $4 * 1/\text{frequency}$
- Amplitude AND SweepEndAmplitude MUST have the same sign!
- Modify not permitted with active Amplitude or Offset Sweep
- Modify not permitted for different waveforms
- Modify not permitted for different control modes
- Modify not permitted for different signs of the Amplitude
- Modify not permitted with RelativeDestination
- Bimodal control sensor must be different to move control sensor
- BimodalValue1 must be smaller than BimodalValue2
- BimodalValue1 (Amplitude) must be positive
- BimodalScale must be smaller than 32767 (EDC point scale)
- Amplitude and Offset Sweeps are not permitted for bi-modal modes: "DYN_BIMODAL_MIN_VALUE" or "DYN_BIMODAL_MAX_VALUE".
- Amplitude and Offset Sweeps are not permitted for bi-modal modes: "DYN_BIMODAL_AMPLITUDE_OFFSET_VAR" or "DYN_BIMODAL_AMPLITUDE_OFFSET_CONST"
- Modify, Sweeps, Bimodal Control and Superposition are not supported by EDCi10

5.6.9 DoPESetPeakCtrl

Modify peak control cycles of a running DoPEDynCycles command.
Refer to 5.6.4 *Peak Control* for details.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short DoPESetPeakCtrl (DoPEHdl, PeakCtrl)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Peak control cycles 0, 1,2,4,8,16
.NET <Edc object>.Move.PeakCtrl(Int16 Cycles)	

5.6.10 DoPESetPeakCtrlValue

Modify peak control of the next coming or a running DoPEDynCycles command.
Refer to 5.6.4 *Peak Control* for details.

Minimum requirements: EDCi with EDCiApp 9149.022, DoPE 10.20

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double DoPESetPeakCtrlValue(DoPEHdl, PeakCtrlMode, PeakCtrlPlusAmplitude, PeakCtrlMinusAmplitude)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Peak control mode (see constants below) Peak control value for plus amplitude in % of the Amplitude Peak control value for minus amplitude in % of the Amplitude
Constants for PeakCtrlMode: PEAK_CTRL_OFF PEAK_CTRL_AUTO PEAK_CTRL_FIX	Peak control off (PeakCtrlPlusAmplitude and PeakCtrlMinusAmplitude will be ignored and set to 100%) Automatic peak control, set start values Fix peak control, set values
.NET <Edc object>.Move.SetPeakCtrlValue(Int16 PeakCtrlMode, Double PeakCtrlPlusAmplitude, Double PeakCtrlMinusAmplitude)	

5.6.11 DoPEGetPeakCtrlValue

Get the current peak control values of the running or the last DoPEDynCycles command.
Refer to 5.6.4 *Peak Control* for details.

Minimum requirements: EDCi with EDCiApp 9149.022, DoPE 10.20

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double DoPEGetPeakCtrlValue(DoPEHdl, *PeakCtrlMode, *PeakCtrlPlusAmplitude, *PeakCtrlMinusAmplitude)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Peak control mode (refer to constants in 5.6.10 <i>DoPESetPeakCtrlValue</i>) Peak control value for plus amplitude in % of the Amplitude Peak control value for minus amplitude in % of the Amplitude
.NET <Edc object>.Move.GetPeakCtrlValue(ref Int16 PeakCtrlMode, ref Double PeakCtrlPlusAmplitude, ref Double PeakCtrlMinusAmplitude)	

6 Synchronized data and movement

There are two commands (DoPESynchronizeSystemMode, DoPESynchronizeSystemStart) to control synchronized movement of two or more EDC systems. These commands are only useful if the synchronization option is installed, and the two or more EDC are correctly connected.

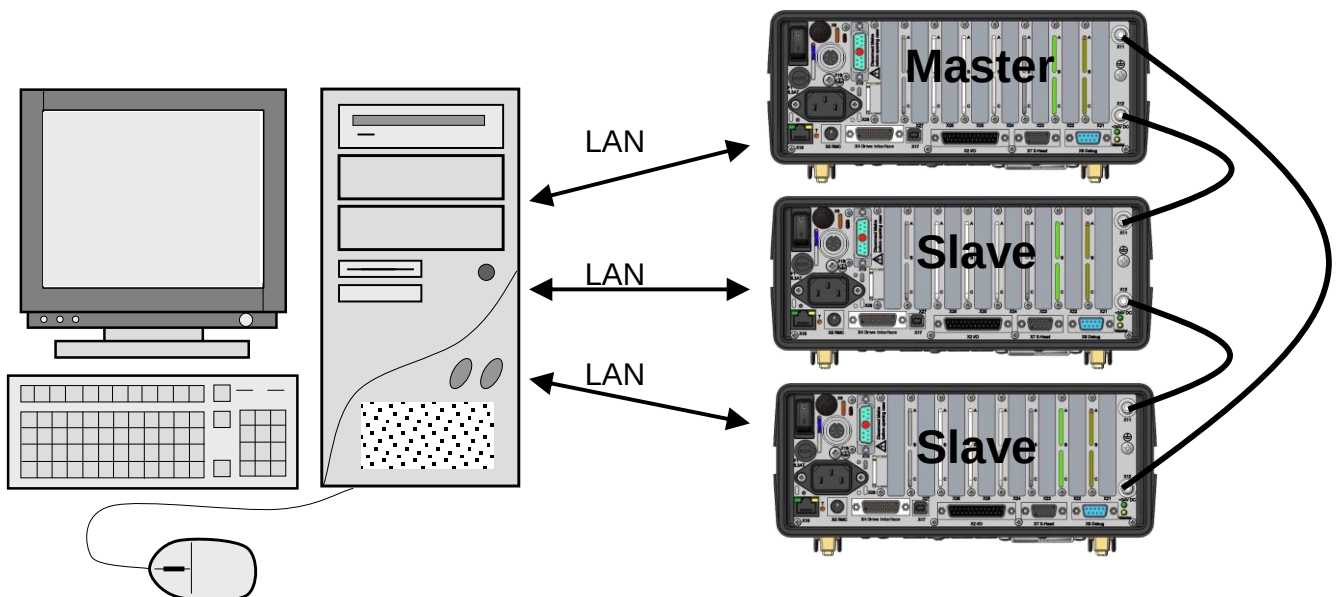
Important: **Make sure that all synchronized EDCs use the same**
 - **System Time**
 - **Data Transmission Rate**

With DoPESynchronizeSystemMode you can synchronize time and movement, with DoPESynchronizeSystemStart you can start the synchronization.

There is also an event handler for synchronized measuring data samples available (DoPESetOnSynchronizeDataHdlr). Activate sample synchronization after EDC initialization with the command DoPESynchronizeData. The event is activated after DoPE receives samples with the same time from all synchronized EDCs.

The parameter SyncOption of DoPEGeneralData must be set correctly in the EDC setup:

SYNC_OPTION_MASTER: for one Master EDC
 SYNC_OPTION_SLAVE: for all other Slave EDCs



There might be undefined measured values if a sync cable is unplugged during operation. We recommend supervising the CTRL_SYNCINPUT Bit of the Controller Status WORD 1 and take actions if the cabling is interrupted (e.g. switching off the drive and reinitializing the EDCs).

6.1 DoPESynchronizeData

Activate sample synchronization for a list of DoPE handles after EDC initialization. Passing an empty link list (first item NULL) stops sample synchronization. Initializing, selecting a setup or closing one or more links stops sample synchronization, too.

Minimum requirements: EDCi with EDCiApp 9149.008, DoPE 10.11

Function declaration	Description
extern unsigned DLLAPI DoPESynchronizeData(DoPE_HANDLE DoPEHdl[] DoPE_HANDLE *pMaster)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle array (terminated by NULL). Pointer to storage for the master DoPE link handle.
.NET <EdcList object>.Syn.SynchronizeData(EdcList ListEdc, out Edc MasterEdc)	

6.2 DoPESetOnSynchronizeDataHdlr

Set the OnSynchronizeData handler.

Minimum requirements: EDCi with EDCiApp 9149.008, DoPE 10.11

Function declaration	Description
extern unsigned DLLAPI DoPESetOnSynchronizeDataHdlr(DoPEOnSynchronizeDataHdlr Hdlr LPVOID lpParameter)	Function returns Error constant (DoPERR_ xxxx) User function to be called with every received synchronized sample. User specific pointer.
User function for the OnSynchronizeDataevent:	
unsigned CALLBACK OnSynchronizeData DoPE_HANDLE DoPEHdl DoPEOnSynchronizeData *Data LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to received data User specific pointer set with DoPESetOnSynchronizeDataHdlr
typedef struct { unsigned DoPEError unsigned nData DoPELinkData *Sample double Occupied } DoPEOnSynchronizeData; typedef struct { DoPE_HANDLE DP DoPEData Data } DoPELinkData	DoPERR_NOERROR event is ok. Number of measuring data records Array of measuring data records. Utilization [% of data buffer size] DoPE link handle Measuring data record
.NET <EdcList object>.Syn.DoPEOnSynchronizeDataHdlr += new DoPE.OnSynchronizeDataHdlr(Hdlr)	

6.3 DoPESetOnSynchronizeDataOverflowHdlr

Set the OnSynchronizeDataOverflow handler.

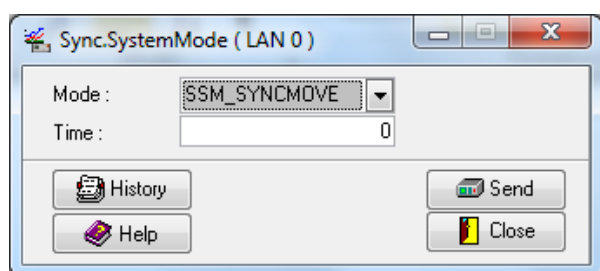
Minimum requirements: EDCi with EDCiApp 9149.008, DoPE 10.11

Function declaration	Description
extern unsigned DLLAPI DoPESetOnSynchronizeDataOverflowHdlr(DoPEOnSynchronizeDataOverflowHdlr Hdlr LPVOID lpParameter)	Function returns Error constant (DoPERR_ xxxx) User function to be called with every overflow of synchronized sample. User specific pointer.
User function for the OnSynchronizeDataOverflow event:	
unsigned CALLBACK OnSynchronizeDataOverflow LPVOID lpParameter	Function should return 0 (reserved for future versions) Userspecific pointer set with DoPESetOnSynchronizeDataOverflowHdlr
.NET <EdcList object>.Syn.DoPEOnSynchronizeDataOverflowHdlr += new DoPE.OnSynchronizeDataOverflowHdlr (Hdlr)	

6.4 DoPESynchronizeSystemMode

DoPESynchronizeSystemMode command is used for synchronized movement (see sample and state diagram below).

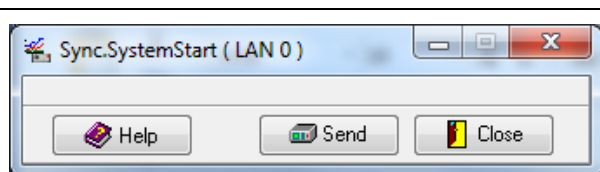
Minimum requirements: EDCi with EDCiApp 9149.008, DoPE 10.11



If Mode is SSM_SYNCMOVE, the next movement command like DoPEPos, DoPECosine or any other movement command will be delayed until synchronization condition is true. The systems will be synchronized by the DoPESynchronizeSystemStart command.

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short	DoPESynchronizeSystemMode(DoPEHdl Mode	Function returns Error constant (DoPERR_ xxxx) DoPE link handle SSM_SYNCMOVE: If Time is ZERO, start movement after synchronize signal is active. If Time is not ZERO, wait Time after synchronize signal is active before starting the movement. SSM_SYSTEMTIME: Set EDC-system time to Time, after synchronize signal is active. SSM_DISCARD: Discard previous DoPESynchronizeSystemMode command. Delay or system time to set with the next DoPESynchronizeSystemStart command.	s
double	Time		
.NET <Edc object>.Syn.SynchronizeSystemMode(DoPE.SSM Mode, Double Time)			

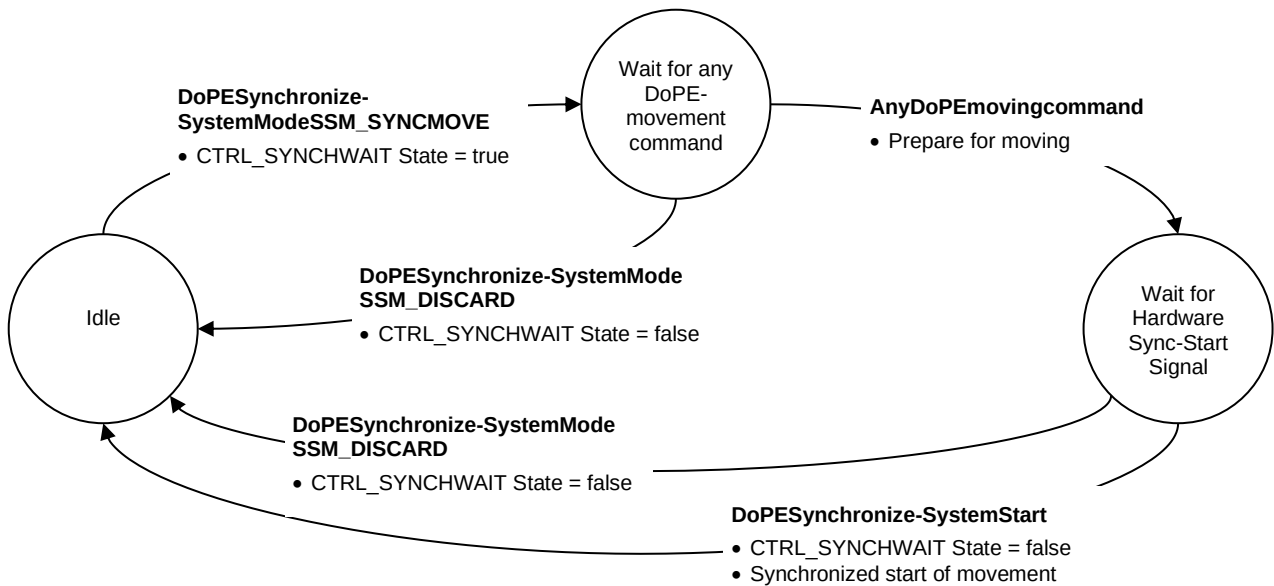
6.5 DoPESynchronizeSystemStart



This function can only be processed on the Master-EDC.
The digital synchronization signal will be activated, and with the next system clock, all involved EDC will start the previously defined actions, set by DoPESynchronizeSystemMode.

Minimum requirements: EDCi with EDCiApp 9149.008, DoPE 10.11

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE	DoPESynchronizeSystemStart (DoPEHdl	Function returns Error constant (DoPERR_ xxxx) DoPE link handle of Master EDC	
.NET <Edc object>.Syn.SynchronizeSystemStart()			



Example1: Two axes should do synchronized sinusoidal cycles with no phase shift.
Attention: A real program must check the return codes!

```

/* activate sample synchronization after EDC initialization once */
DoPE_HANDLE DoPEHdl[] = {DoPEHdl_1, DoPEHdl_2, NULL};
DoPE_HANDLE DoPEHdl_Master;
Error = DoPESynchronizeData(DoPEHdl, &DoPEHdl_Master);
Error = DoPESetOnSynchronizeDataHdlr(MyOnSynchronizeDataHdlr, NULL);

/* move both EDCs to the start position */
Error = DoPEPos(DoPEHdl_1, CTRL_POS, Speed, Offset + Amplitude, &lpusTAN_1);
Error = DoPEPos(DoPEHdl_2, CTRL_POS, Speed, Offset + Amplitude, &lpusTAN_2);

/* wait until both EDCs reached the start position (check OnPosMsg handler) */

/* start cosine after synchronization without delay */
Error = DoPESynchronizeSystemMode(DoPEHdl_1, SSM_SYNCMOVE, 0);
Error = DoPEDynCycles(DoPEHdl_1, DYN_WAVEFORM_COSINE, ...);

/* start cosine after synchronization without delay */
Error = DoPESynchronizeSystemMode(DoPEHdl_2, SSM_SYNCMOVE, 0);
Error = DoPEDynCycles(DoPEHdl_2, DYN_WAVEFORM_COSINE, ...);

/* start synchronization at Master EDC */
Error = DoPESynchronizeSystemStart(DoPEHdl_Master);
  
```

Example2: Two axes should do synchronized sinusoidal cycles with 90° phase shift.
Attention: A real program must check the return codes!

```
/* activate sample synchronization after EDC initialization once */
DoPE_HANDLE DoPEHdl[] = {DoPEHdl_1, DoPEHdl_2, NULL};
DoPE_HANDLE DoPEHdl_Master;
Error = DoPEsynchronizeData(DoPEHdl, &DoPEHdl_Master);
Error = DoPESetOnSynchronizeDataHdlr(MyOnSynchronizeDataHdlr, NULL);

/* move both EDCs to the start position */
Error = DoPEPos(DoPEHdl_1, CTRL_POS, Speed, Offset + Amplitude, &lpusTAN_1);
Error = DoPEPos(DoPEHdl_2, CTRL_POS, Speed, Offset + Amplitude, &lpusTAN_2);

/* wait until both EDCs reached the start position (check OnPosMsg handler) */

/* start cosine after synchronization without delay */
Error = DoPEsynchronizeSystemMode(DoPEHdl_1, SSM_SYNCMOVE, 0);
Error = DoPEDynCycles(DoPEHdl_1, DYN_WAVEFORM_COSINE, ...);

/* start cosine after synchronization 90° phase shift */
Error = DoPEsynchronizeSystemMode(DoPEHdl_2, SSM_SYNCMOVE, 1 / (Frequency * 4));
Error = DoPEDynCycles(DoPEHdl_2, DYN_WAVEFORM_COSINE, ...);

/* start synchronization at Master EDC */
Error = DoPEsynchronizeSystemStart(DoPEHdl_Master);
```

7 Miscellaneous Control Commands

7.1 DoPEOn

Activate drive.
Wait until the CTRL_READY bit in controller status word 1 is active before any movement command is used.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description	Unit
extern unsigned DLLAPI DoPE_HANDLE DoPEOn(DoPEHdl	Function returns Error constant (DoPERR_xxxx) DoPE link handle	
.NET <Edc object>.Move.On()		

7.2 DoPEOff

Deactivate drive.

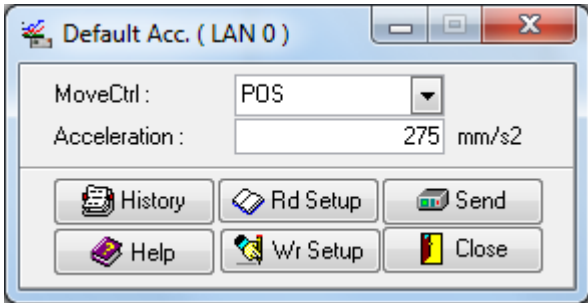
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description	Unit
extern unsigned DLLAPI DoPE_HANDLE DoPEOff(DoPEHdl	Function returns Error constant (DoPERR_xxxx) DoPE link handle	
.NET <Edc object>.Move.Off()		

7.3 DoPEDefaultAcc

Set default acceleration (and deceleration) for all movement commands. After initialization default and nominal acceleration are identical.

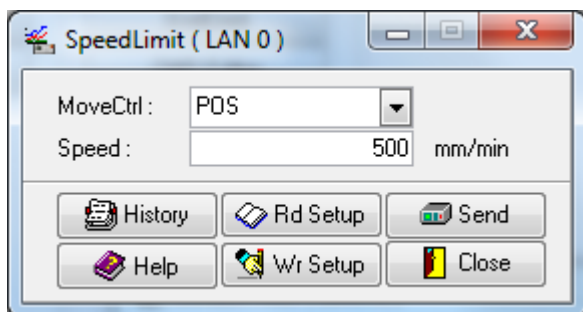
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

		
Function declaration	Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double DoPEDefaultAcc (DoPEHdl MoveCtrl Acc	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) Default Acceleration	
.NET <Edc object>.Move.DefaultAcc(DoPE.CTRL MoveCtrl, Double Acc)		

7.4 DoPESpeedLimit

Set speed limit. The maximum speed for all movement commands will be limited to this speed limit.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



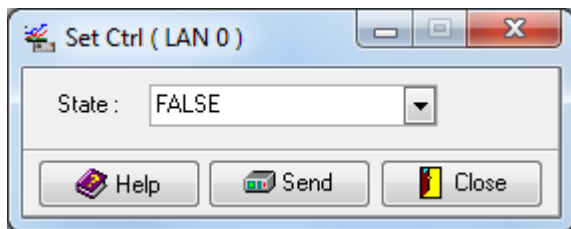
Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double	DoPESpeedLimit (DoPEHdl MoveCtrl Speed	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_XXX) Maximum allowed speed (must be below nominal speed)	Unit/s

.NET <Edc object>.Move.SpeedLimit(DoPE.CTRL MoveCtrl, Double Speed)

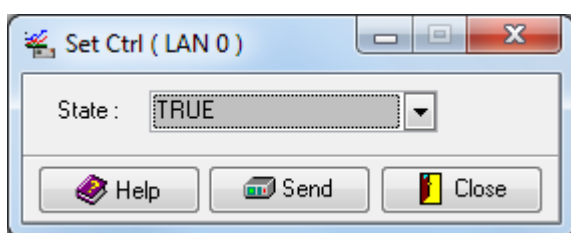
7.5 DoPESetCtrl

Enable / disable closed loop control.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



Use this command to switch off EDC-Closed Loop Control.
After switching closed loop control off, PC must wait until all bits in ActiveCtrl are ZERO.



This command activates closed loop control of EDC.
Wait until the CTRL_READY bit in controller status word 1 is active before any movement command is used!

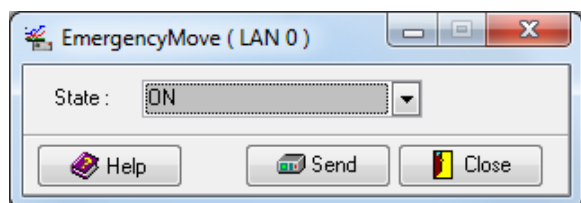
Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned WORD	DoPESetCtrl (DoPEHdl Enable *lpusTAN	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 enables =0 disables closed loop control Pointer to transaction number.	

.NET <Edc object>.Move.SetCtrl(bool Enable, ref Int16 Tan)

7.6 DoPEEmergencyMove

Activate / deactivate emergency movement. Emergency movement is used to move crosshead if the hardware limit switch is active.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

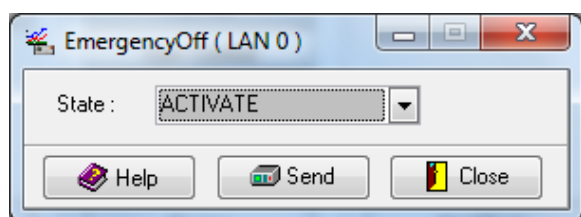


Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE Unsigned short	DoPEEmergencyMove (DoPEHdl State	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 on =0 off	
.NET <Edc object>.Move.EmergencyMove(bool Activate)			

7.7 DoPEEmergencyOff

Activate / deactivate EmergencyOff state.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



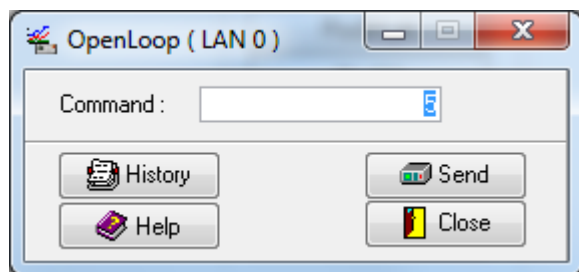
The emergency off state may be activated or deactivated by software.

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE Unsigned short	DoPEEmergencyOff (DoPEHdl State	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 Activate emergency off =0 Deactivate emergency off	
.NET <Edc object>.Move.EmergencyOff(bool Activate)			

7.8 DoPESetOpenLoopCommand

Set command output to a given valve.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



Sets the command output to a given value.
This command is used in open loop controller structure (typically hydraulic concrete testing machines) to move the machine without position feedback.
In all other controller structures the command deactivates the controller and sets the command output to the given "Command" value. The controller will be reactivated by sending any other movement or halt command to the controller.

Function declaration		Description	Unit
extern unsigned DLLAPI DoPE_HANDLE double	DoPESetOpenLoopCommand (DoPEHdl Command	Function returns Error constant (DoPERR_XXXX) DoPE link handle Output value to valve in %.	%
.NET <Edc object>.Output.SetOpenLoopCommand(Double Command)			

8 Event Handler

8.1 Using DoPE Event handler

Whenever necessary, EDC will send a message, possibly with parameters to PC. There are many different reasons for such messages. E.g. if a positioning command has reached the destination, EDC will send a message to PC, reporting the destination has been reached.

To make it easy for the application program to get these messages, DoPE offers an event handler interface. For each event, the application program wants to process, an event handler must be installed. DoPE will call these event handlers, whenever the event occurs.

8.2 Overview of Events:

Event Name	Description	Related DoPE Command
OnLine	EDC goes On/Offline	-
OnDataBlock	New data record(s) with measures values	-
OnCommandError	Error event upon a command	Any command
OnPosMsg	A positioning command has reached the destination	Any positioning command like DoPEPos, DoPEBlockExecute
OnTPosMsg	Trigger position hit	DoPETrig, Positioning command with DoPEBlockHeader/Execute
OnLPosMsg	Limit position reached	DoPEPosExt, DoPEFDPoti when reaching softend
OnSftMsg	Softend reached	DoPESft, Any positioning command
OnOffsCMsg	Offset correction finished	DoPEOffsC
OnCheckMsg	A measuring channel supervision hit.	DoPESetCheck
OnRefSignalMsg	Reference signal message of an encoder	DoPESetRefSignalMode
OnSensorMsg	Message from a serial sensor	-
OnOverflow	Overflow of data records	-
OnRuntimeError	Any error condition, like emergency off active.	-
OnIoSHaltMsg	Upper / Lower S-Halt input triggered	-
OnGuardMsg	Guard option range or speed limit triggered	-
OnKeyMsg	RMC-key state	-
OnSystemMsg	System message, like initialization errors	-
OnDebugMsg	Debug message (for remote support)	DoPEDebugMsgEnable DoPESendDebugCommand
OnRmcEvent	RMC default handling for keys and LEDs	DoPERmcEnable

8.3 DoPESetOnLineHdlr

The OnLine Handler supplies any change of the communication state between EDC and PC.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl DoPEOnLineHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after every line state change. User specific pointer.
User function for the OnLine event:	
unsigned CALLBACK OnLine DoPE_HANDLE DoPEHdl int LineState LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle DoPEOFFLINE (0) link is offline DoPEONLINE (1) link is online DoPERESTART (2) asynchronous restart of the link User specific pointer set with DoPESetOnLineHdlr
.NET <Edc object>.Eh.OnLineHdlr += new DoPE.OnLineHdlr(Hdlr)	

8.4 DoPESetOnDataBlockHdlr

The OnDataBlock event handler is called with each received measuring data record block from EDC. Per default DoPE collects 50 measuring data records in a block. This number can be changed with the DoPESetOnDataBlockSize function.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl DoPEOnDataBlockHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after a new data record block is available. User specific pointer
User function for the OnDataBlock event:	
unsigned CALLBACK OnDataBlock DoPE_HANDLE DoPEHdl DoPEOnDataBlock *pData LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to measuring data record block User specific pointer set with DoPESetOnDataBlockHdlr
typedef struct { unsigned DoPEError; unsigned nData DoPEOnData *pData double Occupied; } DoPEOnDataBlock;	DoPERR_NOERROR event is ok, else at least one measuring data record reports an error. Number of measuring data records in the block Pointer to measuring data records Utilization in percent of the measuring data circular buffer inside DoPE.
typedef struct { unsigned DoPEError DoPEData Data double Occupied; } DoPEOnData;	DoPERR_NOERROR event is ok, Measuring data record Utilization in percent of the measuring data circular buffer inside DoPE
.NET <Edc object>.Eh.OnDataBlockHdlr += new DoPE.OnDataBlockHdlr(Hdlr)	

8.5 DoPESetOnDataBlockSize

Set the number of measuring data records to collect in an OnDataBlock block.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPESetOnDataBlockSize DoPEHdl nData	Function returns Error constant (DoPERR_XXXX) DoPE link handle Number of samples to collect in a block
.NET <Edc object>.Eh.SetOnDataBlockSize(Int32 nData)	

8.6 DoPEGetOnDataBlockSize

Get the number of measuring data record to collect in an OnDataBlock block.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPEGetOnDataBlockSize DoPEHdl *nData	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to storage for number of samples to collect in a block
.NET <Edc object>.Eh.GetOnDataBlockSize(ref Int32 nData)	

8.7 DoPESetOnCommandErrorHdlr

The OnCommandError event handler is called after parameter check of DoPE commands in the EDC firmware.

Normally all DoPE commands report parameter errors as return value, after being checked by the EDC firmware. Therefore you don't need to use the OnCommandError event handler.

If you send a DoPE command from a DoPE realtime handler (see chapter 18.1), DoPE can't wait for the answer of the EDC firmware, due to the high priority level. In this case you must use the OnCommandError event handler to get the parameter errors from the EDC firmware. Some internal DoPE functions also use the realtime handler, e.g. RMC default handler.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESetOnCommandErrorHdlr DoPE_HANDLE DoPEHdl DoPEOnCommandErrorHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called. User specific pointer.
User function for the OnCommandError event:	
unsigned CALLBACK OnCommandError DoPE_HANDLE DoPEHdl DoPEOnCommandError *pError LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to command error structure User specific pointer set with DoPESetOnDataHdlr
typedef struct { unsigned short CommandNumber unsigned short ErrorNumber unsigned short usTAN } DoPEOnCommandError;	EDC internal command number (for internal use). ErrorNumber specifies the error type (see DoPERR_CMD_XXX). Transaction number of the corresponding DoPE command.
.NET <Edc object>.Eh.OnCommandErrorHdlr += new DoPE.OnCommandErrorHdlr(Hdlr)	

8.8 DoPESetOnPosMsgHdlr

Whenever a movement command, like DoPEPos, is finished, EDC will send a message to PC. The movement command is finished after the command generator reached the final destination. DoPE will call the OnPosMsg event handler with a pointer to the DoPEOnPosMsg structure:

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESetOnPosMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnPosMsgHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called. User specific pointer.
User function for the OnPosMsg event:	
unsigned CALLBACK OnPosMsg DoPE_HANDLE DoPEHdl DoPEOnPosMsg * pPosMsg LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to positioning message structure User specific pointer set with DoPESetOnPosMsgHdlr
typedef struct { unsigned DoPEError unsigned short Reached double Time unsigned short Control double Position unsigned short DControl double Destination unsigned short usTAN } DoPEOnPosMsg;	DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. 1 if current position is within the destination window, 0 if current position is outside destination window. EDC-System time when reaching destination. Control mode of the next parameter (Position) If Reached = 1 Destination of the corresponding movement command. If Reached = 0 Current position. Control mode of the next parameter (Destination) Here: DControl has always the same value as Control. Destination of the corresponding movement command. Transaction number of the corresponding movement command.
.NET <Edc object>.Eh.OnPosMsgHdlr += new DoPE.OnPosMsgHdlr(Hdlr)	

8.9 DoPESetOnTPosMsgHdlr

This event is sent if the trigger position of a DoPETrig command hits. DoPE will call the OnTPosMsg event handler with a pointer to the DoPEOnPosMsg structure:

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESetOnTPosMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnTPosMsgHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after the trigger position of DoPETrig command hits. User specific pointer.
User function for the OnTPosMsg event:	
unsigned CALLBACK OnTPosMsg DoPE_HANDLE DoPEHdl DoPEOnTPosMsg * pTPosMsg LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to positioning message structure User specific pointer set with DoPESetOnTPosMsgHdlr
typedef struct { unsigned DoPError unsigned short Reached double Time unsigned short Control double Position unsigned short DControl double Destination unsigned short usTAN } DoPEOnPosMsg;	DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. Always 1. EDC-System time when trigger hits. Control mode of the next parameter (Position) Limit position of DoPETrig com. Control mode of the next parameter (Destination). Trigger position of DoPETrig command. Transaction number of the corresponding DoPETrig command.
.NET <Edc object>.Eh.OnTPosMsgHdlr += new DoPE.OnTPosMsgHdlr(Hdlr)	

8.10 DoPESetOnLPosMsgHdlr

This event is sent if the limit position of a DoPEPosExt command hits, or a softend position exceeds during DoPEFDPoti command. DoPE will call the OnLPosMsg event handler with a pointer to the DoPEOnPosMsg structure:

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESetOnLPosMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnLPosMsgHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after the limit position of DoPEPosExt command hits. User specific pointer.
User function for the OnLPosMsg event:	
unsigned CALLBACK OnLPosMsg DoPE_HANDLE DoPEHdl DoPEOnLPosMsg *pLPosMsg LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to positioning message structure User specific pointer set with DoPESetOnLPosMsgHdlr
typedef struct { unsigned DoPEError unsigned short Reached double Time unsigned short Control double Position unsigned short DControl double Destination unsigned short usTAN } DoPEOnPosMsg;	DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. Always 1. EDC-System time when limit hits. Control mode of the next parameter (Position) Limit position of DoPEPosExt com. Control mode of the next parameter (Destination). Destination position of DoPEPosExt command. Transaction number of the corresponding DoPEPosExt command.
.NET <Edc object>.Eh.OnLPosMsgHdlr += new DoPE.OnLPosMsgHdlr(Hdlr)	

8.11 DoPESetOnSftMsgHdlr

This event is sent if a softend position was exceeded.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESetOnSftMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnSftMsgHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after a positioning command was finished. User specific pointer.
User function for the OnSftMsg event:	
unsigned CALLBACK OnSftMsg DoPE_HANDLE DoPEHdl DoPEOnSftMsg *pSftMsg LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to positioning message structure User specific pointer set with DoPESetOnSftMsgHdlr
typedef struct { unsigned DoPEError unsigned short Upper double Time unsigned short Control double Position unsigned short usTAN } DoPEOnSftMsg;	DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. 1 = upper softend. 0 = lower softend. EDC-System time when softend was exceeded. Control mode of the softend that was exceeded. Position, where the softend was detected. Transaction number of the last positioning command.
.NET <Edc object>.Eh.OnSftMsgHdlr += new DoPE.OnSftMsgHdlr(Hdlr)	

8.12 DoPESetOnOffsCMsgHdlr

Handler for offset correction messages		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration		Description
extern unsigned DLLAPI DoPESetOnOffsCMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnOffsCMsgHdlr Hdlr LPVOID lpParameter		Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after an offset correction, activated withDoPEOffsC command was finished. User specific pointer.
User function for the OnOffsCMsg event:		
unsigned CALLBACK OnOffsCMsg DoPE_HANDLE DoPEHdl DoPEOnOffsCMsg *pOffsCMsg LPVOID lpParameter		Function should return 0 (reserved for future versions) DoPE link handle Pointer to positioning message structure User specific pointer set with DoPESetOnOffsCMsgHdlr
typedef struct { unsigned DoPEError double Time unsigned short Offset unsigned short usTAN } DoPEOnOffsCMsg;		DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. EDC-System time when offset correction was finished. Measured offset of the output channel. Transaction number of the corresponding DoPEOffsC command.
.NET <Edc object>.Eh.OnOffsCMsgHdlr += new DoPE.OnOffsCMsgHdlr(Hdlr)		

8.13 DoPESetOnCheckMsgHdlr

Handler for channel supervision messages		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration		Description
extern unsigned DLLAPI DoPE_HANDLE DoPEOnCheckMsgHdlr LPVOID	DoPESetOnCheckMsgHdlr DoPEHdl Hdlr lpParameter	Function returns Error constant (DoPERR_ xxxx) DoPE link handle User function to be called after a channel supervision condition hits. User specific pointer.
User function for the OnCheckMsg event:		
unsigned CALLBACK OnCheckMsg DoPE_HANDLE DoPEOnCheckMsg LPVOID	DoPEHdl *pCheckMsg lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to check message structure User specific pointer set with DoPESetOnCheckMsgHdlr
typedef struct { unsigned unsigned short double unsigned short Double unsigned short unsigned short } DoPEOnCheckMsg;	DoPEError Action Time CheckId Position SensorNo usTAN	DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. 1 = the action of DoPESetCheck command was started. 0 = the action of DoPESetCheck command was NOT started. (due to an error condition like active softend) EDC-System time. ID of measuring channel check. Position of the supervised sensor. Supervised sensor. Transaction number of the corresponding DoPESetCheck command.
.NET <Edc object>.Eh.OnCheckMsgHdlr += new DoPE.OnCheckMsgHdlr(Hdlr)		

8.14 DoPESetOnRefSignalMsgHdlr

Handler for reference signal messages		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration		Description
extern unsigned DLLAPI DoPESetOnRefSignalMsgHdlr DoPE_HANDLE DoPEHdl DoPESetOnRefSignalMsgHdlr Hdlr LPVOID lpParameter		Function returns Error constant (DoPERR_ xxxx) DoPE link handle User function to be called after a reference signal occurred. User specific pointer.
User function for the OnRefSignalMsg event:		
unsigned CALLBACK OnRefSignalMsg DoPE_HANDLE DoPEHdl DoPEOnRefSignalMsg * pRefSignalMsg LPVOID lpParameter		Function should return 0 (reserved for future versions) DoPE link handle Pointer to reference signal message structure. User specific pointer set with DoPESetOnRefSignalMsgHdlr
typedef struct { unsigned DoPEError double Time unsigned short SensorNo Double Position unsigned short usTAN } DoPEOnRefSignalMsg;		DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. EDC-System time. Sensor where the reference signal occurred. Position at reference signal. Transaction number of the corresponding DoPESetRefSignalMode command.
.NET <Edc object>.Eh.OnRefSignalMsgHdlr += new DoPE.OnRefSignalMsgHdlr(Hdlr)		

8.15 DoPESetOnSensorMsgHdlr

Handler for serial sensor messages		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration		Description
extern unsigned DLLAPI DoPESetOnSensorMsgHdlr DoPE_HANDLE DoPEHdl DoPESetOnSensorMsgHdlr Hdlr LPVOID lpParameter		Function returns Error constant (DoPERR_ xxxx) DoPE link handle User function to be called after a message from a serial sensor has been received. User specific pointer.
User function for the OnSensorMsg event:		
unsigned CALLBACK OnSensorMsg DoPE_HANDLE DoPEHdl DoPEOnSensorMsg * pOnSensorMsg LPVOID lpParameter		Function should return 0 (reserved for future versions) DoPE link handle Pointer to serial sensor message structure. User specific pointer set with DoPESetOnSensorMsgHdlr
typedef struct { unsigned DoPEError double Time unsigned short SensorNo unsigned short Length unsigned char Data[SENSOR_MSG_LEN+1] unsigned short usTAN } DoPEOnSensorMsg;		DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. EDC-System time. Sensor number. Number of bytes in Data. Message from the sensor. Transaction number is unused, always ZERO.
.NET <Edc object>.Eh.OnSensorMsgHdlr += new DoPE.OnSensorMsgHdlr(Hdlr)		

8.16 DoPESetOnIoSHaltMsgHdlr

Handler for IO-SHalt messages.	
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00	
Function declaration	Description
extern unsigned DLLAPI DoPESetOnIoSHaltMsgHdlr DoPE_HANDLE DoPEHdl DoPESetOnIoSHaltMsgHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after a SHalt input has triggered. User specific pointer.
User function for the OnIoSHaltMsg event:	
unsigned CALLBACK OnIoSHaltMsg DoPE_HANDLE DoPEHdl DoPEOnIoSHaltMsg *pOnIoSHaltMsg LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to IO-SHalt message structure. User specific pointer set with DoPESetOnIoSHaltMsgHdlr
typedef struct { unsigned DoPEError unsigned short Upper double Time unsigned short Control double Position unsigned short usTAN } DoPEOnIoSHaltMsg;	DoPERR_NOERROR event is ok. Else error occurred during event generation. 1: Upper SHalt input triggered. 0: Lower SHalt input triggered. EDC-System time. Control mode of Position. Position at SHalt input trigger. Transaction number.
.NET <Edc object>.Eh.OnIoSHaltMsgHdlr += new DoPE.OnIoSHaltMsgHdlr(Hdlr)	

8.17 DoPESetOnGuardMsgHandler

Handler for guard option messages.	
Minimum requirements: EDCi with EDCiApp 9149.022, DoPE 10.19	
Function declaration	Description
extern unsigned DLLAPI DoPESetOnGuardMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnGuardMsgHdlr Hdlr void *lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after a guard event has triggered. User specific pointer.
User function for the OnGuardMsg event:	
unsigned CALLBACK DoPEOnGuardMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnGuardMsg *pOnGuardMsg void lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to guard message structure. User specific pointer set with DoPESetOnGuardMsgHdlr
typedef struct { unsigned DoPEError unsigned short Trigger double Time unsigned short TriggerSensor double TriggerValue unsigned short usTAN } DoPEOnGuardMsg;	DoPERR_NOERROR event is ok. Else error occurred during event generation, Trigger of the guard event message. See Constants. EDC-System time, when triggered. Number of the monitored sensor. (RangeLimit-/SpeedLimitSensor) Value of the monitored sensor when triggered. Transaction number.
Constants for Trigger GUARD_UPPER_RANGE_LIMIT_HALT GUARD_LOWER_RANGE_LIMIT_HALT GUARD_UPPER_RANGE_LIMIT_OVERRUN GUARD_LOWER_RANGE_LIMIT_OVERRUN GUARD_SPEED_LIMIT_OVERRUN	Halt at upper range limit triggered Halt at lower range limit triggered Upper range limit overrun Lower range limit overrun Speed limit overrun
.NET <Edc object>.Eh.OnGuardMsgHdlr += new DoPE.OnGuardMsgHdlr(Hdlr)	

8.18 DoPESetOnKeyMsgHdlr

Handler for key messages. For each activated or deactivated key this message is sent.	
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00	
Function declaration	Description
extern unsigned DLLAPI DoPESetOnKeyMsgHdlr DoPE_HANDLE DoPEHdl DoPESetOnKeyMsgHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after a change of any (RMC) key was detected. User specific pointer.
User function for the OnKeyMsg event:	
unsigned CALLBACK OnKeyMsg DoPE_HANDLE DoPEHdl DoPEOnKeyMsg * pOnKeyMsg LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to key message structure. User specific pointer set with DoPESetOnKeyMsgHdlr
typedef struct { unsigned DoPEError double Time unsigned __int64 Keys unsigned __int64 NewKeys unsigned __int64 GoneKeys unsigned short OemKeys unsigned short NewOemKeys unsigned short GoneOemKeys unsigned short usTAN } DoPEOnKeyMsg;	DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. EDC-System time. Current key state. All new keys. All gone keys. Current OEM key state (for future use). New OEM keys (for future use). Gone OEM keys (for future use). Transaction number.
.NET <Edc object>.Eh.OnKeyMsgHdlr += new DoPE.OnKeyMsgHdlr(Hdlr)	

8.19 DoPESetOnRmcEventHdlr

The OnRmcEvent Handler supplies any change of RMC default handler key input and LED output.	
Function declaration	Description
extern unsigned DLLAPI DoPESetOnRmcEventHdlr DoPE_HANDLE DoPEHdl DoPEOnRmcEventHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after every RMC default handler key and LED changes. User specific pointer.
User function for the OnRmcEvent event:	
unsigned CALLBACK OnRmcEvent DoPE_HANDLE DoPEHdl DoPEOnRmcEvent * pRmcEvent LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to received RMC event structure User specific pointer set with DoPESetOnRmcEventHdlr
typedef struct { unsigned __int64 Keys unsigned __int64 NewKeys unsigned __int64 GoneKeys unsigned __int64 Leds unsigned __int64 NewLeds unsigned __int64 GoneLeds } DoPEOnRmcEvent	Current key state New keys Gone keys Current LED state New LED's Gone LED's
.NET <Edc object>.Eh.OnRmcEventHdlr += new DoPE.OnRmcEventHdlr (Hdlr)	

8.20 DoPESetOnRuntimeErrorHdlr

Handler for runtime error messages.	
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00	
Function declaration	Description
extern unsigned DLLAPI DoPESetOnRuntimeErrorHdlr DoPE_HANDLE DoPEHdl DoPEOnRuntimeErrorHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after a runtime error occurred. User specific pointer.
User function for the OnOnRuntimeError event:	
unsigned CALLBACK OnRuntimeError DoPE_HANDLE DoPEHdl pOnRuntimeError * pOnRuntimeError LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to runtime error structure. User specific pointer set with DoPESetOnRuntimeErrorHdlr
typedef struct { unsigned DoPEError unsigned short ErrorNumber double Time unsigned short Device unsigned short Bits unsigned short usTAN } DoPEOnRuntimeError;	DoPERR_NOERROR event is ok. Else error occurred during event generation, e.g. data conversion error. Number of run time error. System time the error occurred. Device Number responsible for the error. Responsible bits of the device. Transaction number. Note: Device and Bits are only information for DOLI service personal.
.NET <Edc object>.Eh.OnRuntimeErrorHdlr += new DoPE.OnRuntimeErrorHdlr(Hdlr)	

8.20.1 List of Runtime errors

Error number	Error constant	Description
104	RTE_EMOVE_END	Error at end of emergency movement still active
105	RTE_CTRL_DEVIATION	Controller deviation error (Control channel see device)
201	RTE_DRIVE_OFF	Drive off or emergency off
202	RTE_E_MOVE_RQ	emergency off, emergency drive required
203	RTE_UPPER_LIMIT_SWITCH	Upper hard-limit switch active
204	RTE_LOWER_LIMIT_SWITCH	Lower hard-limit switch active
205	RTE_STOP	Drive not ready
206	RTE_DF_KEY	Drive free withdrawn by key
207	RTE_SHALT	Signal S-HALT activated
301	RTE_UPPER_LIMIT	Upper range limit exceeded
302	RTE_LOWER_LIMIT	Lower range limit exceeded

8.21 DoPESetOnOverflowHdlr

Handler for overflow messages.

An overflow may occur, if data records or messages from EDC are not processed by the application program for a longer time. The application program can adjust the number of data records, and messages when opening the link to EDC with DoPE open functions.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESetOnOverflowHdlr DoPE_HANDLE DoPEHdl DoPEOnOverflowHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called after overflow detection. User specific pointer.
User function for the OnOverflow event:	
unsigned CALLBACK OnOverflow DoPE_HANDLE DoPEHdl int Overflowtype LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle 0=measuring data records lost else =message lost. User specific pointer set with DoPESetOnOverflowHdlr
.NET <Edc object>.Eh.OnOverflowHdlr += new DoPE.OnOverflowHdlr(Hdlr)	

8.22 DoPESetOnSystemMsgHdlr

Handler for EDC system messages.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESetOnSystemMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnSystemMsgHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called for system messages. User specific pointer.
User function for the OnSystemMsg event:	
unsigned CALLBACK OnSystemMsg DoPE_HANDLE DoPEHdl DoPEOnSystemMsg *SystemMsg LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to received system message User specific pointer set with DoPESetOnSystemMsgHdlr
typedef struct { unsigned DoPEError unsigned short MsgNumber double Time wchar_t Text[SYSTEM_MSG_TEXT_LEN] } DoPEOnSystemMsg;	DoPERR_NOERROR event is ok. Number of system message (SYSTEM_MSG_XXX) System time the message was sent. Message text
.NET <Edc object>.Eh.OnSystemMsgHdlr += new DoPE.OnSystemMsgHdlr(Hdlr)	

8.23 DoPESetOnDebugMsgHdlr

Handler for EDC debug messages.	
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00	
Function declaration	Description
extern unsigned DLLAPI DoPESetOnDebugMsgHdlr DoPE_HANDLE DoPEHdl DoPEOnDebugMsgHdlr Hdlr LPVOID lpParameter	Function returns Error constant (DoPERR_XXXX) DoPE link handle User function to be called for debug messages. User specific pointer.
User function for the OnDebugMsg event:	
unsigned CALLBACK OnDebugMsg DoPE_HANDLE DoPEHdl DoPEOnDebugMsg *DebugMsg LPVOID lpParameter	Function should return 0 (reserved for future versions) DoPE link handle Pointer to received debug message User specific pointer set with DoPESetOnDebugMsgHdlr
typedef struct { unsigned DoPEError unsigned short MsgType double Time wchar_t Text[DEBUG_MSG_TEXT_LEN] } DoPEOnDebugMsg;	DoPERR_NOERROR event is ok. Type of debug message System time the message was sent. Message text
Constants for MsgType DEBUG_MSG DEBUG_MSG_DATA	General debug message Measured values debug message (typically sent every 500ms)
.NET <Edc object>.Eh.OnDebugMsgHdlr += new DoPE.OnDebugMsgHdlr(Hdlr)	

8.24 DoPEThreadPollHdlr

In case, the application program uses an extra thread for communication with DoPE, the DoPEThreadPollHdlr must be called periodically.

1. The application program must install a tread message queue for the thread, using the windows API-function MsgWaitForMultipleObjectsEx.
2. Open communication with a DoPE open function.
3. Install all event handlers you want to process.
4. Use MsgWaitForMultipleObjectsEx function, and wait for tread messages.
5. Call DoPEThreadPollHdlr function if MsgWaitForMultipleObjectsEx stops waiting due to the QS_POSTMESSAGE wait mask. (See Microsoft MSDN for details)

Here a “C” sample program:

```

UINT Thread( LPVOID pParam)
{
    DoPE_HANDLE DoPEHdl = NULL;
    unsigned DoPEErr;
    // create a message queue for the thread
    MsgWaitForMultipleObjectsEx(
        0,                //DWORD nCount,                // number of handles in the handle array
        NULL,             //LPHANDLE pHandles,        // pointer to the object-handle array
        0,                //DWORD dwMilliseconds,    // time-out interval in milliseconds
        QS_POSTMESSAGE,  //DWORD dwWakeMask        // type of input events to wait for
        MWMO_INPUTAVAILABLE //DWORD dwFlags        // wait flags
    );

    DoPEErr = DoPEErr = DoPEOpenFunctionID( ... &DoPEHdl ); // open the communication link
    if( DoPEErr != DoPERR_NOERROR)
        //add errorhandling here
    DoPEErr = DoPESelMachine ( DoPEHdl, 1, UserScale );
    if(DoPEErr != DoPERR_NOERROR)
        //add errorhandling here
    // install the on data handler
    DoPEErr = DoPESetOnDataBlockHdlr ( DoPEHdl, OnData, NULL);
    if(DoPEErr != DoPERR_NOERROR)
        //add errorhandling here
    for ( BOOL Terminate = FALSE; !Terminate; ) // thread loop
    {
        DWORD n;
        #define nCOUNT 0
        switch(n=MsgWaitForMultipleObjectsEx(
            nCOUNT,        //DWORD nCount,                // number of handles in the handle array
            NULL,           //LPHANDLE pHandles,        // pointer to the object-handle array
            1000,          //DWORD dwMilliseconds,    // time-out interval in milliseconds
            QS_POSTMESSAGE, //DWORD dwWakeMask        // type of input events to wait for
            MWMO_INPUTAVAILABLE //DWORD dwFlags        // wait flags
        ))
        {
            case WAIT_TIMEOUT:
                break;
            case WAIT_OBJECT_0 + nCOUNT:
                // New input of the type specified in the dwWakeMask parameter
                // is available in the thread's input queue.
                DoPEErr=DoPEThreadPollHdlr ( DoPEHdl );
                if(DoPEErr!=DoPERR_NOERROR)
                {
                    //add errorhandling here
                    Terminate = TRUE;
                }
                // process thread messages with PeekMessage ( &Msg, NULL, WM_USER, WM_USER, PM_REMOVE )
                break;
        }
    }
    return 0;
}

```

9 Configuration

9.1 DoPEwRdVersion

Read version strings.	
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00	
Function declaration	Description
extern unsigned DLLAPI DoPEwRdVersion DoPE_HANDLE DoPEHdl DoPEwVersion *Version	Function returns Error constant (DoPERR_XXXX) DoPE link handle Version strings.
typedef struct { wchar_t PeInterface[6]; wchar_t Application [13]; wchar_t Subsy[6]; wchar_t SubsyCustVer[6]; wchar_t SubsyCustName[9]; wchar_t Bios[6]; wchar_t HwCtrl[7]; wchar_t PeInterfacePC[6]; wchar_t DpxVer[6]; wchar_t SerialNumber[17]; } DoPEwVersion;	PE interface EDC VERSION "xx.xx" EDC appl. Version "xxxxxxxx.xxx" Subsystem version "xx.xx" Subsystem customer version "xx.xx" Subsystem customer name "xxxxxxxx" EDC BIOS version "xx.xx" EDC controller hardware no "xxxx.x" PE interface PC Version "xx.xx" DPX version "xx.xx" EDC serial number "xxxxxxxxxxxxxxxxxxx" Strings are zero terminated '\0'
.NET <Edc object>.Info.RdVersion(ref DoPE.VersionInfo VersionInfo)	

9.2 DoPEwRdLanguageInfo

Read language info.	
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00	
Function declaration	Description
extern unsigned DLLAPI DoPEwRdLanguageInfo DoPE_HANDLE DoPEHdl DoPEwLanguageInfo *Info	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to storage for the language info.
typedef struct { wchar_t Name0[LANGINFO_NAME_LEN]; wchar_t Name1[LANGINFO_NAME_LEN]; wchar_t Name2[LANGINFO_NAME_LEN]; wchar_t Name3[LANGINFO_NAME_LEN]; } DoPEwLanguageInfo;	Language name 0 (e.g. "English") Language name 1 (e.g. "German") Language name 2 (e.g. "Spanish") Language name 3 (e.g. "French")
.NET <Edc object>.Info.RdLanguageInfo(ref DoPE.LanguageInfo LanguageInfo)	

9.3 Data Acquisition commands

9.3.1 DoPESetDataTransmissionRate

Set time base for data transmission. The default refresh time is defined in the set-up data. You may change it according to your needs. The EDC system time is the minimum possible value.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl double Refresh)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Refresh time s
.NET <Edc object>.Data.SetDataTransmissionRate(Double Refresh)	

9.3.2 DoPESetTime

Set internal EDC time counter to a desired value.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl unsigned sort Mode double Time)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle SETTIME_MODE_IMMEDIATE : set time without a delay SETTIME_MODE_MOVE : set time at the beginning of the next movement command SETTIME_MODE_CYCLE : set time at the beginning of the first cycle of the next cyclic command New value for Time s
.NET <Edc object>.Data.SetTime(DoPE.SETTIME_MODE Mode, Double Time)	

9.3.3 DoPETransmitData

Activate / Deactivate transmission of data. If deactivated NO measuring data will be transmitted to the PC!

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl unsigned short Enable)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle TRUE = Activate transmission FALSE = Deactivate transmission
.NET <Edc object>.Data.TransmitData(bool Enable)	

9.3.4 DoPEFilterTime

Sets the filter time for a measuring channel. Normally filter time of analogue measuring channels is set to 20 ms at with static tests. If this time is increased to a higher value e.g. 100 ms, noise will be decreased. For example, you have a 100 kN load cell and you display 1 N as the smallest value. Now you increase the time to 100 ms and you still display only 1 N steps, your signal noise will be reduced. You may use this effect for a stable load display while no test is running. Set filter time always to 250 ms while no test is running and just before starting a test reduce it to e.g. 20 ms.

Note: The filter time is independent from the refresh time (DoPESetDataTransmissionRate). It is possible to set the refresh time to 20 ms and the filter time to 100ms.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short double	DoPEFiltertime (DoPEHdl SensorNo FilterTime)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Sensor Number Filter time for measuring channels in sec. The maximum time is 1sec. The minimum time is the system time. (see machine set-up data)	s
.NET <Edc object>.Data.FilterTime(DoPE.SENSOR SensorNo, Double FilterTime)			

9.4 Channel supervision

9.4.1 DoPESetCheck

Activate measuring channel supervision.

As support for special condition movement sequences, up to ten measurement channel supervisions are provided, all may be simultaneously active. Each supervision consists of the condition to supervise and the associated action. Supervisions are applicable to all configured measuring channels also for calculated channels if configured. All possible conditions and actions are listed below.

The condition of all active supervisions will be checked every 20 ms.

If a supervision condition hits, the associated action will be executed, an appropriate message transmitted, the current supervision and all other supervisions (which are not marked with the CHK_NOCLEAR bit) are deactivated.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short	DoPESetCheck (DoPEHdl CheckId	Function returns Error constant (DoPERR_ xxxx) DoPE link handle ID of this check, use the CheckId constants (CHK_ID0 ... CHK_ID9) Sensor to be supervised Limit to be supervised Mode, how the limit is detected. (see below) This action will be activated if the check hits. (see below) Control mode for selected action (CTRL_ xxx) Acceleration Maximum speed Deceleration Final destination Pointer to transaction number.	Unit Unit/s ² Unit/s Unit/s ² Unit
unsigned short double unsigned short unsigned short unsigned short double double double double double unsigned short *	SensorNo Limit Mode Action Ctrl Acc Speed Dec Destination lpusTAN)		
Possible modes: CHK_BELOW Action if Value < Limit CHK_ABOVE Action if Value > Limit CHK_PERCENT_MAX Action if MaxValue – Value > MaxValue * Limit / 100 CHK_PERCENT_MIN Action if Value – MinValue > MinValue * Limit / 100 CHK_ABS_MAX Action if MaxValue – Value > Limit CHK_ABS_MIN Action if Value – MinValue > Limit Possible actions: ACTION_HALT HALT with default deceleration ACTION_HALT_A HALT with specified deceleration ACTION_POS Go to a position with default deceleration ACTION_POS_A Go to a position with specified deceleration ACTION_XPCONT Change control mode, go on with current speed ACTION_STOP Switch off closed loop control. (Use this action for machines without position sensor) ACTION_NOACTION No action, only message to host ACTION_SETOL Set command output (only in open loop structure) ACTION_SHALT Immediate Halt in X-head position control ACTION_UP Move Up with specified speed ACTION_DOWN Move Down with specified speed ACTION_DRIVE_OFF Switch drive OFF			
.NET <Edc object>.Check.SetCheck(DoPE.CHK_ID CheckId, DoPE.SENSOR SensorNo, Double Limit, DoPE.CHK_MODE Mode, DoPE.ACTION Action, DoPE.CTRL Ctrl, Double Acc, Double Speed, Double Dec, Double Destination, ref Int16 Tan)			

9.4.2 DoPEClrCheck

Deactivate a measuring channel supervision			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short	DoPEClrCheck (DoPEHdl CheckId)	Function returns Error constant (DoPERR_XXXX) DoPE link handle ID of this check, use the CheckId constants (CHK_ID0 ... CHK_ID9)	
.NET <Edc object>.Check.ClrCheck(DoPE.CHK_ID CheckId)			

9.4.3 DoPESetCheckLimit

Activate limit supervision for a measuring channel. If the measured value is outside UprLimitSet or LwrLimitSet a digital output will be set. This function may be used to prevent opening of clamps under load (see DoPEIOGrip configuration). The allocation of the digital output depends on the device selected in EDC-Set-up. Attention: The values for UprLimitSet, UprLimitReset, LwrLimitReset and LwrLimitSet are calculated with the current Tare and BasicTare. If BasicTare is changed, while limit check is still active, these values are recalculated with the new BasicTare. A new Tare does not affect these values!			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double double double	DoPESetCheckLimit (DoPEHdl SensorNo UprLimitSet UprLimitReset LwrLimitReset LwrLimitSet)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Defines the sensor to be supervised. For all values above, the digital output will be set. For all values below, the digital output will be reset. For all values above, the digital output will be reset For all values below, the digital output will be set.	
.NET <Edc object>.Check.SetCheckLimit(DoPE.SENSOR SensorNo, Double UprLimitSet, Double UprLimitReset, Double LwrLimitReset, Double LwrLimitSet)			

9.4.4 DoPEClrCheckLimit

Deactivate check limit function. The digital output will be set inactive.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE	DoPEClrCheckLimit (DoPEHdl)	Function returns Error constant (DoPERR_XXXX) DoPE link handle	
.NET <Edc object>.Check.ClrCheckLimit()			

9.4.5 DoPESetCheckLimitIO

Set / reset measuring channel supervision IO.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short	DoPESetCheckLimitIO (DoPEHdl Value)	Function returns Error constant (DoPERR_XXXX) DoPE link handle 0 = reset, 1 = Set	
.NET <Edc object>.Check.			

9.5 Controller Parameter

All controller parameters can be modified at any time!

9.5.1 DoPEPosPID

Set parameters for closed loop position controller. Parameters will be applied for the current control mode (LowPressure / HighPressure). Parameters will not be stored in the machine setup.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double double	DoPEPosPID (DoPEHdl MoveCtrl P I D) Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) Proportional gain of the position controller Integration time constant Derivative time constant
Attention: Normally there is a speed control loop with an integration part. In such cases set I in PosPID to ZERO. Otherwise, the control loop has two integrating parts which leads to instability!	
.NET <Edc object>.Ctrl.PosPID(DoPE.CTRL MoveCtrl, Double PosP, Double PosI, Double PosD)	

9.5.2 DoPERdPosPID

Get parameters for closed loop position controller. The currently active parameters are read. (Not from the machine setup)

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned double double double	DoPERdPosPID (DoPEHdl MoveCtrl HighPressure *P *I *D) Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) 0 get parameter for low pressure !=0 get parameter for high pressure Pointer to Proportional gain of the position controller Pointer to Integration time constant Pointer to Derivative time constant
.NET <Edc object>.Ctrl.RdPosPID(DoPE.CTRL MoveCtrl, bool HighPressure, ref Double PosP, ref Double PosI, ref Double PosD)	

9.5.3 DoPEWrPosPID

Set parameters for closed loop position controller. Parameters will not be stored in the machine setup.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned double double double	DoPEWrPosPID (DoPEHdl MoveCtrl HighPressure P I D) Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) 0 set parameter for low pressure !=0 set parameter for high pressure Proportional gain of the position controller Integration time constant Derivative time constant
Attention: Normally there is a speed control loop with an integration part. In such cases set I in PosPID to ZERO. Otherwise, the control loop has two integrating parts which leads to instability!	
.NET <Edc object>.Ctrl.WrPosPID(DoPE.CTRL MoveCtrl, bool HighPressure, Double PosP, Double PosI, Double PosD)	

9.5.4 DoPESpeedPID

Set parameters for closed loop speed controller. Parameters will be applied for the current control mode (LowPressure / HighPressure). Parameters will not be stored in the machine setup.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESpeedPID (DoPE_HANDLE DoPEHdl unsigned short MoveCtrl double P double I double D)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) Proportional gain of the speed controller Integration time constant Derivative time constant
.NET <Edc object>.Ctrl.SpeedPID(DoPE.CTRL MoveCtrl, Double SpeedP, Double SpeedI, Double SpeedD)	

9.5.5 DoPERdSpeedPID

Get parameters for closed loop speed controller. The currently active parameters are read. (Not from the machine setup).

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPERdSpeedPID (DoPE_HANDLE DoPEHdl unsigned short MoveCtrl unsigned HighPressure double *P double *I double *D)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) 0 get parameter for low pressure !=0 get parameter for high pressure Pointer to Proportional gain of the speed controller Pointer to Integration time constant Pointer to Derivative time constant
.NET <Edc object>.Ctrl.RdSpeedPID(DoPE.CTRL MoveCtrl, bool HighPressure, ref Double SpeedP, ref Double SpeedI, ref Double SpeedD)	

9.5.6 DoPEWrSpeedPID

Set parameters for closed loop speed controller. Parameters will not be stored in the machine setup.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPEWrSpeedPID (DoPE_HANDLE DoPEHdl unsigned short MoveCtrl unsigned HighPressure double P double I double D)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) 0 set parameter for low pressure !=0 set parameter for high pressure Proportional gain of the speed controller Integration time constant Derivative time constant
.NET <Edc object>.Ctrl.WrSpeedPID(DoPE.CTRL MoveCtrl, bool HighPressure, Double SpeedP, Double SpeedI, Double SpeedD)	

9.5.7 DoPEFeedForward

Set feed forward parameters. Parameters will be applied for the current control mode (LowPressure / HighPressure). Parameters will not be stored in the machine setup.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double DoPEFeedForward (DoPEHdl MoveCtrl SpeedFFP PosDelay)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) FeedForward for speed controller in % of command speed Delay for Position command
.NET <Edc object>.Ctrl.FeedForward(DoPE.CTRL MoveCtrl, Double SpeedFFP, Double PosDelay)	

9.5.8 DoPERdFeedForward

Get feed forward parameters. The currently active parameters are read. (Not from the machine setup).

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned double double DoPERdFeedForward (DoPEHdl MoveCtrl HighPressure *SpeedFFP *PosDelay)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) 0 get parameter for low pressure !=0 get parameter for high pressure Pointer to FeedForward for speed controller in % of command speed Pointer to Delay for Position command
.NET <Edc object>.Ctrl.RdFeedForward(DoPE.CTRL MoveCtrl, bool HighPressure, ref Double SpeedFFP, ref Double PosDelay)	

9.5.9 DoPEWrFeedForward

Set feed forward parameter. Parameters will not be stored in the machine setup.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned double double DoPEWrFeedForward (DoPEHdl MoveCtrl HighPressure SpeedFFP PosDelay)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) 0 set parameter for low pressure !=0 set parameter for high pressure FeedForward for speed controller in % of command speed Delay for Position command
.NET <Edc object>.Ctrl.WrFeedForward(DoPE.CTRL MoveCtrl, bool HighPressure, Double SpeedFFP, Double PosDelay)	

9.5.10 DoPEOptimizeFeedForward

Optimize feed forward parameter.

For optimization you specify e.g. a cosine cycle and the optimization function will find SpeedFFP and PosDelay values.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short double double double	DoPEOptimizeFeedForward(DoPEHdl MoveCtrl Mode Offset Amplitude Frequency)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_XXX) Waveform: OPTIMIZE_COSINE, OPTIMIZE_TRIANGLE, or OPTIMIZE_RECTANGLE Offset Amplitude Frequency	Unit Unit Hz
.NET <Edc object>.Ctrl.OptimizeFeedForward(DoPE.CTRL MoveCtrl, DoPE.OPTIMIZE Mode, Double Offset, Double Amplitude, Double Frequency, ref Int16 Tan)			

9.5.11 DoPECurrentPID

Sets the current closed loop controller parameters for internal DOLI drive amplifiers.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned long unsigned short unsigned short	DoPECurrentPID (DoPEHdl MoveCtrl P I D)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_XXX) Proportional gain of the current controller Integration time constant Derivative time constant	
.NET <Edc object>.Output.CurrentPID(DoPE.OUT Output, Int32 P, Int16 I, Int16 D)			

9.5.12 DoPEDestWnd

Definitions for destination window.

The closed loop controller supervises the controlled channel. After the command value reaches the final destination, the controlled channel has to reach this position within the specified destination window.

If the actual value reaches the destination window within the specified time a DoPEOnPosMsg event is sent to the application program. The Reached parameter of the event is set to 1.

If it does not reach it, the event is sent with the Reached set to 0.

By default, the destination limit is set to:

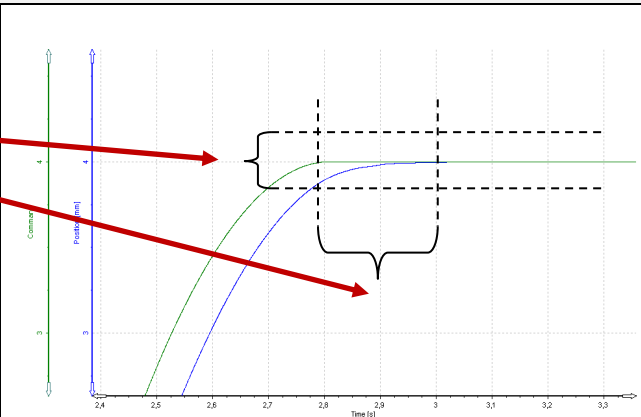
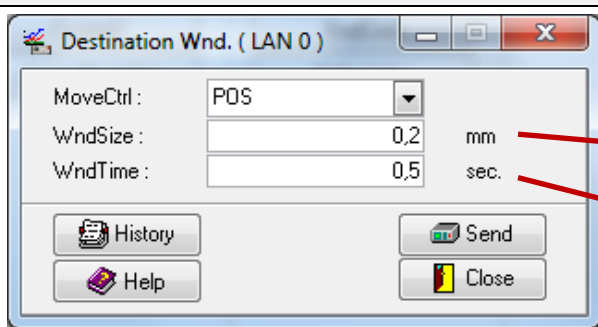
For absolute sensors WndSize = 0.2% of the nominal sensor range.

For incremental position sensors WndSize = 0.05 mm

For incremental extension sensors WndSize = 0.005 mm

For all sensors WndTime = 0.5 s.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00



Function declaration

```
extern unsigned DLLAPI DoPEDestWnd (
    DoPE_HANDLE DoPEHdl
    unsigned short MoveCtrl
    double WndSize
    double WndTime)
```

Description

Function returns Error constant (DoPERR_XXXX)
DoPE link handle
Control mode (CTRL_XXX)
Size of destination window
Time for destination window

Unit
s

.NET <Edc object>.Ctrl.DestWnd(DoPE.CTRL MoveCtrl, Double WndSize, Double WndTime)

9.5.13 DoPESft

Definitions of limits supervised by software (softend).

If Reaction is defined as REACT_ACTION all movement commands will limit destinations to this softends.

In contrast to the range limits of the measurement channels, softends are working limits that can be changed at any time.

By default, all softends are set to the range limits and REACT_STATUS.

The state of the softends is maintained in UpperSft and LowerSft (see default measuring data record).

Attention: The values for UpperSft and LowerSft are calculated with the current Tare and BasicTare.

If BasicTare is changed, these values are recalculated with the new BasicTare.

A new Tare does not affect these values!

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double unsigned short DoPESft(DoPEHdl MoveCtrl UpperSft LowerSft Reaction)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Control mode (CTRL_ xxx) Upper soft limit Lower soft limit REACT_STATUS No reaction if softend is reached, only status bits are set REACT_ACTION Halt in X-head position control if softend is reached	Unit Unit
.NET <Edc object>.Ctrl.Sft(DoPE.CTRL MoveCtrl, Double UpperSft, Double LowerSft, DoPE.REACT Reaction)		

9.5.14 DoPECtrlSpeedTimeBase

Define maximum time base for speed calculation.

When changing control mode, the current speed of the new control channel is used as the start speed after control mode was changed. Normally this speed is determined by differentiation by two following position values. In case of a small resolution of the transducer for the new control mode, the speed values are very small and thus also inaccurate. Using this command, the time base can be increased and thus a higher accuracy for speed calculation can be archived. Use this command only if speed is not changing too fast otherwise with a big time base the calculated speed is incorrect. This time base exclusively works for the determination of the start speed when control mode is changed. It has no effect for speed calculation for the digital speed controller.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description	Unit
extern unsigned DLLAPI DoPE_HANDLE double) DoPECtrlSpeedTimeBase(DoPEHdl Time	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Time base in for speed calculation Note: for EdcApp versions up to 9140.015: internally only 1, 2, 4, 8, 16, 32, 64, 128 x position controller time base will be used!	s
.NET <Edc object>.Ctrl.CtrlSpeedTimeBase(Double Time)		

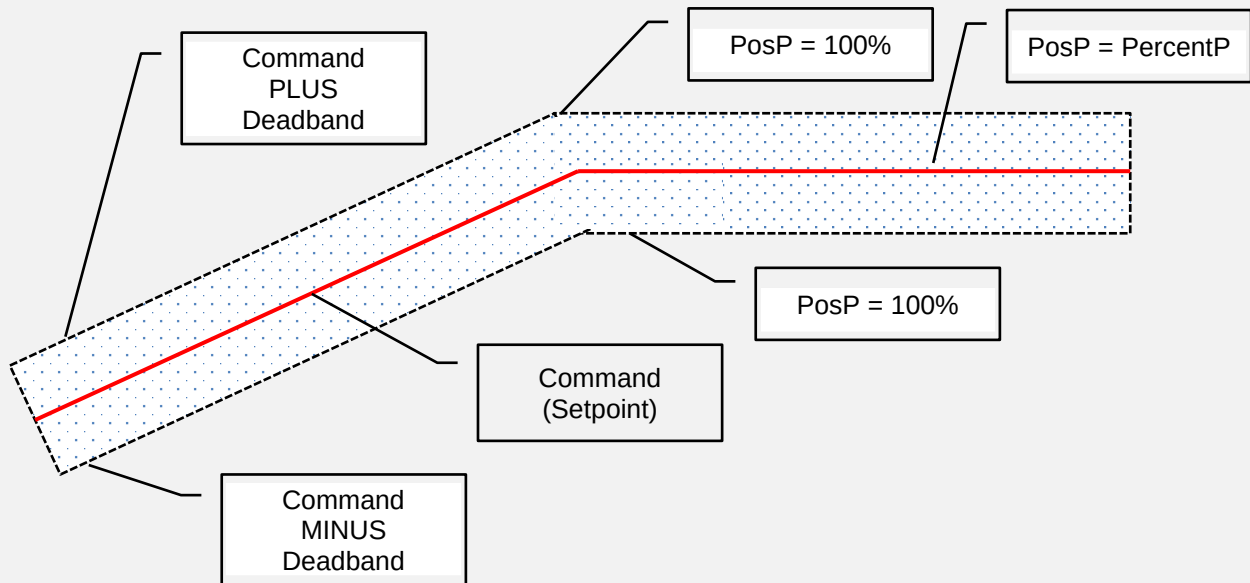
9.5.15 DoPEDeadbandCtrl

Set parameter for error deadband controller.

A deadband can be used to reduce or eliminate a limit cycle due to friction.

Inside an area "setpoint plus Deadband and setpoint minus Deadband" the gain of position controller is continuously reduced from PosP = 100% to a minimum value (PercentP of PosP = 100%).

Note: a well-adjusted feed forward is needed for a good function of Deadband control!



Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short double unsigned short	DoPEDeadbandCtrl (DoPEHdl MoveCtrl Deadband PercentP)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Control mode (CTRL_xxx) Width of error deadband around setpoint Smallest P inside deadband PercentP range is 0..100% 100% disables error deadband controller 0% sets the PosP to the smallest value (1)	[%PosP]
.NET <Edc object>.Ctrl.DeadbandCtrl(DoPE.CTRL MoveCtrl, Double Deadband, Int16 PercentP)			

9.5.16 DoPERdCtrlParameter

Read closed loop controller parameter			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short DoPECtrlParameter	DoPERdCtrlParameter (DoPEHdl MoveCtrl *CtrlParameter)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Control mode (CTRL_XXX) Pointer for DoPECtrlParameter structure	
typedef struct Controller parameter			
// Controller parameter			
double	PosP	Pos. contr. P: gain	[No]
double	PosI	Pos. contr. I: time constant	[No]
double	PosD	Pos. contr. D: time constant	[No]
double	SpeedP	Speed contr. P: gain	[No]
double	SpeedI	Speed contr. I: time constant	[No]
double	SpeedD	Speed contr. D: time constant	[No]
double	SpeedFFP	Speed feed forward	[No]
double	PosDelay	Delay for Command	[No]
double	Acceleration	default acceleration	[Unit/s ²]
double	Speed	speed limit	[Unit/s]
double	Deviation	Max. deviation of controller	[Unit]
unsigned short	DevReaction	Reaction if deviation exceeded	[No]
double	DestinationWnd	Size of 1/2 destination window	[Unit]
double	DestinationTime	Time until controlled channel must reach destination window	s
double	UpperSoftEnd	Upper softend	[Unit]
double	LowerSoftEnd	Lower softend	[Unit]
unsigned short	SoftEndReaktion	Reaction if softend is reached	[No]
// numerical limitations for acceleration and speed parameters			
double	MinAcceleration	minimum acceleration	[Unit/s ²]
double	MaxAcceleration	maximum acceleration	[Unit/s ²]
double	MinDeceleration	minimum deceleration	[Unit/s ²]
double	MaxDeceleration	maximum deceleration	[Unit/s ²]
double	MinSpeed	minimum speed	[Unit/s]
double	MaxSpeed	maximum speed	[Unit/s]
// error dead band controller parameter			
double	Deadband	Error deadband around setpoint	[Unit]
unsigned short	PercentP	Smallest P inside deadband	[%PosP]
// dither parameter			
double	DitherFrequency	Dither frequency	[Hz]

double	DitherAmplitude	Dither amplitude	[%]
} DoPECtrlParameter			
.NET <Edc object>.Ctrl.RdCtrlParameter(DoPE.CTRL MoveCtrl, ref DoPE.CtrlParameter CtrlParameter)			

9.5.17 DoPESetNominalAccSpeed

Set nominal values for the position generator.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE Unsigned short double double	DoPESetNominalAccSpeed (DoPEHdl MoveCtrl Acc Speed)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Control mode (CTRL_ xxx) Acceleration Nominal speed	
.NET <Edc object>.Ctrl.SetNominalAccSpeed(DoPE.CTRL MoveCtrl, Double Acc, Double Speed)			

9.6 Tare functions

DoPE offers two different tare functions, Tare and BasicTare.

The **BasicTare** functions should be used for set-up the machine configuration like changing grips. The Basic-Tare value is stored in the machine set-up inside EDC. After switching mains power off / on, the Basic-Tare value is still valid.

The **DoPETare**, **DoPETareSetValue** and **DoPETareSetDisplay** functions can be used at any time. The tare value will be lost after reset.

9.6.1 DoPE(Basic)TareSetValue

Set (basic) tare of the measuring channel. Note: DoPEBasicTareSetValue clears the ordinary tare value			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short double	DoPEBasicTareSetValue (DoPEHdl SensorNo Value)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor number Value will be subtracted This is useful to compensate the weight of grips.	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double	DoPETareSetValue (DoPEHdl SensorNo Value)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor number Value will be subtracted This is useful to compensate the weight of grips.	Unit
.NET <Edc object>.Tare.BasicTareSetValue(DoPE.SENSOR SensorNo, Double Value) <Edc object>.Tare.TareSetValue(DoPE.SENSOR SensorNo, Double Value)			

9.6.2 DoPE(Basic)TareSetDisplay

Set (basic) tare of the measuring channel. Note: DoPEBasicTareSetDisplay clears the ordinary tare value			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short double	DoPEBasicTareSetDisplay (DoPEHdl SensorNo Value)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor number Value represents the desired measuring value. This is useful to set crosshead position for systems with encoder.	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double	DoPETareSetDisplay (DoPEHdl SensorNo Value)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor number Value represents the desired measuring value. This is useful to set crosshead position for systems with encoder.	Unit
.NET <Edc object>.Tare.BasicTareSetDisplay(DoPE.SENSOR SensorNo, Double Value) <Edc object>.Tare.TareSetDisplay(DoPE.SENSOR SensorNo, Double Value)			

9.6.3 DoPE(Basic)Tare

Enable / disable (basic) tare of the measuring channel.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned	DoPEBasicTare (DoPEHdl SensorNo Enable)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Sensor number != 0: activates basic tare, like DoPEBasicTareSetDisplay(..., 0) == 0: clears basic tare, like DoPE(Basic)TareSetValue(..., 0)	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned	DoPETare (DoPEHdl SensorNo Enable)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Sensor number != 0: activates tare, like DoPETareSetDisplay(..., 0) == 0: clears tare, like DoPETareSetValue(..., 0)	Unit
.NET <Edc object>.Tare.BasicTare(DoPE.SENSOR SensorNo, bool Enable) <Edc object>.Tare.Tare(DoPE.SENSOR SensorNo, bool Enable)			

9.6.4 DoPE(Basic)TareGetValue

Read (basic) tare value of the measuring channel.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short double	DoPEGetBasicTareValue (DoPEHdl SensorNo * Value	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Sensor number Pointer for basic tare value	Unit
extern unsigned DLLAPI DoPE_HANDLE unsigned short double	DoPEGet(Basic)TareValue (DoPEHdl SensorNo * Value	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Sensor number Pointer for tare value	Unit
.NET <Edc object>.Tare.BasicTareGetValue(DoPE.SENSOR SensorNo, ref Double Value) <Edc object>.Tare.TareGetValue(DoPE.SENSOR SensorNo, ref Double Value)			

9.7 Reference signal handling of incremental sensors

Incremental sensors like encoder or linear gauges may use the build in reference to set the counter-value. Encoders have one reference pulse per revolution. Linear gauges have either one pulse at a certain position, or so-called distance coded references.

Currently, DoPE supports the reference pulse to set the counter to a certain value.

9.7.1 DoPESetRefSignalMode

After the reference pulse occurred, the counter value is stored in a Hardware register. This command defines what to do with this value:

1. Ignore it, don't send any message. (REFSIG_NON)
2. Send a message after every occurrence of the reference pulse. (REFSIG_ON)
3. Send a message after the next occurrence of the reference pulse. (REFSIG_ONCE)

With the message (DoPERefSignalMsg) you get the exact position of the reference pulse.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
<pre>extern unsigned DLLAPI DoPESetRefSignalMode (DoPE_HANDLE DoPEHdl unsigned short SensorNo unsigned short Mode unsigned short * lpusTAN)</pre>	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Sensor Number Mode reference signal messages will be reported: REFSIG_NON: never REFSIG_ON: always REFSIG_ONCE: only once Pointer to transaction number.
<pre>typedef struct DoPERefSignalMsg/ { unsigned short MsgId; double Time; unsigned short SensorNo; double Position; } DoPERefSignalM;</pre>	ID of message System time for the message Control mode of position Position
.NET <Edc object>.Tare.SetRefSignalMode(DoPE.SENSOR SensorNo, DoPE.REFSIG_MODE Mode, ref Int16 Tan)	

9.7.2 DoPESetRefSignalTare

Set the measuring channel to the tare value at the next occurrence of the reference signal.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
<pre>extern unsigned DLLAPI DoPESetRefSignalTare (DoPE_HANDLE DoPEHdl unsigned short SensorNo unsigned short Mode double Tare unsigned short * lpusTAN)</pre>	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Sensor Number REFSIG_TARE: set the measuring channel to the tare value REFSIG_NON: don't affect the measuring channel Tare value for the measuring channel. This tare value will be stored in EDC's nonvolatile basic tare memory. This function clears the current basic tare and ordinary tare the value at the next occurrence of the reference signal. Pointer to transaction number.
.NET <Edc object>.Tare.SetRefSignalTare(DoPE.SENSOR SensorNo, DoPE.REFSIG_TARE Mode, Double Value, ref Int16 Tan)	

9.8 Calculated measuring channels

9.8.1 DoPEConfPeakValue

Configure peak values to the measuring data record.

The peak values are detected by the EDC. They may be transmitted to PC instead of an unused measuring channel. With this command a peak value of a measuring channel may be configured to a measuring channel in the data record. The peak value detection is available for cyclic commands only. The peak values are refreshed after each cycle. These calculated channels are not allowed as the command channel in DoPEExt2 or DoPEFDPot command!

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short unsigned short	DoPEConfPeakValue (DoPEHdl SensorNo DataAddress Mode)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Sensor number to be supervised Position of Min/Max value in the measuring data record One of the DoPE_PEAK_VALUE_... modes (see below)	
Possible modes: DoPE_PEAK_VALUE_OFF disable the Min/Max detection the measuring data record at position DataAddress is restored to the sensor specified in the setup DoPE_PEAK_VALUE_MIN enable Min value detection for the sensor SensorNo and report the results in the measuring data record at position DataAddress DoPE_PEAK_VALUE_MAX enable Max value detection for the sensor SensorNo and report the results in the measuring data record at position DataAddress			
.NET <Edc object>.Cmc.ConfPeakValue(DoPE.SENSOR SensorNo, DoPE.SENSOR DataAddress, DoPE.PEAK_VALUE Mode)			

9.8.2 DoPEMc2Output

Output of a measured value to an analogue output channel. Any measured channel may be scaled and via a DAC converted to an analogue signal.

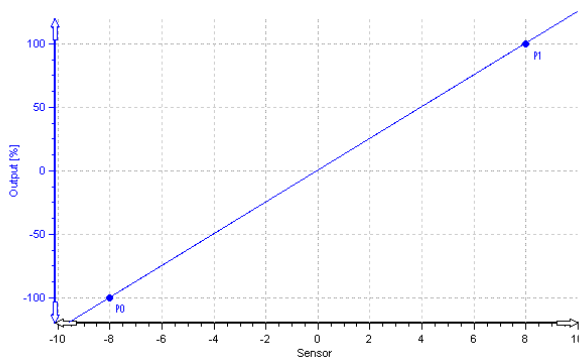
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short unsigned short double double	DoPEMc2Output (DoPEHdl Mode SensorNo Output SensorPoints[3] OutputPoints[3])	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Mode flags (see below) Sensor number Number of analogue output channel Sensor points table Output points table	Unit %
.NET <Edc object>.Output. Mc2Output(DoPE.MC2OUT_MODE Mode, DoPE.SENSOR SensorNo, DoPE.OUT Output, Double[] SensorPoint, Double[] OutputPoint)			

Definition of Mode:

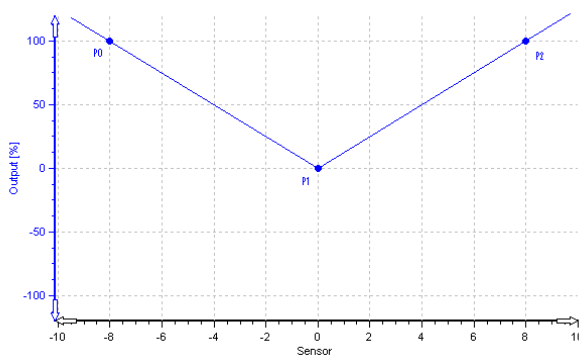
MC2OUT_MODE_OFF
MC2OUT_MODE_2POINTS

Stop output to this analogue output channel.
Output definition by 2 points.



MC2OUT_MODE_3POINTS

Output definition by 3 points.



9.9 Sensor Correction

9.9.1 DoPESetSensorCorrection

Set sensor correction table.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPESetSensorCorrection(DoPE_HANDLE DoPEHdl Unsigned short CalculatedSensor DoPESensorCorrection *CorrTable)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor number of the corresponding Calculated Measuring Channel (with formula F_SensorCorrection) Pointer to sensor correction table.	
<pre> SENSOR_CORR_MAX 32 /* maximum number of sensor correction points */ typedef struct { DoPE_HANDLE DoPEHdl; unsigned CorrNo; /* Number of valid entries */ double S1Correction [SENSOR_CORR_MAX]; /* Correction values for S1 */ double S2Value [SENSOR_CORR_MAX]; /* S2 values (must be in ascending order) */ } DoPESensorCorrection; </pre>		
.NET <Edc object>.Corr.SetSensorCorrection(DoPE.SENSOR CalculatedSensor, ref DoPE.SensorCorrectionTable SensorCorrectionTable)		

9.9.2 DoPESetStiffnessCorrection

Set stiffness correction table.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPESetStiffnessCorrection(DoPE_HANDLE DoPEHdl DoPEStiffnessCorrection *CorrTable)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to stiffness correction table.	
<pre> STIFF_CORR_MAX 32 /* maximum number of stiffness correction points */ typedef struct { DoPE_HANDLE DoPEHdl; unsigned CorrNo; /* Number of valid entries */ double Load[STIFF_CORR_MAX]; /* Load value */ double Deformation[STIFF_CORR_MAX]; /* Machine deformation at load value */ } DoPEStiffnessCorrection; </pre>		
.NET <Edc object>.Corr.SetStiffnessCorrection(ref DoPE.StiffnessCorrectionTable StiffnessCorrectionTable)		

9.10 Serial Sensors

9.10.1 DoPEWrSensorMsg

Write a message to a sensor. The message is not interpreted by DoPE!
Maximum message length is 80 Bytes.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPEWrSensorMsg(DoPE_HANDLE DoPEHdl unsigned short SensorNo void * Buffer unsigned Length)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor Number Pointer to message to transmit Message length in Byte's.
.NET <Edc object>.Sensen.WrSensorMsg(DoPE.SENSOR SensorNo, String Buffer)	

9.10.2 DoPESerialSensorDef

Definition for serial sensor.
A message for a serial sensor will be transmitted after:

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESerialSensorDef (DoPE_HANDLE DoPEHdl unsigned short SensorNo double Timeout unsigned short MaxLength char EndChar1 char EndChar2 WORD EndCharMode)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor Number Timeout, 0 = no timeout Transmit after Number of bytes received (must be <= SERSEN_TRANSFER) First end character Second end character Mode for detection end character (see below)
SERSEN_ENDCHAR_NO SERSEN_ENDCHAR_1 SERSEN_ENDCHAR_1_OR_2 SERSEN_ENDCHAR_1_AND_2 SERSEN_ENDCHAR_1_PLUS1	No Endcharacter active, message will be transmitted after timeout, or MaxLength Detect only EndChar1 Detect EndChar1 or EndChar2 Detect Sequence EndChar1 + EndChar2 Detect EndChar1 plus one Character
.NET <Edc object>.Sensen.SerialSensorDef(DoPE.SENSOR SensorNo, Double Timeout, Int16 MaxLength, Byte EndChar1, Byte EndChar2, DoPE.SERSEN_ENDCHAR EndCharMode)	

9.10.3 DoPESetSerialSensor

Set value (e.g. Temperature) for serial channel.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPESetSerialSensor (DoPE_HANDLE DoPEHdl unsigned short SensorNo unsigned short Mode double Value double Speed)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor Number SERSEN_SET_COMMAND: Set Command value SERSEN_SET_FEEDBACK: Set Feedback value Value to set Speed for ramp
.NET <Edc object>.Sensen.SetSerialSensor(DoPE.SENSOR SensorNo, DoPE.SERSEN_SET_VALUE Mode, Double Value, Double Speed)	

9.10.4 DoPESetSerialSensorTransparent

Set serial channel to transparent mode or the previously selected protocol.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration		Description
extern unsigned DLLAPI DoPESetSerialSensorTransparent (DoPE_HANDLE DoPEHdl unsigned short SensorNo unsigned short Mode)		Function returns Error constant (DoPERR_XXXX) DoPE link handle Sensor Number SERSEN_SET_PROTOCOL: Set sensor to the protocol (Set-up) SERSEN_SET_TRANSPARENT: Set sensor to transparent mode
.NET <Edc object>.Sersen. SetSerialSensorTransparent(DoPE.SENSOR SensorNo, DoPE.SERSEN_SET_PROTOCOL Mode)		

9.11 Debug Messages

For service purposes it is sometimes necessary to record the debug output of the EDC and send it to the DOLI support team. To facilitate this, the OnDebugMsg Event Handler were implemented.

9.11.1 DoPEDebugMsgEnable

Enable or disable debug messages.

Attention: Unfortunately, the debug messages influence the runtime behavior of the application and should be turned off by default and turned on only in e.g. a remote support session

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPEDebugMsgEnable(DoPE_HANDLE DoPEHdl unsigned Enable)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle !=0 enables =0 disables debug messages.
.NET <Edc object>.Debug.DebugMsgEnable(bool Enable)	

9.11.2 DoPESendDebugCommand

Send a command to the debug interface.

The “?” command prints a list of the available commands to the debug interface.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPEDebugMsgEnable(DoPE_HANDLE DoPEHdl char *Text)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Pointer to zero terminated text to transmit.
.NET <Edc object>.Debug.SendDebugCommand(String Text)	

10 Input / Output-Commands

10.1 Analogue output

10.1.1 DoPESetOutput

Set an analogue output channel. The output channel must be assigned in EDC set-up.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short double DoPESetOutput (DoPEHdl Output Value)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Number of analogue output channel New value of output channel in % of max. value %
.NET <Edc object>.Output.SetOutput(DoPE.OUT Output, Double Value)	

10.1.2 DoPESetOutChannelOffset

Set an analogue output channel offset.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short double DoPESetOutChannelOffset(DoPEHdl Output Offset)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Number of analogue output channel New Offset value of output channel in % of max. value %
.NET <Edc object>.Output.SetOutChannelOffset(DoPE.OUT Output, Double Offset)	

10.1.3 DoPESetDither

Set an analogue output channel dither. Some DOLI output amplifier like D03I, D16I, D32I have a digital dither generator function. The dither amplitude and frequency can be set by this command.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned short double double DoPESetDither (DoPEHdl Output Frequency Amplitude)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Number of analogue output channel Dither frequency in Hz Dither amplitude in % of nominal output current Hz %
.NET <Edc object>.Ctrl.SetDither(DoPE.CTRL MoveCtrl, Double Frequency, Double Amplitude)	

10.1.4 DoPEOfflineActionOutput

Definition of an action for an initialized analogue output channel after EDC has detected offline. With this command, the PC software can specify the state of any analogue output channel, after the communication between PC and EDC was disturbed or interrupted(OFFLINE). EDC-Software checks automatically communication with PC, and if PC does not answer for a period of 2 Seconds, EDC regards PC to be offline. This may happen if the line between PC and EDC was disconnected, or PC program has crashed. If PC program terminates regularly by using DoPECloseLink command, EDC will reinitialize, and DoPEOfflineActionBitOutput command has no effect.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short double	DoPEOfflineActionOutput (DoPEHdl Output Mode Value)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Number of analogue output channel. If closed loop control, Output channel 0 cannot be selected! DO_NOTHING: Don't modify this digital output USE_INIT_VALUE: Use Initial value from set-up USE_VALUE: Use defined value Output value used in USE_VALUE mode in % of max. value	
.NET <Edc object>.Output.OfflineActionOutput(DoPE.OUT Output, DoPE.OFFLINE_ACTION Mode, Double Value)			

10.2 Bit I/O-Commands

These commands can be used to read or write digital I/O's.

Reading digital inputs is normally not necessary since all configured inputs are automatically read and transferred in the measuring data record.

General digital bit outputs are set with the command DoPESetBit. The used outputs have to be assigned in the EDC setup.

For dedicated output bits, DoPE supplies dedicated functions like DoPEBeep to activate/deactivate the beep at an EDC.

For Digital input/output bits that are not often used, DoPE provides functions to read or write these devices without having initialized it (not assigned in the EDC setup).

10.2.1 DoPESetBit

Set, Reset, Flash Bits. This command is used to set any output bit on a digital output device. The devices are specified in the set-up data.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short unsigned short unsigned short	DoPESetBit (DoPEHdl BitOutputNo SetB ResB FlashB)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Number of bit output device These bits will be set These bits will be reset These bits will flash	
The three data words will be processed in the following sequence (important with conflicting data): 1. Flash bits. 2. Reset bits. 3. Set bits.			
.NET <Edc object>.Io.SetBit(DoPE.BOUT BitOutput, Int16 SetB, Int16 ResB, Int16 FlashB)			

10.2.2 DoPECaOut

Activate / deactivate calibration output on EDC. The calibration contact may be used to trigger a reference measuring system during load calibration (see also DoPEIOMisc).

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPECaOut (DoPE_HANDLE DoPEHdl unsigned short Cal)	Function returns Error constant (DoPERR_xxxx) DoPE link handle !=0 activate Calibration output 0 deactivate Calibration output
.NET <Edc object>.Io.CalOut(bool cal)	

10.2.3 DoPEBeep

Activate / deactivate beep on EDC.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPEBeep (DoPE_HANDLE DoPEHdl unsigned short Beep)	Function returns Error constant (DoPERR_xxxx) DoPE link handle !=0 activate Beep 0 deactivate Beep Pointer to transaction number (not for Sync. version).
.NET <Edc object>.Io.Beep(bool beep)	

10.2.4 DoPEBypass

Activate/Deactivate bypass output bits at EDC.

Function declaration	Description
extern unsigned DLLAPI DoPEBypass (DoPE_HANDLE DoPEHdl unsigned short Bypass)	Function returns Error constant (DoPERR_xxxx) DoPE link handle !=0 activates bypass output 0 deactivates bypass output
.NET <Edc object>.Io.Bypass(bool bypass)	

10.2.5 DoPERdBitInput

Read a digital input device.

Note: This command will also work on not initialized I/O-devices.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE WORD	DoPERdBitInput (DoPEHdl Connector	Function returns Error constant (DoPERR_XXXX) DoPE link handle Connector number, for the I/O device Use connector constants from header file like "CON_X2"	
WORD	*Value	Data pointer to store result	
WORD	*lpusTAN)	Pointer to transaction number (not for Sync. version).	
.NET <Edc object>.Io.RdBitInput(DoPE.CONNECTOR Connector, ref Int16 Value)			

10.2.6 DoPEWrBitOutput

Write to a digital output device.

Note: This command will also work on not initialized I/O-devices.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE WORD	DoPEWrBitOutput(DoPEHdl Connector	Function returns Error constant (DoPERR_XXXX) DoPE link handle Connector number, for the I/O device Use connector constants from header file like "CON_X2"	
WORD	Value)	Data to be written to the output device.	
.NET <Edc object>.Io.WrBitOutput(DoPE.CONNECTOR Connector, Int16 Value)			

10.2.7 DoPEOfflineActionBitOutput

Definition of an action for an initialized digital output device after EDC has detected offline.

With this command, the PC-Software can specify the state of any digital output, after the communication between PC and EDC was disturbed or interrupted (OFFLINE).

EDC-Software checks automatically communication with PC, and if PC does not answer for a period of 2 Seconds, EDC regards PC to be offline. This may happen if the line between PC and EDC was disconnected, or PC-Program has crashed.

If PC-Program terminates regularly by using DoPECloseLink command, EDC will reinitialize, and DoPEOfflineActionBitOutput command has no effect.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned short unsigned short	DoPEOfflineActionBitOutput (DoPEHdl BitOutputNo Mode	Function returns Error constant (DoPERR_XXXX) DoPE link handle Number of bit output device DO_NOTHING: Don't modify this digital output USE_INIT_VALUE: Use Initial value from set-up after offline USE_VALUE: Use defined value after offline Use defined value after offline.	
WORD	Value)		
.NET <Edc object>.Io.OfflineActionBitOutput(DoPE.BOUT BitOutput, DoPE.OFFLINE ACTION Mode, Int16 Value)			

10.3 IO Signals

Following functions are supported:

- Control of grips
- Control of extensometer
- Fixed XHead adjustment
- High/Low pressure selection
- Guard option

All functions must be configured in the EDC set-up. The control functions may be activated via keys on RMC, or via commands from PC.

Each control function uses digital I/O with certain functions. The I/O bits for one function must be configured as a whole block to any I/O device. The first bit of a block may be assigned to any bit of the I/O device.

The number of reserved In- and Output bits for each block are identical, even if not all are currently used. The Miscellaneous Input- and Output-bits may be defined separately.

Note: For more detailed information please refer to EDC Installation Manual.

10.3.1 DoPEIOGripEnable

Enable or disable grip IO handling.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE Unsigned WORD	DoPEIOGripEnable (DoPEHdl Enable * lpusTAN)	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 enables 0 disables grip IO handling Pointer to transaction number (not for Sync. version).	
.NET <Edc object>.IoSignal.IOGripEnable(bool Enable)			

10.3.2 DoPEIOGripSet

Perform a grip action.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned unsigned	DoPEIOGripSet (DoPEHdl Grip Action)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Grip to use (use IO_GRIP_ defines) Action to perform (use IO_GRIP_ACTION_ defines)	
Constants for Grip IO_GRIP_UPPER IO_GRIP_LOWER IO_GRIP_BOTH Constants for Action IO_GRIP_ACTION_OPEN IO_GRIP_ACTION_CLOSE IO_GRIP_ACTION_HIGH_PRESSURE1 IO_GRIP_ACTION_HIGH_PRESSURE2			
.NET <Edc object>.IoSignal.IOGripSet(DoPE.IO_GRIP Grip, DoPE.IO_GRIP_ACTION Action)			

10.3.3 DoPEIOExtEnable

Enable or disable extensometer IO handling.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPEIOExtEnable (DoPEHdl Enable)	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 enables 0 disables extensometer IO handling	
.NET <Edc object>.IoSignal.IOExtEnable(bool Enable)			

10.3.4 DoPEIOExtSet

Perform an extensometer action.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned unsigned	DoPEIOExtSet(DoPEHdl Ext Action)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Grip to use (use IO_EXT_ defines) Action to perform (use IO_EXT_ACTION_ defines)	
IO_EXT_UPPER IO_EXT_LOWER IO_EXT_BOTH IO_EXT_ACTION_OPEN IO_EXT_ACTION_CLOSE			
.NET <Edc object>.IoSignal.IOExtSet(DoPE.IO_EXT Ext, DoPE.IO_EXT_ACTION Action)			

10.3.5 DoPEIOFixedXHeadEnable

Enable or disable fixed cross head IO handling.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPEIOFixedXHeadEnable (DoPEHdl Enable)	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 enables 0 disables fixed cross head IO handling	
.NET <Edc object>.IoSignal.IOFixedXHeadEnable(bool Enable)			

10.3.6 DoPEIOFixedXHeadSet

Move fixed cross head.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPEIOFixedXHeadSet (DoPEHdl Direction)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Direction of movement (use MOVE_HALT, MOVE_UP, MOVE_DOWN defines)	
.NET <Edc object>.IoSignal.IOFixedXHeadSet(DoPE.MOVE Direction)			

10.3.7 DoPEIOHighPressureEnable

Enable or disable high pressure IO handling.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPEIOHighPressureEnable (DoPEHdl Enable)	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 enables 0 disables high pressure IO handling	
.NET <Edc object>.IoSignal.IOHighPressureEnable(bool Enable)			

10.3.8 DoPEIOHighPressureSet

Set high or low pressure.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPEIOHighPressureSet (DoPEHdl Enable)	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 selects high pressure 0 selects low pressure	
.NET <Edc object>.IoSignal.IOHighPressureSet(bool HighPressure)			

10.3.9 DoPEIOGuardSetLimitedMode

Activate or deactivate limited mode.			
Minimum requirements: EDCi with EDCiApp 9149.015, DoPE 10.16			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPEIOGuardSetLimitedMode (DoPEHdl LimitedMode)	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 activates 0 deactivates limited mode	
.NET <Edc object>.IoSignal.GuardSetLimitedMode (bool LimitedMode)			

10.3.10 DoPEIOGuardSetUnlockRequest

Activate or deactivate guard unlock request.			
Minimum requirements: EDCi with EDCiApp 9149.019, DoPE 10.18			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPEIOGuardSetUnlockRequest (DoPEHdl UnlockRequest)	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 activates 0 deactivates guard unlock request	
.NET <Edc object>.IoSignal.GuardSetUnlockRequest (bool UnlockRequest)			

10.3.11 DoPEIOGuardSetRangeLimit

Set upper/lower range limit and overrun tolerance.			
Minimum requirements: EDCi with EDCiApp 9149.015, DoPE 10.16			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE double double double	DoPEIOGuardSetRangeLimit (DoPEHdl UpperRangeLimit LowerRangeLimit RangeLimitTolerance)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Upper range limit Lower range limit Range limit overrun tolerance	
.NET <Edc object>.IoSignal.GuardSetRangeLimit (...)			

10.3.12 DoPEIOGuardGetRangeLimit

Get upper/lower range limit and overrun tolerance.

Minimum requirements: EDCi with EDCiApp 9149.015, DoPE 10.16

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE double double double DoPEIOGuardGetRangeLimit (DoPEHdl *UpperRangeLimit *LowerRangeLimit *RangeLimitTolerance)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer for Upper range limit Pointer for Lower range limit Pointer for Range limit overrun tolerance
.NET <Edc object>.IoSignal.GuardGetRangeLimit (...)	

10.3.13 DoPEIOGuardSetMuteGuard

Mutes the guard monitoring. If activated, the guard is not monitored.

Minimum requirements: EDCi with EDCiApp 9149.019, DoPE 10.18

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPEIOGuardSetMuteGuard (DoPEHdl MuteGuard)	Function returns Error constant (DoPERR_XXXX) DoPE link handle IO_GUARD_MUTE_OFF 0 Muting off IO_GUARD_MUTE_SINGLE 1 Muting active for one opening IO_GUARD_MUTE_CONTINUOUS 2 Muting active continuously
.NET <Edc object>.IoSignal.GuardSetMuteGuard (int32 MuteGuard)	

10.3.14 DoPEIOGuardGetMuteGuard

Get the state of the guard closed muting.

Minimum requirements: EDCi with EDCiApp 9149.019, DoPE 10.18

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPEIOGuardGetMuteGuard (DoPEHdl *MuteGuard)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer for the state of the guard muting
.NET <Edc object>.IoSignal.GuardGetMuteGuard (ref int32 MuteGuard)	

10.3.15 DoPEIOGuardSetMuteSpeedLimit

Mutes the speed limit monitoring. If "Low Pressure Action Enable" is enabled, it will also be muted.
(Only if the "SpeedLimitEnable" is enabled in the machine setup)

Minimum requirements: EDCi with EDCiApp 9149.019, DoPE 10.18

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPEIOGuardSetMuteSpeedLimit(DoPEHdl MuteSpeedLimit)	Function returns Error constant (DoPERR_XXXX) DoPE link handle IO_GUARD_MUTE_OFF 0 Muting off IO_GUARD_MUTE_SINGLE 1 Muting active for one opening IO_GUARD_MUTE_CONTINUOUS 2 Muting active continuously
.NET <Edc object>.IoSignal.GuardSetMuteSpeedLimit (int32 MuteSpeedLimit)	

10.3.16 DoPEIOGuardGetMuteSpeedLimit

Get the state of the speed limit monitoring muting.		
Minimum requirements: EDCi with EDCiApp 9149.019, DoPE 10.18		
Function declaration	Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPEIOGuardGetMuteSpeedLimit (DoPEHdl *MuteSpeedLimit)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer for the state of the speed limit muting	
.NET <Edc object>.IoSignal.GuardGetMuteSpeedLimit (ref int32 MuteSpeedLimit)		

10.3.17 DoPEIOGuardSetMuteRangeLimit

Mutes the range limit monitoring. If "Low Pressure Action Enable" is enabled, it will also be muted. (Only if the "RangeLimitEnable" is enabled in the machine setup)		
Minimum requirements: EDCi with EDCiApp 9149.019, DoPE 10.18		
Function declaration	Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPEIOGuardSetMuteRangeLimit (DoPEHdl MuteRangeLimit)	Function returns Error constant (DoPERR_XXXX) DoPE link handle IO_GUARD_MUTE_OFF 0 Muting off IO_GUARD_MUTE_SINGLE 1 Muting active for one opening IO_GUARD_MUTE_CONTINUOUS 2 Muting active continuously	
.NET <Edc object>.IoSignal.GuardSetMuteRangeLimit (int32 MuteRangeLimit)		

10.3.18 DoPEIOGuardGetMuteRangeLimit

Get the state of the range limit monitoring muting.		
Minimum requirements: EDCi with EDCiApp 9149.019, DoPE 10.18		
Function declaration	Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPEIOGuardGetMuteRangeLimit (DoPEHdl *MuteRangeLimit)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer for the state of the range limit muting	
.NET <Edc object>.IoSignal.GuardGetMuteRangeLimit (ref int32 MuteRangeLimit)		

10.4 Display Commands

DoPE offers functions to write data to the EDC display. The display is subdivided into:

1. Headline section
2. Function key section
3. Value section for two values

Following functions can be used to write data to the display.

Note: The display should not be updated faster than 3 times per second!

10.4.1 DoPEDspClear

Clear LCD-display at EDC front panel.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE	DoPEDspClear (DoPEHdl)	Function returns Error constant (DoPERR_XXXX) DoPE link handle	
.NET <Edc object>.Display.Clear()			

10.4.2 DoPEwDspHeadLine

Display HeadLine on LCD-display at EDC front panel.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE wchar_t	DoPEwDspHeadLine (DoPEHdl * HeadLine)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Wide character string	
DSP_FKEYSLINE_LEN (22) Number of characters in Value field (including terminating zero '\0')			
.NET <Edc object>.Display.HeadLine(String HeadLine)			

10.4.3 DoPEwDspFKeys

Display Function-Keys on LCD-display at EDC front panel.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE wchar_t	DoPEwDspFKeys(DoPEHdl * FKeys)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Wide Character string	
DSP_FKEYSLINE_LEN (22) Number of characters in Value field (including terminating zero '\0')			
.NET <Edc object>.Display.FKeys(String FKeys)			

10.4.4 DoPEwDspMValue

Display values with dimension on LCD-display at EDC front panel.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE wchar_t wchar_t wchar_t wchar_t WORD	DoPEwDspMValue (DoPEHdl * Value1 * Value2 * Dim1 * Dim2 * lpustAN)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle Wide character string for first value Wide character string for second value Wide character string for first dimension Wide character string for second dimension Pointer to transaction number (not for Sync. version).	
DSP_VALUE_LEN (10)	Number of characters in Value field (including terminating zero '\0')		
DSP_DIM_LEN (7)	Number of characters in Dim field (including terminating zero '\0')		
On the RMCi1/8 values can be displayed as “2 characters sensor name + 7 digits” or “8 digits” without sensor name.			
.NET	<Edc object>.Display.MValue(String Value1, String Value2, String Dim1, String Dim2)		

11 EDC keyboard interface

DoPE offers some function to handle keys and LEDs of EDC keyboards (Front-Panel and RMC).

11.1 DoPERdNumberOfKeyboards

Read the number of connected keyboards.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPERdNumberOfKeyboards (DoPEHdl *pNumber	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to storage for the number of keyboards	
.NET <Edc object>.Key.RdNumberOfKeyboards(ref Int32 Number)			

11.2 DoPESetKeyShiftState

Set the keyboard shift state.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPESetKeyShiftState (DoPEHdl Enable)	Function returns Error constant (DoPERR_XXXX) DoPE link handle 0=disable shift state, else enable shift state	
.NET <Edc object>.Key.SetKeyShiftState(bool Enable)			

11.3 DoPERdKeyShiftState

Read the keyboard shift state.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPERdKeyShiftState (DoPEHdl *pEnabled	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to storage for keyboard shift state	
.NET <Edc object>.Key.RdKeyShiftState(ref bool Enabled)			

11.4 DoPESetLed

Switch On/Off LEDs at the EDC keyboard.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned __int64 unsigned __int64 unsigned __int64 WORD WORD WORD	DoPESetLed (DoPEHdl LedOn LedOff LedFlash OemLedOn OemLedOff OemLedFlash)	Function returns Error constant (DoPERR_XXXX) DoPE link handle These LEDs will be set These LEDs will be reset These LEDs 'flash' These LEDs will be set These LEDs will be reset These LEDs 'flash' Pointer to transaction number (not for Sync. version).	
The parameters will be processed in the following sequence (important with conflicting data): 1. Flashing LEDs. (lowest) 2. Resetting of the LEDs. 3. Setting of the LEDs. (highest)			
.NET <Edc object>.Key.SetLed(Int64 LedOn, Int64 LedOff, Int64 LedFlash, Int16 OemLedOn, Int16 OemLedOff, Int16 OemLedFlash)			

11.5 DoPELedMask

Set the LED matrix bit for a given LED code. Each LED is represented by a bit in the LED matrix. This function converts a LED code to the matrix bit and sets the bit in the LED or OEM LED matrix.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE __int32 unsigned unsigned __int64 WORD DoPELedMask (DoPEHdl LedCode State *pLedMask *pOemLedMask	Function returns Error constant (DoPERR_XXXX) DoPE link handle LED code to convert 0=clear bit, else set bit Pointer to the LED matrix Pointer to the OEM-LED matrix (NULL pointers are accepted)
.NET <Edc object>.Key.LedMask(DoPE.LED LedCode, Int32 State, ref Int64 LedMask, ref Int16 OemLedMask)	

11.6 DoPECurrentKeys

Read the current keys.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned __int64 WORD DoPECurrentKeys(DoPEHdl *pKeys *pOemKeys	Function returns Error constant (DoPERR_XXXX) DoPE link handle current key matrix current OEM-key matrix
.NET <Edc object>.Key.CurrentKeys(ref Int64 Keys, ref Int16 OemKeys)	

11.7 DoPEKeyPressed

Check if the key with the given key code is pressed.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern intDLLAPI DoPE_HANDLE __int32 unsigned __int64 unsigned short DoPEKeyPressed (DoPEHdl KeyCode Keys OemKeys	Function returns 0 if key is not pressed, else key is pressed DoPE link handle Key code to check Key matrix OEM-Key matrix
.NET <Edc object>.Key.KeyPressed(DoPE.KEY KeyCode, Int64 Keys, Int16 OemKeys)	

11.8 DoPEKeyGet

Get the key code of a pressed key. Each pressed key is represented by a set bit in the key matrix. This function converts a bit from the key matrix to its key code and resets the bit in the key matrix.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern intDLLAPI DoPE_HANDLE unsigned __int64 unsigned short DoPEKeyGet (DoPEHdl *pKeys *OemKeys	The key code of a pressed key or DoPE_KEY_NONE DoPE link handle Pointer to the key matrix Pointer to the OEM-Key matrix
.NET <Edc object>.Key.KeyGet(ref Int64 Keys, ref Int16 OemKeys)	

11.9 Definition of keys

Key constant	Key code	Description
DOLI keys		
DoPE_KEY_NONE	-1	invalid (no key pressed)
DoPE_KEY_HALT	0x00	'HALT'
DoPE_KEY_UP	0x01	'UP'
DoPE_KEY_DOWN	0x02	'DOWN'
DoPE_KEY_DPOTI	0x03	'DigiPoti pressed'
DoPE_KEY_F1	0x04	'F1' function key
DoPE_KEY_F2	0x05	'F2' function key
DoPE_KEY_F3	0x06	'F3' function key
DoPE_KEY_ON	0x07	'ON'
DoPE_KEY_TEST	0x08	'TEST'
DoPE_KEY_UPR_GRIP_OPEN	0x09	'Upper Grip Open'
DoPE_KEY_UPR_GRIP_CLOSE	0x0A	'Upper Grip Close'
DoPE_KEY_LWR_GRIP_OPEN	0x0B	'Lower Grip Open'
DoPE_KEY_LWR_GRIP_CLOSE	0x0C	'Lower Grip Close'
DoPE_KEY_EXTMETER_OPEN	0x0D	'Extensometer Open'
DoPE_KEY_EXTMETER_CLOSE	0x0E	'Extensometer Close'
DoPE_KEY_FIXED_XHEAD_UP	0x0F	'fixed X-Head Up'
DoPE_KEY_FIXED_XHEAD_DOWN	0x10	'fixed X-Head Down'
DoPE_KEY_DPOTI_CONTROL	0x11	'DigiPoti Speed-Position Mode'
DoPE_KEY_BF1	0x12	'blind F1' function key (no LCD)
DoPE_KEY_BF2	0x13	'blind F2' function key (no LCD)
DoPE_KEY_BF3	0x14	'blind F3' function key (no LCD)
DoPE_KEY_HIGH_PRESSURE	0x15	'Activate High Pressure'
DoPE_KEY_LOW_PRESSURE	0x16	'Activate Low Pressure'
DoPE_KEY_OFF	0x17	'OFF'
DoPE_KEY_UP_FAST	0x18	'Move UP fast'
DoPE_KEY_DOWN_FAST	0x19	'Move DOWN fast'
DoPE_KEY_TEST_STOP	0x1A	'Test stop'
DoPE_KEY_TEST_RETURN	0x1B	'Test return'
DoPE_KEY_TEST_HALT_CONTINUE	0x1C	'Test halt / continue'
DoPE_KEY_reserved1D	0x1D	
DoPE_KEY_reserved1E	0x1E	
DoPE_KEY_reserved1F	0x1F	
DoPE_KEY_reserved20	0x20	
DoPE_KEY_CTRL_SENSOR3	0x21	
DoPE_KEY_CTRL_SENSOR4	0x22	
DoPE_KEY_CTRL_SENSOR5	0x23	
DoPE_KEY_CTRL_SENSOR6	0x24	
DoPE_KEY_CTRL_SENSOR7	0x25	
DoPE_KEY_CTRL_SENSOR8	0x26	
DoPE_KEY_CTRL_SENSOR9	0x27	
DoPE_KEY_CTRL_SENSOR10	0x28	
DoPE_KEY_CTRL_SENSOR11	0x29	
DoPE_KEY_CTRL_SENSOR12	0x2A	
DoPE_KEY_CTRL_SENSOR13	0x2B	
DoPE_KEY_CTRL_SENSOR14	0x2C	
DoPE_KEY_CTRL_SENSOR15	0x2D	
DoPE_KEY_DP	0x2E	'.' period (decimal point)
DoPE_KEY_SIGN	0x2F	'±' sign
DoPE_KEY_0	0x30	'0'
DoPE_KEY_1	0x31	'1'
DoPE_KEY_2	0x32	'2'
DoPE_KEY_3	0x33	'3'

DoPE_KEY_4	0x34	'4'
DoPE_KEY_5	0x35	'5'
DoPE_KEY_6	0x36	'6'
DoPE_KEY_7	0x37	'7'
DoPE_KEY_8	0x38	'8'
DoPE_KEY_9	0x39	'9'
DoPE_KEY_CTRL_POS	0x3A	Position Control
DoPE_KEY_CTRL_LOAD	0x3B	Load Control
DoPE_KEY_CTRL_EXTENSION	0x3C	Extension Control
DoPE_KEY_reserved3D	0x3D	
DoPE_KEY_LINK	0x3E	Link
DoPE_KEY_CONNECT	0x3F	
OEM keys		
DoPE_KEY_OEM0	0x40	
DoPE_KEY_OEM1	0x41	
DoPE_KEY_OEM2	0x42	
DoPE_KEY_OEM3	0x43	
DoPE_KEY_OEM4	0x44	
DoPE_KEY_OEM5	0x45	
DoPE_KEY_OEM6	0x46	
DoPE_KEY_OEM7	0x47	
DoPE_KEY_OEM8	0x48	
DoPE_KEY_OEM9	0x49	
DoPE_KEY_OEMA	0x4A	
DoPE_KEY_OEMB	0x4B	
DoPE_KEY_OEMC	0x4C	
DoPE_KEY_OEMD	0x4D	
DoPE_KEY_OEME	0x4E	
DoPE_KEY_OEMF	0x4F	

11.10 Keyboard Lock Commands

The RMCi features a Link Key to allow an exclusive access if more than one RMCi is connected to the EDCi. DoPE offers a function to disable RMCi keyboard input and let the PC gain exclusive access.

11.10.1 DoPESetKeyboardLockState

Set the lock state of all keyboards.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.01		
Function declaration	Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned DoPESetKeyboardLockState (DoPEHdl, Enable)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle true: set all keyboards to the Locked state - all keys are disabled - the Link LED is off false: In DoPEGeneralData RmcActiveMode Single Mode - set all keyboards to the Idle state - only the Link key is enabled - the Link LED is blinking In DoPEGeneralData RmcActiveMode Multi Mode - set all keyboards to the Linked state - all keys are enabled - the Link LED is on	
.NET <Edc object>.Key.SetKeyboardLockState(bool Enable)		

11.10.2 DoPEGetKeyboardLockState

Set the lock state of all keyboards.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.01			
Function declaration		Description	
extern unsigned DLLAPI DoPE_HANDLE unsigned	DoPESetKeyboardLockState (DoPEHdl * pState)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to storage for keyboards state	
DoPE_KEYBOARD_IDLE all keyboard are in the Idle state - only the Link key is enabled - the Link LED is blinking DoPE_KEYBOARD_LINKED at least one keyboard is in the Linked state - all keys are enabled - the Link LED is on DoPE_KEYBOARD_LOCKED all keyboards were set to the Locked state by DoPESetKeyboardLockState - all keys are locked - the Link LED is off			
.NET <Edc object>.Key.GetKeyboardLockState(ref DoPE.KEYBOARD_STATE State)			

11.11 Definition of LEDs

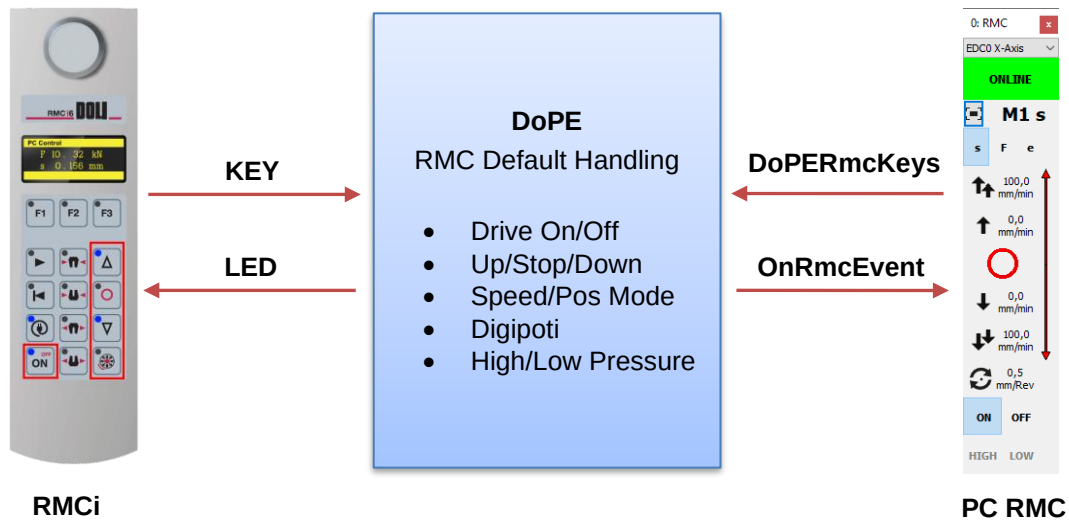
LED constant	LED refers to key
DOLI LEDs	
DoPE_LED_HALT	DoPE_KEY_HALT
DoPE_LED_UP	DoPE_KEY_UP
DoPE_LED_DOWN	DoPE_KEY_DOWN
DoPE_LED_DPOTI	DoPE_KEY_DPOTI
DoPE_LED_F1	DoPE_KEY_F1
DoPE_LED_F2	DoPE_KEY_F2
DoPE_LED_F3	DoPE_KEY_F3
DoPE_LED_ON	DoPE_KEY_ON
DoPE_LED_TEST	DoPE_KEY_TEST
DoPE_LED_UPR_GRIP_OPEN	DoPE_KEY_UPR_GRIP_OPEN
DoPE_LED_UPR_GRIP_CLOSE	DoPE_KEY_UPR_GRIP_CLOSE
DoPE_LED_LWR_GRIP_OPEN	DoPE_KEY_LWR_GRIP_OPEN
DoPE_LED_LWR_GRIP_CLOSE	DoPE_KEY_LWR_GRIP_CLOSE
DoPE_LED_EXTMETER_OPEN	DoPE_KEY_EXTMETER_OPEN
DoPE_LED_EXTMETER_CLOSE	DoPE_KEY_EXTMETER_CLOSE
DoPE_LED_FIXED_XHEAD_UP	DoPE_KEY_FIXED_XHEAD_UP
DoPE_LED_FIXED_XHEAD_DOWN	DoPE_KEY_FIXED_XHEAD_DOWN
DoPE_LED_DPOTI_CONTROL	DoPE_KEY_DPOTI_CONTROL
DoPE_LED_BF1	DoPE_KEY_BF1
DoPE_LED_BF2	DoPE_KEY_BF2
DoPE_LED_BF3	DoPE_KEY_BF3
DoPE_LED_HIGH_PRESSURE	DoPE_KEY_HIGH_PRESSURE
DoPE_LED_LOW_PRESSURE	DoPE_KEY_LOW_PRESSURE
DoPE_LED_OFF	DoPE_KEY_OFF
DoPE_LED_UP_FAST	DoPE_KEY_UP_FAST
DoPE_LED_DOWN_FAST	DoPE_KEY_DOWN_FAST
DoPE_LED_TEST_STOP	DoPE_KEY_TEST_STOP
DoPE_LED_TEST_RETURN	DoPE_KEY_TEST_RETURN
DoPE_LED_TEST_HALT_CONTINUE	DoPE_KEY_TEST_HALT_CONTINUE
DoPE_LED_reserved1D	DoPE_KEY_reserved1D
DoPE_LED_reserved1E	DoPE_KEY_reserved1E
DoPE_LED_reserved1F	DoPE_KEY_reserved1F
DoPE_LED_reserved20	DoPE_KEY_reserved20
DoPE_LED_CTRL_SENSOR3	DoPE_KEY_reserved21
DoPE_LED_CTRL_SENSOR4	DoPE_KEY_reserved22
DoPE_LED_CTRL_SENSOR5	DoPE_KEY_reserved23
DoPE_LED_CTRL_SENSOR6	DoPE_KEY_reserved24
DoPE_LED_CTRL_SENSOR7	DoPE_KEY_reserved25
DoPE_LED_CTRL_SENSOR8	DoPE_KEY_reserved26
DoPE_LED_CTRL_SENSOR9	DoPE_KEY_reserved27
DoPE_LED_CTRL_SENSOR10	DoPE_KEY_reserved28
DoPE_LED_CTRL_SENSOR11	DoPE_KEY_reserved29
DoPE_LED_CTRL_SENSOR12	DoPE_KEY_reserved2A
DoPE_LED_CTRL_SENSOR13	DoPE_KEY_reserved2B
DoPE_LED_CTRL_SENSOR14	DoPE_KEY_reserved2C
DoPE_LED_CTRL_SENSOR15	DoPE_KEY_reserved2D
DoPE_LED_DP	DoPE_KEY_DP
DoPE_LED_SIGN	DoPE_KEY_SIGN
DoPE_LED_0	DoPE_KEY_0
DoPE_LED_1	DoPE_KEY_1
DoPE_LED_2	DoPE_KEY_2
DoPE_LED_3	DoPE_KEY_3
DoPE_LED_4	DoPE_KEY_4

DoPE_LED_5	DoPE_KEY_5
DoPE_LED_6	DoPE_KEY_6
DoPE_LED_7	DoPE_KEY_7
DoPE_LED_8	DoPE_KEY_8
DoPE_LED_9	DoPE_KEY_9
DoPE_LED_CTRL_POS	DoPE_KEY_reserved3A
DoPE_LED_CTRL_LOAD	DoPE_KEY_reserved3B
DoPE_LED_CTRL_EXTENSION	DoPE_KEY_reserved3C
DoPE_LED_reserved3D	DoPE_KEY_reserved3D
DoPE_LED_LINK	DoPE_KEY_LINK
DoPE_LED_CONNECT	DoPE_KEY_CONNECT
OEM LEDs	
DoPE_LED_OEM0	DoPE_KEY_OEM0
DoPE_LED_OEM1	DoPE_KEY_OEM1
DoPE_LED_OEM2	DoPE_KEY_OEM2
DoPE_LED_OEM3	DoPE_KEY_OEM3
DoPE_LED_OEM4	DoPE_KEY_OEM4
DoPE_LED_OEM5	DoPE_KEY_OEM5
DoPE_LED_OEM6	DoPE_KEY_OEM6
DoPE_LED_OEM7	DoPE_KEY_OEM7
DoPE_LED_OEM8	DoPE_KEY_OEM8
DoPE_LED_OEM9	DoPE_KEY_OEM9
DoPE_LED_OEMA	DoPE_KEY_OEMA
DoPE_LED_OEMB	DoPE_KEY_OEMB
DoPE_LED_OEMC	DoPE_KEY_OEMC
DoPE_LED_OEMD	DoPE_KEY_OEMD
DoPE_LED_OEME	DoPE_KEY_OEME
DoPE_LED_OEMF	DoPE_KEY_OEMF

11.12 RMC Default Handling

DoPE provides a standardized handling of the basic key functions and LEDs on all DOLI RMCi remote controls and IO keys.

The next picture shows an overview of the DoPE RMC default handling:



General DoPE RMC default handling:

- DoPE handles the red marked RMCi keys. The other keys are handled by the EDC, if these functions are enabled in the EDC IO Signals set-up.
- The Up and Down LEDs show the crosshead movement, even if it was caused by DoPE users directly via a positioning command. The Up and Down LEDs light up constantly if the frequency of a cycle command exceeds 2 Hz.
- The DoPE user can also emulate the keys and LEDs for a software PC RMC via a simple software interface.

Software interface for RMCi and PC RMC emulation:

- Activate the DoPE RMC default handling after initialization of the EDC with the command `DoPERmcEnable(DoPEHdl, -1, -1)`.
- The pressed keys of the software PC RMC are emulated with the command `DoPERmcKeys`.
- You can show the LED states of the RMCi (e.g. On/Off, Up/Down) by using the `OnRmcEvent` handler.
- Alternatively, you can poll the LED states of the RMCi with the command `DoPERmcState`.

The following table shows the used commands for the specific keys:

Key	DoPE Command
On	DoPEOn
Off (Shift + On)	Closed loop: DoPEOff Open loop: DoPESetCtrl false and DoPESetCtrl true
Stop	Closed loop: DoPESHalt Open loop: DoPESetCtrl false and DoPESetCtrl true
Up/Down	Digipoti speed mode: DoPEFDPoti with mode EXT_SPEED_UP or EXT_SPEED_DOWN. Up or Down LED is on.
Digipoti Mode	Digipoti position mode: DoPEFDPoti with mode EXT_POS_UP_DOWN. Up and Down LED are on. Press Stop to leave position mode.
Shift + Digipoti Mode	High/Low Pressure toggle: On: DoPEIOHighPressureSet true (LED on) Off: DoPEIOHighPressureSet false (LED off)

11.12.1 DoPERmcEnable

Enable / disable the RMC default handling for keys and LEDs.
(Sets the PushMode and RmcDPoti settings to the default values defined in the setup RMC definition)

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE unsigned __int64 unsigned __int64 DoPERmcEnable(DoPEHdl, EnableKeys, EnableLeds)	Function returns Error constant (DoPERR_ xxxx) DoPE link handle mask of enabled RMC emulation keys mask of enabled RMC emulation LEDs
.NET <Edc object>.DoPERmc.Enable(int64 EnableKeys, int64 EnableLeds)	
Supported keys: DoPE_KEY_CTRL_POS DoPE_KEY_CTRL_LOAD DoPE_KEY_CTRL_EXTENSION DoPE_KEY_CTRL_SENSOR3 DoPE_KEY_CTRL_SENSOR4 DoPE_KEY_CTRL_SENSOR5 DoPE_KEY_CTRL_SENSOR6 DoPE_KEY_CTRL_SENSOR7 DoPE_KEY_CTRL_SENSOR8 DoPE_KEY_CTRL_SENSOR9 DoPE_KEY_CTRL_SENSOR10 DoPE_KEY_CTRL_SENSOR11 DoPE_KEY_CTRL_SENSOR12 DoPE_KEY_CTRL_SENSOR13 DoPE_KEY_CTRL_SENSOR14 DoPE_KEY_CTRL_SENSOR15 DoPE_KEY_CTRL_HALT DoPE_KEY_UP DoPE_KEY_DOWN DoPE_KEY_UP_FAST DoPE_KEY_DOWN_FAST DoPE_KEY_ON DoPE_KEY_OFF DoPE_KEY_DPOTI_CONTROL DoPE_KEY_HIGH_PRESSURE DoPE_KEY_LOW_PRESSURE	Activate Position control (or Open Loop mode) Activate Load control Activate Extension control Activate Sensor 3 control Activate Sensor 4 control Activate Sensor 5 control Activate Sensor 6 control Activate Sensor 7 control Activate Sensor 8 control Activate Sensor 9 control Activate Sensor 10 control Activate Sensor 11 control Activate Sensor 12 control Activate Sensor 13 control Activate Sensor 14 control Activate Sensor 15 control Apply SHalt (or Control Off in Open Loop mode) Move crosshead UP with slow speed Move crosshead DOWN with slow speed Move crosshead UP with fast speed Move crosshead DOWN with slow speed Turn on drive Turn off drive Move crosshead UP/DOWN with DigiPoti in position mode Enable High Pressure mode Enable Low Pressure mode

Supported LED's	
DoPE_LED_CTRL_POS	Position Control (or Open Loop mode) active
DoPE_LED_CTRL_LOAD	Load Control active
DoPE_LED_CTRL_EXTENSION	Extension Control active
DoPE_LED_HALT	Crosshead is halted
DoPE_LED_UP	Crosshead is moving UP with slow or fast speed
DoPE_LED_DOWN	Crosshead is moving DOWN with slow or fast speed
DoPE_LED_UP+ DoPE_LED_DOWN	DigiPoti position mode active (or fast cycles)
DoPE_LED_UP_FAST	Crosshead is moving UP with fast speed
DoPE_LED_DOWN_FAST	Crosshead is moving DOWN with fast speed
DoPE_LED_OFF	Drive is turned off
DoPE_LED_ON	Drive is turned on
DoPE_LED_DPOTI_CONTROL	DigiPoti position mode active
DoPE_LED_HIGH_PRESSURE	High Pressure mode active
DoPE_LED_LOW_PRESSURE	Low Pressure mode active

11.12.2 DoPERmcKeys

Set / reset / toggle keys for the RMC default handling.

The three key patterns will be processed in the following sequence (important with conflicting data):

- 1.) Flashing bits. (lowest priority)
- 2.) Resetting of the bits.
- 3.) Setting of the bits. (highest priority)

The RMC default handler is a DoPE realtime handler. Therefor the OnCommandError handler must be used.

Function declaration	Description
extern unsigned DoPERmcKeys (DoPE_HANDLE DoPEHdl[] unsigned __int64 SetKeys unsigned __int64 ResKeys unsigned __int64 ToggleKeys	Function returns Error constant (DoPERR_ xxxx) DoPE link handle array (terminated by NULL). RMC emulation keys to set RMC emulation keys to clear RMC emulation keys to toggle
.NET <Edc object>.DoPERmc.Keys(int64 SetKeys, int64 ResKeys, int64 ToggleKeys)	

11.12.3 DoPERmcState

Get keys and LED's state of the RMC default handling.

This command supplies any change of RMC default handler key input and LED output since the last call.

Function declaration	Description
extern unsigned DoPERmcState (DoPE_HANDLE DoPEHdl[] DoPEOnRmcEvent *pRmcEvent	Function returns Error constant (DoPERR_ xxxx) DoPE link handle array (terminated by NULL). Pointer to RMC event structure (refere to OnRmcEvent handler)
.NET <Edc object>.DoPERmc.State(ref DoPE.OnRmcEvent RmcEvent)	

11.12.4 DoPESetOnRmcEventHdlr

Please see chapter 8.19 DoPESetOnRmcEventHdlr.

12 Setup Commands

You can read or write machine set-up data.

Set-up data from machine number one onwards are stored inside the EDC flash disk.

Machine number zero is the working set-up. All data written to machine zero are not stored but can be used to initialize EDC.

12.1 GeneralData

12.1.1 DoPEGeneralData Structure

```

DoPEGeneralData:
typedef struct                                /* General data */
{                                              /* ----- */
    __uint16 MachineNo;                      /* Number of active machines [No] */
    __uint16 MachineNoIo;                   /* Machine select IO's config. [No] */
                                              /* (see MACHINE_NO_IO_... definitions) */

    __uint16 SuperPassword;                 /* Supervisor Password(0=inactive) [No] */
    __uint16 UserPassword;                 /* User Password(0=inactive) [No] */
    __uint16 Logo;                         /* DOLI Logo (0 = inactive) [No] */
    __uint32 Language;                     /* Language [No] */
    __uint32 FunctionID;                   /* User defined function ID [No] */

    __uint32 nRmc;                         /* Required number of RMC's [No] */
    __uint16 RmcPushMode;                 /* RMC push mode for Up/Down keys[0/1] */
    __uint16 RmcActiveMode;               /* RMC active mode RMC_ACTIVE_MODE_xxx */

    __uint16 EDMEEnable;                   /* External device monitoring [1/0] */
    __uint16 SupEstopEnable;               /* Estop supply [1/0] */
    __uint16 UpStopEnable;                 /* Upper stop switch STOP_xxx [No] */
    __uint16 DownStopEnable;              /* Lower stop switch STOP_xxx [No] */

    __uint16 SyncOption;                   /* Sync option (SYNC_OPTION_xxx) [No] */
    __uint16 SyncStopEnable;               /* Sync enable Stop STOP_xxx [No] */
    __uint16 SyncDrvOnOffEnable;           /* Sync enable Drive On/Off [1/0] */

    __uint16 Day;                         /* Date of last change [No] */
    __uint16 Month;                       /* Date of last change [No] */
    __uint16 Year;                       /* Date of last change [No] */

    wchar_t Info[INFO_TEXT_LEN];           /* EDC info text */
} DoPEGeneralData;

```

12.1.2 DoPERdGeneralData

Read general data of the machine.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPERdGeneralData (DoPE_HANDLE DoPEHdl DoPEGeneralData * GeneralData)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer for DoPEGeneralData structure
.NET <Edc object>.Setup.RdGeneralData(ref DoPE.GeneralData GeneralData)	

12.1.3 DoPEWrGeneralData

Write general data of the machine.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration		Description	
extern unsigned DLLAPI	DoPEWrGeneralData (	Function returns Error constant (DoPERR_XXXX)	
DoPE_HANDLE	DoPEHdl	DoPE link handle	
DoPEGeneralData	* GeneralData	Pointer to DoPEGeneralData structure	
WORD	* lpusTAN)	Pointer to transaction number (not for Sync. version).	
.NET <Edc object>.Setup.WrGeneralData(ref DoPE.GeneralData GeneralData)			

12.2 Machine Data

12.2.1 DoPEMachine Structure

```
typedef struct                                /* EDC Machine Data */
{                                              /* ----- */
    DoPEMachineDef  MDef;                    /* Machine definition data */
    DoPESenDef      SDef[MAX_MC];            /* Sensor definition data */
    DoPECtrlSenDef  CSDef[MAX_CTRL];         /* Control-Sensor definition */
    DoPECtrlSenDef  CSDefHigh[MAX_CTRL];    /* Control-Sensor def. (high pressure) */
    DoPEOutChaDef   ODef[MAX_OC];           /* Analogue output definition data */
    DoPEBitDef      BitDef[MAX_BIT];        /* Digital bit input/output definition */
    DoPEIOSignals   IOSignals;              /* IO signal definition */
    DoPEMainMenu     MainMenu[MAX_MAIN_MENU]; /* EDC main menu definition */
} DoPEMachine;
```

12.2.1.1 DoPEMachineDef

```
/* Constants for 'CtrlStructure' */
/* ----- */
#define CTRL_STRUCTURE_SPINDLE 0 /* Digital position controller */
/* for speed controlled drives */
#define CTRL_STRUCTURE_HYDRAULIC 1 /* Hydraulic */
#define CTRL_STRUCTURE_OPENLOOP 3 /* No control (only for CTRL_POS) */
#define CTRL_STRUCTURE_SPINDLE_SP 6 /* Digital position and speed controller*/

#define XHEAD_INITIAL_MODE_AUTOMATIC 0 /* Set XHead to last known position */
#define XHEAD_INITIAL_MODE_MANUAL 1 /* Set XHead to manual position */

#define X4PIN14_MODE_BYPASS 0 /* X4 Pin14 mode bypass */
#define X4PIN14_MODE_EDC_READY 1 /* X4 Pin14 mode EDC ready */

typedef struct                                /* Definition of machine data */
{                                              /* ----- */
    double      SystemTime;                  /* EDC system time [s] */
    __uint16_t  CtrlStructure;               /* Closed loop control structure [No] */
    double      DataTransmissionRate;        /* Data transmission rate [s] */
    __uint16_t  Mode;                        /* Data acquisition or control [1/0] */
    __uint16_t  XheadDir;                    /* Machine moves up/down [1/0] */
    /* with positive output signal */
    double      EncXheadRatio;               /* Ratio encoder-Xhead [Rev/Unit] */
    __uint16_t  XheadInitialMode;            /* XHEAD_INITIAL_MODE_xxx [No] */
    double      XheadInitialValue;           /* Initial XHead value (manual) [Unit] */
    double      BrakeOpen;                  /* Delay time to open brake after [s] */
    /* closed loop control is active */
    double      BrakeClose;                 /* Delay time to close brake before [s] */
    /* closed loop control is deactivated */
    double      PistonArea;                  /* Area of piston for hydraulic [m²] */
    double      LoadMax;                    /* Max. load capacity of machine [N] */
    double      Load100;                    /* Nominal load of machine [N] */
    double      Stiffness;                   /* Over all stiffness of machine [N/m] */
    __uint16_t  CtrlOnMode;                  /* CTRL ON mode: 0=NoCtrl, 1=Ctrl */
    double      FixValue;                    /* Fixing output value [%] */
    double      InitValue;                   /* Initial output value [%] */
    double      ReturnValue;                 /* Return output value [%] */
    wchar_t     Info[INFO_TEXT_LEN];         /* Machine info text */
} DoPEMachineDef;
```

12.2.1.2 DoPESenDef

```

/* Constants for 'LimitCtrl' */
/* ----- */
#define REACT_STATE 0 /* only status bits are set */
#define REACT_DRIVE_OFF 1 /* turn off drive */
#define REACT_SHALT 2 /* halt crosshead in position control */

typedef struct /* Sensor definition data */
{
/* ----- */
__uint16 Connector; /* Connector number of sensor [No]*/
__uint16 Sign; /* Invert sign of sensor [1/0]*/
__uint16 CtrlChannel; /* Activate control channel [1/0]*/
__uint16 LimitCtrl; /* If limit exceeded REACT_xxx [No]*/
__uint16 ConnectedCtrl; /* Stop if disconnected [1/0]*/
__uint16 UseEeprom; /* Use sensor EEPROM data [1/0]*/
double FilterTime; /* Filter time [s]*/
__uint32 SensorID; /* Sensor ID for free use [No]*/
/* Only for sensors without EEPROM: */
__uint16 Init; /* Sensor init [No]*/
double NominalValue; /* Nominal value of sensor [Unit]*/
__uint16 Unit; /* Unit of sensor UNIT_xxx [No]*/
double Offset; /* Offset of sensor [Unit]*/
double UpperLimit; /* Upper range limit of sensor [%]*/
double LowerLimit; /* Lower range limit of sensor [%]*/
/* Only for incremental sensors: */
double Scale; /* Scale of sensor [inc/revolution]*/
/* or [Unit/inc]*/
double Correction; /* Correction value of sensor [No]*/
} DoPESenDef;

```

12.2.1.3 DoPECtrlSenDef

```

typedef struct                                     /* Control Sensor definition data */
{
    double    Acceleration;                       /* ----- */
    double    Speed;                              /* Nominal acceleration [Unit/s²] */
    double    Deviation;                          /* Nominal speed [Unit/s] */
    double    DevReaction;                        /* Max. deviation of controller [Unit] */
    __uint16  PosP;                              /* Reaction if deviation exceeded [No] */
    double    PosI;                              /* Pos. contr. P: gain [No] */
    double    PosD;                              /* Pos. contr. I: time constant [s] */
    double    SpeedP;                             /* Pos. contr. D: time constant [s] */
    double    SpeedI;                             /* Speed contr. P: gain [No] */
    double    SpeedD;                             /* Speed contr. I: time constant [s] */
    double    SpeedFFP;                           /* Speed contr. D: time constant [s] */
    double    PosDelay;                           /* Speed feed forward [%] */
    double    FilterTime;                         /* Delay for command [s] */
    double    FilterTime;                         /* Filter time [s] */

    /* Deadband definitions */
    __uint16  DeadbandEnable;                     /* Deadband values enabled [1/0] */
    double    Deadband;                           /* Deadband around setpoint [Unit] */
    __uint16  PercentP;                           /* Smallest P inside deadband [% of PosP] */

    /* Dither definitions */
    __uint16  DitherEnable;                       /* Dither values enabled [1/0] */
    double    DitherFrequency;                     /* Dither frequency [Hz] */
    double    DitherAmplitude;                     /* Dither amplitude [%] */

    /* RMC definitions */
    double    RmcSpeedSlow;                       /* RMC slow speed [Unit/s] */
    double    RmcSpeedFast;                       /* RMC fast speed [Unit/s] */
    double    DPotiSpeedSens;                      /* Sensitivity Speed/Openloop [Rev/Nom] */
    double    DPotiPosSens;                        /* Sensitivity Pos [Unit/Rev] */
} DoPECtrlSenDef;

```

12.2.1.4 DoPEOutChaDef

```

typedef struct                                /* Definition of output channel */
{
    /* ----- */
    __uint16 Connector;                       /* Connector number of channel [No] */
    __uint16 Sign;                           /* Invert sign of channel [1/0] */
    double   MaxValue;                       /* Maximum output value [%] */
    double   MinValue;                      /* Minimum output value [%] */
    double   InitValue;                     /* Initial output value [%] */
    double   Offset;                        /* Offset of channel [%] */
    DoPEMc2OutputDef Mc2Output;              /* Measured channel to analogue output */

    /* from here only for COMMAND_OUT */
    __uint16 Signal;                         /* Digital command output signal [No] */
    double   SignalFrequency;                /* Digital command output signal [Hz] */

    /* Only DOLI drives */
    double   PaVoltage;                     /* Max. voltage of power amplifier [V] */
    double   PaCurrent;                     /* Max. current of power amplifier [A] */
    double   MaxCurrTime;                   /* Max. time for max. current (I²t) [s] */
    __uint32 CurrentP;                       /* current.contr. P: gain [No] */
    __uint16 CurrentI;                       /* current.contr. P: time constant [No] */
    __uint16 CurrentD;                       /* current.contr. P: time constant [No] */
} DoPEOutChaDef;

/* Constants for Signal */
/* ----- */
#define OUTSIGNAL_A_B 0 /* A/B */
#define OUTSIGNAL_PULSE_SIGN 1 /* pulse/sign */
#define OUTSIGNAL_UP_DOWN 2 /* up/down */
#define OUTSIGNAL_ANALOGUE 3 /* analogue (+/- 10V) */
#define OUTSIGNAL_DC_MOTOR 4 /* PWM with direction/amplitude signals */
#define OUTSIGNAL_DC_LINEAR_MOTOR 5 /* PWM with single, variable duty-cycle signal */
#define OUTSIGNAL_SIGN_MAGNITUDE 4 /* legacy: use OUTSIGNAL_DC_MOTOR */
#define OUTSIGNAL_LOCKEDANTIPHASE 5 /* legacy: use OUTSIGNAL_DC_LINEAR_MOTOR */
#define OUTSIGNAL_MAX 6 /* number of signals */

typedef struct                                /* Measured channel to analogue output */
{
    /* ----- */
    __uint16 Mode;                           /* MC2OUT_MODE... (see DoPEoutput.h) */
    __uint16 SensorNo;                       /* Sensor number [0..15] */
    double   SensorPoint[MC2OUT_MAX];        /* Sensor Point Values [Unit] */
    double   OutputPoint[MC2OUT_MAX];        /* Output Point Values [%] */
} DoPEMc2OutputDef;

```

12.2.1.5 DoPEBitDef

```

typedef struct                                /* Definition of bit input/output */
{
    /* ----- */
    __uint16 Connector;                       /* Connector number of device [No] */
    __uint16 OutInitValue;                   /* OUT initial value of device [No] */
    __uint16 OutWriteProtect;                /* OUT write protect mask [No] */
} DoPEBitDef;

```

12.2.1.6 DoPEIOSignals

```
typedef struct                                /* IO signal definition */
{
    DoPEIOGrip      Grip;                    /* Grip */
    DoPEIOExt       Ext;                    /* Extensometer */
    DoPEIOFixedXHead FixedXHead;            /* FixedXHead */
    DoPEIOHighPressure HighPressure;        /* High pressure */
    DoPEIOMisc      Misc;                  /* Miscellaneous */
    DoPEIOSHalt     SHalt;                 /* SHalt */
    DoPEIOKey       Key[IO_KEY_MAX];       /* Key [0..15] */
    DoPEIOLimitPosition LimitPosition;      /* Limit XHead position */
    DoPEIOLimitSpeed LimitSpeed;            /* Limit XHead speed */
    DoPEIOLimitLoad  LimitLoad;             /* Limit Load */
    DoPEIOGuard      Guard;                 /* Guard option */
} DoPEIOSignals;
```

12.2.1.6.1 DoPEIOGrip

```
/* Grip modes */
#define IO_GRIP_MODE_0_OFF 0 /* grip disabled */
#define IO_GRIP_MODE_1_TANSPARENT 1 /* digital outputs are set according to
    Grip Keys grip leds are set according
    to digital inputs */
#define IO_GRIP_MODE_2_LIMIT_CTRL 2 /* only IO_SIGNAL_OUT_GRIP_OPEN_ENABLE is
    active */
#define IO_GRIP_MODE_3_LIMIT_CTRL_INVERTED 3 /* only IO_SIGNAL_OUT_GRIP_OPEN_ENABLE is
    active (inverted) */

/* Grip IO signal bits */
#define IO_SIGNAL_IN_GRIP_CONNECTED 0x0001
#define IO_SIGNAL_IN_GRIP_UPPER_OPENED 0x0002
#define IO_SIGNAL_IN_GRIP_LOWER_OPENED 0x0004
#define IO_SIGNAL_IN_GRIP_UPPER_CLOSED 0x0008
#define IO_SIGNAL_IN_GRIP_LOWER_CLOSED 0x0010

#define IO_SIGNAL_OUT_GRIP_OPEN_ENABLE 0x0001
#define IO_SIGNAL_OUT_GRIP_UPPER_OPEN 0x0002
#define IO_SIGNAL_OUT_GRIP_LOWER_OPEN 0x0004
#define IO_SIGNAL_OUT_GRIP_UPPER_CLOSE 0x0008
#define IO_SIGNAL_OUT_GRIP_LOWER_CLOSE 0x0010

#define IO_SIGNAL_IN_GRIP_MASK 0x001F
#define IO_SIGNAL_OUT_GRIP_MASK 0x001F

typedef struct                                /* Grip IO signal definition */
{
    __uint16 Mode;                          /* Mode (0=disabled) [No] */
    __uint16 BitDevice;                    /* BitIn/BitOut device [0...9] */
    __uint16 BitNo;                       /* First bit of block [0..15] */

    __uint16 LimitSensorNo;                /* Limit sensor number [No] */
    double LimitMax;                       /* Limit Max [Unit] */
    double LimitMin;                       /* Limit Max [Unit] */
} DoPEIOGrip;
```


12.2.1.6.2 DoPEIOExt

```

/* Extensometer IO signal definition */
#define IO_SIGNAL_IN_EXT_UPPER_OPENED      0x0001
#define IO_SIGNAL_IN_EXT_LOWER_OPENED      0x0002
#define IO_SIGNAL_IN_EXT_UPPER_CLOSED      0x0004
#define IO_SIGNAL_IN_EXT_LOWER_CLOSED      0x0008

#define IO_SIGNAL_OUT_EXT_UPPER_OPEN        0x0001
#define IO_SIGNAL_OUT_EXT_LOWER_OPEN        0x0002
#define IO_SIGNAL_OUT_EXT_UPPER_CLOSE      0x0004
#define IO_SIGNAL_OUT_EXT_LOWER_CLOSE      0x0008

#define IO_SIGNAL_IN_EXT_MASK               0x000F
#define IO_SIGNAL_OUT_EXT_MASK              0x000F

typedef struct                               /* Extensometer IO signal definition */
{                                           /* ----- */
    __uint16 Mode;                          /* Mode (0=disabled)          [No] */
    __uint16 BitDevice;                     /* BitIn/BitOut device        [0...9] */
    __uint16 BitNo;                         /* First bit of block         [0..15] */
    double   IOTime;                       /* Time for BitIn/BitOut control [s] */
} DoPEIOExt;

```

12.2.1.6.3 DoPEIOFixedXHead

```

/* FixedXHead IO signal definition */
#define IO_SIGNAL_IN_FIXED_XHEAD_UNLOCKED  0x0001

#define IO_SIGNAL_OUT_FIXED_XHEAD_UNLOCK    0x0001
#define IO_SIGNAL_OUT_FIXED_XHEAD_UP       0x0002
#define IO_SIGNAL_OUT_FIXED_XHEAD_DOWN     0x0004

#define IO_SIGNAL_IN_FIXED_XHEAD_MASK       0x0001
#define IO_SIGNAL_OUT_FIXED_XHEAD_MASK      0x0007

typedef struct                               /* FixedXHead IO signal definition */
{                                           /* ----- */
    __uint16 Mode;                          /* Mode (0=disabled)          [No] */
    __uint16 BitDevice;                     /* BitIn/BitOut device        [0...9] */
    __uint16 BitNo;                         /* First bit of block         [0..15] */
    double   IOTime;                       /* Time for BitIn/BitOut control [s] */
} DoPEIOFixedXHead;

```

12.2.1.6.4 DoPEIOHighPressure

```

/* High pressure IO signal definition */
#define IO_SIGNAL_IN_HIGH_PRESSURE          0x0001
#define IO_SIGNAL_IN_HIGH_PRESSURE_OK       0x0002
#define IO_SIGNAL_IN_LOW_PRESSURE           0x0004

#define IO_SIGNAL_OUT_HIGH_PRESSURE         0x0001
#define IO_SIGNAL_OUT_HIGH_PRESSURE_OK      0x0002
#define IO_SIGNAL_OUT_LOW_PRESSURE          0x0004
#define IO_SIGNAL_OUT_LOW_PRESSURE_OK       0x0008

#define IO_SIGNAL_IN_HIGH_PRESSURE_MASK     0x0007
#define IO_SIGNAL_OUT_HIGH_PRESSURE_MASK    0x000F

typedef struct                               /* High pressure IO signal definition */
{
    /* ----- */
    __uint16 Mode;                          /* Mode (0=disabled) [No]*/
    __uint16 BitDevice;                     /* BitIn/BitOut device [0...9]*/
    __uint16 BitNo;                         /* First bit of block [0..15]*/
    double   IOTime;                        /* Time for BitIn/BitOut control [s]*/

    __uint16 PressureOutputEnabled;         /* Pressure output enabled [0/1]*/
    __uint16 OutChaNo;                      /* Analog output for pressure [0..15]*/
    double   OutChaLowPressure;              /* Low Pressure value [%]*/
    double   OutChaHighPressure;             /* High Pressure value [%]*/
    double   OutChaRampTime;                 /* Time for low/high pressure ramp [s]*/
} DoPEIOHighPressure;

```

12.2.1.6.5 DoPEIOMisc

```

/* Miscellaneous IO signal definition */
#define IO_SIGNAL_IN_MISC_TEMPERATURE1      0x0001 /* warning */
#define IO_SIGNAL_IN_MISC_TEMPERATURE2      0x0002 /* emergency off */
#define IO_SIGNAL_IN_MISC_OIL_LEVEL         0x0004 /* emergency off */
#define IO_SIGNAL_IN_MISC_OIL_FILTER        0x0008 /* warning */
#define IO_SIGNAL_IN_MISC_POWER_FAIL        0x0010 /* emergency off */

#define IO_SIGNAL_OUT_MISC_CAL               0x0001
#define IO_SIGNAL_OUT_MISC_NO_SENSOR_LIMIT  0x0002

#define IO_SIGNAL_IN_MISC_MASK              0x001F
#define IO_SIGNAL_OUT_MISC_MASK             0x0003

typedef struct                               /* Miscellaneous IO signal definition */
{
    /* ----- */
    __uint16 BitInMode;                      /* BitIn mode (0=disabled) [No]*/
    __uint16 BitInDevice;                    /* BitIn device [0...9]*/
    __uint16 BitInNo;                       /* BitIn number (first of bl.) [0..15]*/

    __uint16 BitOutMode;                     /* BitOut mode (0=disabled) [No]*/
    __uint16 BitOutDevice;                   /* BitOut device [0...9]*/
    __uint16 BitOutNo;                      /* BitOut number (first of bl.) [0..15]*/
} DoPEIOMisc;

```

12.2.1.6.6 DoPEIOShalt

```

/* SHALT IO signal modes */
#define IO_SHALT_MODE_0_OFF          0 /* SHalt IO signal disabled */
#define IO_SHALT_MODE_1_ENABLED      1 /* SHalt IO signal enabled (active high) */
#define IO_SHALT_MODE_2_ENABLED_INVERTED 2 /* SHalt IO signal enabled (active low) */

/* SHALT IO signal definition */
#define IO_SIGNAL_IN_SHALT_UPPER      0x0001
#define IO_SIGNAL_IN_SHALT_LOWER      0x0002
#define IO_SIGNAL_IN_SHALT_MASK       0x0003

typedef struct /* SHalt IO signal definition */
{
    /* ----- */
    __uint16 Mode; /* Mode (0=disabled) [No] */
    __uint16 BitDevice; /* BitIn/BitOut device [0...9] */
    __uint16 BitNo; /* First bit of block [0..15] */
} DoPEIOShalt;

```

12.2.1.6.7 DoPEIOKey

```

#define IO_KEY_MAX 16

typedef struct /* Key IO signal definition */
{
    /* ----- */
    __int32 KeyCode; /* Key code DoPE_KEY_xxx [No] */
    __uint16 Device; /* BitIn/BitOut device [0...9] */
    __uint16 KeyMask; /* Bit mask for key [No] */
    __uint16 LedMask; /* Bit mask for LED [No] */
} DoPEIOKey;

```

12.2.1.6.8 DoPEIOLimitLoad

```

/* Limit IO signal bits */
#define IO_SIGNAL_OUT_LIMIT_OK        0x0001
#define IO_SIGNAL_OUT_LIMIT_MASK      0x0001

typedef struct /* Limit XHead position IO signal def. */
{
    /* ----- */
    __uint16 Mode; /* Mode (0=disabled) [No] */
    __uint16 BitDevice; /* BitIn/BitOut device [0...9] */
    __uint16 BitNo; /* First bit of block [0..15] */

    double PositionMax; /* XHead position limit Max [m] */
    double PositionMin; /* XHead position limit Min [m] */
} DoPEIOLimitPosition;

typedef struct /* Limit XHead speed IO signal def. */
{
    /* ----- */
    __uint16 Mode; /* Mode (0=disabled) [No] */
    __uint16 BitDevice; /* BitIn/BitOut device [0...9] */
    __uint16 BitNo; /* First bit of block [0..15] */

    double SpeedMax; /* XHead speed limit Max [m/s] */
    double SpeedMin; /* XHead speed limit Min [m/s] */
} DoPEIOLimitSpeed;

typedef struct /* Limit Load IO signal definition */
{
    /* ----- */
    __uint16 Mode; /* Mode (0=disabled) [No] */
    __uint16 BitDevice; /* BitIn/BitOut device [0...9] */
    __uint16 BitNo; /* First bit of block [0..15] */

    double LoadMax; /* Load limit Max [N] */
    double LoadMin; /* Load limit Min [N] */
} DoPEIOLimitLoad;

```

12.2.1.6.9 DoPEIOLimitLoad

```

/* Limit IO signal bits */
#define IO_SIGNAL_OUT_LIMIT_OK                0x0001
#define IO_SIGNAL_OUT_LIMIT_MASK             0x0001

typedef struct                                /* Limit XHead position IO signal def.*/
{
    /* ----- */
    __uint16 Mode;                            /* Mode (0=disabled)                [No]*/
    __uint16 BitDevice;                       /* BitIn/BitOut device              [0...9]*/
    __uint16 BitNo;                           /* First bit of block               [0..15]*/

    double   PositionMax;                     /* XHead position limit Max         [m]*/
    double   PositionMin;                     /* XHead position limit Min         [m]*/
} DoPEIOLimitPosition;

typedef struct                                /* Limit XHead speed IO signal def. */
{
    /* ----- */
    __uint16 Mode;                            /* Mode (0=disabled)                [No]*/
    __uint16 BitDevice;                       /* BitIn/BitOut device              [0...9]*/
    __uint16 BitNo;                           /* First bit of block               [0..15]*/

    double   SpeedMax;                        /* XHead speed limit Max            [m/s]*/
    double   SpeedMin;                        /* XHead speed limit Min            [m/s]*/
} DoPEIOLimitSpeed;

typedef struct                                /* Limit Load IO signal definition */
{
    /* ----- */
    __uint16 Mode;                            /* Mode (0=disabled)                [No]*/
    __uint16 BitDevice;                       /* BitIn/BitOut device              [0...9]*/
    __uint16 BitNo;                           /* First bit of block               [0..15]*/

    double   LoadMax;                        /* Load limit Max                   [N]*/
    double   LoadMin;                        /* Load limit Min                   [N]*/
} DoPEIOLimitLoad;

```

12.2.1.6.10DoPEIOGuard

```

/* Guard option modes */
#define IO_GUARD_MODE_0_OFF 0 /* Guard option disabled */
#define IO_GUARD_MODE_1_LIMITED 1 /* Guard option with limited speed and/or
load if guard is open */
#define IO_GUARD_MODE_2_CTRLOFF 2 /* Guard option with inactive controller if
guard is open */

/* Guard option overrun reaction */
#define IO_GUARD_LIMIT_OVERRUN_REACTION_0_STATE 0
#define IO_GUARD_LIMIT_OVERRUN_REACTION_1_DRIVEOFF 1
#define IO_GUARD_LIMIT_OVERRUN_REACTION_2_SHALT 2

/* Guard option request button mode */
#define IO_GUARD_REQUEST_BUTTON_MODE_0_OFF 0
#define IO_GUARD_REQUEST_BUTTON_MODE_1_TOGGLE 1
#define IO_GUARD_REQUEST_BUTTON_MODE_2_SET 2
#define IO_GUARD_REQUEST_BUTTON_MODE_3_STATIC 3

/* Guard option IO bits */
#define IO_SIGNAL_IN_GUARD_CLOSED 0x0001
#define IO_SIGNAL_IN_GUARD_UNLOCK_REQ 0x0002

#define IO_SIGNAL_OUT_GUARD_LIM_OVERRUN 0x0001
#define IO_SIGNAL_OUT_GUARD_UNLOCK_REQ 0x0002
#define IO_SIGNAL_OUT_GUARD_LIM_MODE_ACK 0x0004

#define IO_SIGNAL_IN_GUARD_MASK 0x0003
#define IO_SIGNAL_OUT_GUARD_MASK 0x0007

typedef struct /* Guard Option IO signal def. */
{
    /* ----- */
    __uint16 Mode; /* Mode (0=disabled) [No] */
    __uint16 BitDevice; /* BitIn/BitOut device [0...9] */
    __uint16 BitNo; /* First bit of block [0..15] */

    __uint16 SpeedLimitEnable; /* Speed limit enable [0/1] */
    __uint16 SpeedLimitSensor; /* Control sensor for speed limit [0..15] */
    double SpeedLimit; /* Speed limit [Unit/s] */
    double SpeedLimitTolerance; /* Speed limit overrun tolerance [Unit/s] */
    __uint16 RangeLimitEnable; /* Range limit enable [0/1] */
    __uint16 RangeLimitSensor; /* Control sensor for range limit [0..15] */
    double UpperRangeLimit; /* Upper range limit [Unit] */
    double LowerRangeLimit; /* Lower range limit [Unit] */
    double RangeLimitTolerance; /* Range limit overrun tolerance [Unit] */
    __uint16 LimitOverrunReaction; /* Reaction to a limit overrun [0..2] */
    __uint16 LowPressureActEnable; /* Low pressure activation enable [0/1] */
    __uint16 RequestButtonMode; /* Mode of guard request button [0..3] */
    double MonitoringDelay; /* Delay time for guard monitoring [s] */
} DoPEIOGuard;

```

12.2.1.7 DoPEMainMenu:

```
#define MAX_MAIN_MENU 40          /* max count of EDC main menu entries */
enum                             /* Test numbers of EDC main menu */
{
    TEST_NO_PC_CONTROL = 10000,  /* 10000 no licence needed */
    TEST_NO_TENSION_COMPRESSION, /* 10001 no licence needed */
    TEST_NO_TEAR,                 /* 10002 no licence needed */
    TEST_NO_METAL_TENSION,        /* 10003 licence needed */
    TEST_NO_CONCRETE_PRESSURE,    /* 10004 no licence needed */
    TEST_NO_CONCRETE_BENDING,     /* 10005 no licence needed */
    TEST_NO_CONCRETE_BRASILIAN,   /* 10006 no licence needed */
    TEST_NO_CONCRETE_CIRCEL_BENDING, /* 10007 no licence needed */
    TEST_NO_CYCLES,               /* 10008 licence needed */
    TEST_NO_EXTERNAL_COMMAND,     /* 10009 licence needed */
    TEST_NO_CREEP,                /* 10010 licence needed */
    TEST_NO_BLOCK_COMMAND,        /* 10011 licence needed */
    TEST_NO_ADJUSTING,            /* 10012 no licence needed */
    TEST_NO_CALIBRATION,          /* 10013 no licence needed */
    TEST_NO_CLOSED_LOOP_SETUP,    /* 10014 no licence needed */
    TEST_NO_DRIVE_SETUP,          /* 10015 no licence needed */
    TEST_NO_PROTOCOL_SETUP,       /* 10016 no licence needed */
    TEST_NO_USER_SETUP,           /* 10017 no licence needed */
    TEST_NO_SYNC_OPTION,          /* 10018 licence needed */
    TEST_NO_GUARD_OPTION,         /* 10019 licence needed */

    TEST_NO_MAX,
};

typedef struct                   /* EDC main menu definition */
{
    __int32 TestNo;              /* Test number [No] */
    __uint16 Visible;            /* Main menu entry visible [0/1] */
} DoPEMainMenu;
```

12.2.2 DoPERdMachine

Read the total machine structure.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl unsigned short MachineNo DoPEMachine *Machine	Function returns Error constant (DoPERR_XXXX) DoPE link handle Machine Number Pointer to machine data structure

.NET <Edc object>.Setup.RdMachine(DoPE.MACHINE_NUMBER MachineNo, ref DoPE.Machine Machine)

12.2.3 DoPEWrMachine

Write the total machine structure.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl unsigned short MachineNo DoPEMachine *Machine	Function returns Error constant (DoPERR_XXXX) DoPE link handle Machine Number Pointer to machine data structure

.NET <Edc object>.Setup.WrMachine(DoPE.MACHINE_NUMBER MachineNo, DoPE.Machine Machine)

12.2.4 DoPEMachineScale

Sets the machine user scale.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl unsigned short MachineNo UserScale US)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Machine Number User scale for all machine sensor data e.g. use this to convert the SI unit meter into mm by setting the UserScale to 1000 for the position sensor Default values 1.0 will be used if US is NULL.
.NET <Edc object>.Setup.MachineScale(DoPE.MACHINE_NUMBER MachineNo, DoPE.UserScale US)	

12.2.5 DoPERdMachineNumber

Read currently selected machine number.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl unsigned short * MachineNo)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Pointer to store machine number
.NET <Edc object>.Setup.RdMachineNumber(ref DoPE.MACHINE_NUMBER MachineNo)	

12.2.6 DoPESelMachine

Select a machine and initialize.

Machine number 1 to 8 are stored inside the EDC flash disk. Machine number 0 is the working set. If machine 1 to 8 is selected, machine data and basic tare values are copied from the flash disk to the working set 0. If machine 0 is selected, set-up data in the working set 0 are not altered. Basic tare values cannot be stored permanently inside EDC. Thus if machine 0 was written completely by PC, this data are used to initialize the system. The DoPESelMachine function calls DoPEInitialize to do system initialization.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl unsigned short MachineNo UserScale US)	Function returns Error constant (DoPERR_XXXX) DoPE link handle Machine No. User scale for all measuring channels. The measured data will be multiplied with the value in US. E.g. use this to convert the SI unit meter into mm by setting the UserScale to 1000 for the position channel. If US = NULL, user scale will be set to 1.0
.NET <Edc object>.Setup.SelMachine(DoPE.MACHINE_NUMBER MachineNo, DoPE.UserScale US)	

12.3 DoPEInitialize

Initialize System with selected machine data. This command must be called after a change of machine data was made without selecting a new machine.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEInitialize (DoPEHdl)	Function returns Error constant (DoPERR_xxxx) DoPE link handle TAN's are used to identify initialization errors sent in addition to the synchronized return code. (This will be implemented in future EDC versions. In Version 2.11 we have synchronized return codes but only one error code sent on the PE_INITIALIZE command.)
.NET <Edc object>.Setup.Initialize()	

12.4 DoPEInitializeResetXHead

Initialize System with selected machine data and reset the crosshead position.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEInitializeResetXHead (DoPEHdl)	Function returns Error constant (DoPERR_xxxx) DoPE link handle
.NET <Edc object>.Setup.InitializeResetXHead()	

12.5 DoPERdSensorInfo

Read summary sensor information's.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPERdSensorInfo (DoPEHdl unsigned short SensorNo DoPESumSenInfo *Info)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Sensor Number. Pointer for SumSenInfo structure.

DoPESumSenInfo structure:

```
typedef struct                                /* Summary Sensor Information */
{                                              /* ----- */
    WORD    Connector;                        /* Connector number of sensor [No] */
    double  NominalValue;                     /* Nominal value of sensor [Unit] */
    WORD    Unit;                             /* Unit of sensor UNIT_XXX [No] */
    double  Offset;                           /* Offset of sensor [Unit] */
    double  UpperLimit;                       /* Upper range limit of sensor [Unit] */
    double  LowerLimit;                       /* Lower range limit of sensor [Unit] */
    WORD    SensorState;                      /* Sensor state SEN_STATE_XXX [No] */
    WORD    McType;                           /* Measuring channel type [No] */
    double  UpperSoftLimit;                   /* Upper soft limit [Unit] */
    double  LowerSoftLimit;                   /* Lower soft limit [Unit] */
    WORD    SoftLimitReaction;                 /* reaction if soft limit [No] */
    double  BasicTare;                         /* Basic tare [Unit] */
    double  Tare;                             /* Tare [Unit] */
    double  UserScale;                         /* User scale [No] */
    double  McFilterTime;                      /* Measuring channel filter time [s] */
    double  CtrlFilterTime;                    /* Closed loop control filter time [s] */
    double  HwDelayTime;                       /* Hardware delaytime [s] */
    double  McDelayTime;                       /* Measuring channel delaytime [s] */
    double  McDelayTimeCorr;                   /* Delaytime correction [s] */
} DoPESumSenInfo;
```

.NET <Edc object>.Setup.RdSensorInfo(DoPE.SENSOR SensorNo, ref DoPE.SumSenInfo Info)

12.6 DoPERdSensorUserData

Read sensor user data (128 Byte). The application program may use this 128 bytes EEPROM data to store information about the sensor. The EEPROM is located inside the sensor plug!

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl WORD Connector BYTE *SenUsrData unsigned Length	Function returns Error constant (DoPERR_xxxx) DoPE link handle Connector number of sensor Pointer for sensor user data User data buffer length in BYTE's
.NET <Edc object>. Sensor.RdSensorUserData(DoPE.CONNECTOR Connector, ref Byte[] SenUsrData)	

12.7 DoPEWrSensorUserData

Write sensor user data (128 Byte).

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl WORD Connector BYTE *SenUsrData unsigned Length)	Function returns Error constant (DoPERR_xxxx) DoPE link handle Connector number of sensor Pointer for sensor user data User data buffer length in BYTE's.
.NET <Edc object>. Sensor.WrSensorUserData(DoPE.CONNECTOR Connector, Byte[] SenUsrData)	

12.8 DoPEwRdModuleInfo

Read module info.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPE_HANDLE DoPEHdl unsigned ModulNo DoPEwModuleInfo *Info	Function returns Error constant (DoPERR_xxxx) DoPE link handle Module number (0 ... MAX_MODULE-1) Pointer to storage for the module info
<pre>typedef struct { DWORD ID; DWORD Revision; DWORD DeviceID; DWORD FunctionID; DWORD SerNr; DWORD Status; wchar_t Name[MODINFO_NAME_LEN]; } DoPEModuleInfo;</pre> /* Definition of module info */ /* ----- */ /* Module hardware ID [No] */ /* Module hardware revision [No] */ /* Device ID [No] */ /* Function ID (from MachineDef) [No] */ /* Serial number [No] */ /* Module status [No] */ /* Module name (e.g. "2INC 1742.000(-)") */	
.NET <Edc object>. Info.RdModuleInfo(Int32 ModulNo, ref DoPE.ModuleInfo ModuleInfo)	

12.9 DoPEwRdDriveInfo

Read drive info.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI	DoPEwRdDriveInfo(	Function returns Error constant (DoPERR_XXXX)	
DoPE_HANDLE	DoPEHdl	DoPE link handle	
WORD	Connector	Module number (0 ... MAX_MODULE-1)	
DoPEwDriveInfo	*Info	Pointer to storage for the drive info	
<pre> typedef struct /* Definition of drive info */ { /* ----- */ DWORD ID; /* Drive hardware ID [No] */ DWORD Revision; /* Drive hardware revision [No] */ DWORD SerNr; /* Serial number [No] */ double NominalCurrent; /* Nominal current [A] */ double MinCurrent; /* Minimum current [A] */ double MaxCurrent; /* Maximum current [A] */ double MaxCurrentTime; /* Maximum current time (I²t) [s] */ int NominalVoltageCount; /* Number of nominal voltages [No] */ double NominalVoltage[MAX_NOMV]; /* Nominal voltage [V] */ double MaxPower; /* Maximum power [W] */ double FeedbackPower; /* Average feedback power [W] */ double MinDither; /* Minimum dither frequency [Hz] */ double MaxDither; /* Maximum dither frequency [Hz] */ double CurrentControllerCycle; /* Current controller cycle time [s] */ /* (0 = analogue or not adjustable) */ wchar_t Name[DRIVEINFO_NAME_LEN]; /* Drive name (e.g. "DC2500 1860.000(-)") */ } DoPEwDriveInfo; </pre>			
.NET	<Edc object>.Info.RdDriveInfo(DoPE.CONNECTOR Connector, ref DoPE.DriveInfo DriveInfo)		

13 Sensor EEPROM Handling

Following functions support read and write of sensor-EEPROM data.

ATTENTION: **Use these functions very carefully.**
 DoPE cannot check the total integrity of the data!
 UserScale has no effect on the Sensor EEPROM data!

These functions work on initialized and not initialized sensors. The sensors are selected by the connector number.

The data inside the sensor EEPROM are divided into two sections. The first section (DoPESensorHeaderData) is identical for all sensors. The second section depends of the sensor class. If you want to change data inside the sensor EEPROM, please follow the procedure described below:

1. Use DoPERdSensorConKey function and check if a sensor is connected.
2. Use DoPERdSensorHeaderData function to read header data.
3. Analyze the sensor class,
 for DoPESensorHeaderData.Class = SEN_ANALOGUE use DoPERdSensorAnalogueData,
 for DoPESensorHeaderData.Class = SEN_INC use DoPERdSensorIncData,
 for DoPESensorHeaderData.Class = SEN_ABS use DoPERdSensorAbsData
 to read the sensor class specific data.
4. Modify data.
5. Use the appropriate write function to store the data into the sensor EEPROM.

13.1 DoPERdSensorConKey

Read sensor plug connected and key state.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI	DoPERdSensorConKey (	Function returns Error constant (DoPERR_ xxxx)	
DoPE_HANDLE	DoPEHdl	DoPE link handle	
WORD	Connector	Connector number of sensor	
WORD	*Connected	Pointer to the sensor plug connected state (0=not connected, 1=connected)	
WORD	*KeyPressed	Pointer to the sensor plug key state (0=not pressed, 1=pressed)	
.NET <Edc object>.Sensor. RdSensorConKey(DoPE.CONNECTOR Connector, ref bool Connected, ref bool KeyPressed)			

13.2 DoPERdSensorHeaderData

Read sensor EEPROM data header.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00			
Function declaration		Description	
extern unsigned DLLAPI	DoPERdSensorHeaderData (	Function returns Error constant (DoPERR_ xxxx)	
DoPE_HANDLE	DoPEHdl	DoPE link handle	
WORD	Connector	Connector number of sensor	
DoPESensorHeaderData	*SenHdrData	Pointer to sensor data header structure	
.NET <Edc object>.Sensor.RdSensorHeaderData(DoPE.CONNECTOR Connector, ref DoPE.SensorHeaderData SenHdrData)			

13.3 DoPEWrSensorHeaderData

Write sensor EEPROM data header.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPEWrSensorHeaderData (DoPE_HANDLE DoPEHdl WORD Connector DoPESensorHeaderData *SenHdrData	Function returns Error constant (DoPERR_XXXX) DoPE link handle Connector number of sensor Pointer to sensor data header structure
DoPESensorHeaderData structure: <pre> typedef struct /* Sensor EEPROM header data */ { /* ----- */ WORD PartNo; /* Part indent number [No] */ BYTE Version; /* Part revision [No] */ DWORD SerNo; /* Part serial number [No] */ WORD Class; /* Sensor class [No] */ BYTE DatVersion; /* Version of data [No] */ } DoPESensorHeaderData; Write protected /* Sensor classes /* ----- */ #define SEN_UNDEF 0 /* unknown sensor class #define SEN_ANALOGUE 1 /* analogue sensor #define SEN_INC 2 /* incremental sensor #define SEN_ABS 3 /* absolute value sensor /* Analogue sensor types /* ----- */ #define SIG_STRAINGAUGE 0 /* Strain gauge #define SIG_LVDT 1 /* LVDT #define SIG_DC 2 /* DC /* Incremental sensor types /* ----- */ #define SIG_TTL 0 /* TTL Signal #define SIG_LINE 1 /* RS422 (line driver) #define SIG_SINE11uA 2 /* Sine 11uA #define SIG_SINE1V 3 /* Sine 1V /* Absolute value sensor types /* ----- */ #define SIG_UNDEF 0 /* undefined #define SIG_TR_LT_S 1 /* TR LT-S Sensor /* Transducer types /* ----- */ #define TRANSDUCER_LINEAR 0 /* Linear transducer #define TRANSDUCER_ROTARY 1 /* Rotary transducer /* Reference mark types /* ----- */ #define REFMARK_NON 0 /* Transducer has no reference mark #define REFMARK_ONE 1 /* Transducer has one reference mark #define REFMARK_DISTCODE 2 /* Transducer has distance coded </pre>	
.NET	<Edc object>.Sensor.WrSensorHeaderData(DoPE.CONNECTOR Connector, ref DoPE.SensorHeaderData SenHdrData)

13.4 DoPERdSensorAnalogueData

Read analogue sensor data.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPERdSensorAnalogueData (DoPE_HANDLE DoPEHdl WORD Connector DoPESensorAnalogueData *SenAnalogueData	Function returns Error constant (DoPERR_XXXX) DoPE link handle Connector number of sensor Pointer to analogue sensor data structure	
.NET <Edc object>.Sensor. RdSensorAnalogueData(DoPE.CONNECTOR Connector, ref DoPE.SensorAnalogueData SensorAnalogueData)		

13.5 DoPEWrSensorAnalogueData

Write analogue sensor data.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPEWrSensorAnalogueData (DoPE_HANDLE DoPEHdl WORD Connector DoPESensorAnalogueData *SenAnalogueData	Function returns Error constant (DoPERR_XXXX) DoPE link handle Connector number of sensor Pointer to analogue sensor data structure	
DoPESensorAnalogueData structure: <pre> typedef struct { float MaxExcitation; /* Analogue sensor EEPROM data */ WORD MinImpedance; /* ----- */ float NominalValue; /* Maximum excitation voltage [V] */ WORD Unit; /* Impedance [Ohm] */ float Offset; /* Nominal value of the sensor [Unit] */ WORD NegLimit; /* Unit of sensor UNIT_XXX [No] */ WORD PosLimit; /* Sensor offset [Unit] */ float Reference; /* Range limit - min. [%] */ double CorrReference; /* Range limit - max. [%] */ WORD Sensortype; /* Nominal value of the reference [*] */ double NominalSensitive; /* Corr. value of the reference [No] */ WORD Sign; /* Sensor type */ double InvertSign; /* Sensitivity at Nominal value [*] */ int Day; /* Invert sign of channel [1/0] */ int Month; /* Date of last change [No] */ int Year; /* Date of last change [No] */ WORD LinPoint; /* Date of last change [No] */ struct LinVal { double MeasValue; /* Number of linearization steps [No] */ double RefValue; /* Measured value [Unit] */ double LinV[SEN_LIN_DATA_MAX]; /* Reference [Unit] */ } LinV[SEN_LIN_DATA_MAX]; /* Linearization table */ } DoPESensorAnalogueData; </pre> Write protected		
.NET <Edc object>.Sensor.WrSensorAnalogueData(DoPE.CONNECTOR Connector, DoPE.SensorAnalogueData SensorAnalogueData)		

13.6 DoPERdSensorIncData

Read incremental sensor data.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPERdSensorIncData (DoPE_HANDLE DoPEHdl WORD Connector DoPESensorIncData *SenIncData	Function returns Error constant (DoPERR_xxxx) DoPE link handle Connector number of sensor Pointer to incremental sensor data structure	
.NET <Edc object>.Sensor.RdSensorIncData(DoPE.CONNECTOR Connector, ref DoPE.SensorIncData SenIncData)		

13.7 DoPEWrSensorIncData

Write incremental sensor data.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPEWrSensorIncData (DoPE_HANDLE DoPEHdl WORD Connector DoPESensorIncData *SenIncData	Function returns Error constant (DoPERR_xxxx) DoPE link handle Connector number of sensor Pointer to incremental sensor data structure	
DoPESensorIncData structure: <pre> typedef struct /* Incremental sensor EEPROM data */ { /* ----- */ float Voltage1; /* Supply voltage 1 [V] */ float Voltage2; /* Supply voltage 2 [V] */ float Voltage3; /* Supply voltage 3 [V] */ float Current1; /* Current for supply voltage 1 [A] */ float Current2; /* Current for supply voltage 2 [A] */ float Current3; /* Current for supply voltage 3 [A] */ WORD InputSignal; /* Signal type at input SIG_xxx [No] */ WORD OutputSignal; /* Signal type at output SIG_xxx [No] */ WORD InterpolationFactor; /* Factor for interpolation [No] */ float MaxInputFreq; /* Maximum input frequency [Hz] */ float MaxOutputFreq; /* Maximum output frequency [Hz] */ WORD TransducerType; /* Transducer type TRANSDUCER_xxx [No] */ WORD Unit; /* Unit of sensor UNIT_xxx [No] */ double SignalPeriod; /* Signal period [Unit] */ double CorrFactor; /* Correction factor [No] */ double MeasuringRange; /* Measuring range [Unit] */ WORD SignalType; /* Transducer signal type SIG_xxx [No] */ WORD ReferenceMark; /* Reference mark type REFMARK_xxx [No] */ double FirstDistance; /* First distance of the reference [Unit] */ double NominalDistance; /* Nominal distance of the reference [Unit] */ double Delta; /* Dislocation of the mean reference [Unit] */ float LimitFrequency; /* Limit frequency of the transducer [Hz] */ WORD Sign; /* Invert sign of channel [1/0] */ BYTE NegLimit; /* Range limit - min. [%] */ BYTE PosLimit; /* Range limit - max. [%] */ int Day; /* Date of last change [No] */ int Month; /* Date of last change [No] */ int Year; /* Date of last change [No] */ } DoPESensorIncData; </pre> Write protected		
.NET <Edc object>.Sensor.WrSensorIncData(DoPE.CONNECTOR Connector, DoPE.SensorIncData SenIncData)		

13.8 DoPERdSensorAbsData

Read absolute sensor data.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPERdSensorAbsData (DoPE_HANDLE DoPEHdl WORD Connector DoPESensorAbsData *SenAbsData	Function returns Error constant (DoPERR_XXXX) DoPE link handle Connector number of sensor Pointer to absolute sensor data structure	
.NET <Edc object>.Sensor.RdSensorAbsData(DoPE.CONNECTOR Connector, ref DoPE.SensorAbsData SenAbsData)		

13.9 DoPEWrSensorAbsData

Write absolute sensor data.		
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00		
Function declaration	Description	
extern unsigned DLLAPI DoPEWrSensorAbsData (DoPE_HANDLE DoPEHdl WORD Connector DoPESensorAbsData *SenAbsData	Function returns Error constant (DoPERR_XXXX) DoPE link handle Connector number of sensor Pointer to absolute sensor data structure	
DoPESensorAbsData structure: <pre> typedef struct { float Voltage1; /* Supply voltage 1 [V] */ float Voltage2; /* Supply voltage 2 [V] */ float Voltage3; /* Supply voltage 3 [V] */ float Current1; /* Current for supply voltage 1 [A] */ float Current2; /* Current for supply voltage 2 [A] */ float Current3; /* Current for supply voltage 3 [A] */ WORD InputSignal; /* Signal type at input SIG_XXX [No] */ WORD OutputSignal; /* Signal type at output SIG_XXX [No] */ float MaxInputFreq; /* Maximum input frequency [Hz] */ float MaxOutputFreq; /* Maximum output frequency [Hz] */ BYTE DelayTime; /* Sensors signal delay time [ms/10] */ WORD Unit; /* Unit of sensor UNIT_XXX [No] */ double SignalPeriod; /* Signal period [Unit] */ float Offset; /* Sensor offset [Unit] */ double CorrFactor; /* Correction factor [No] */ double NominalValue; /* Nominal value of the sensor [Unit] */ WORD SignalType; /* Transducer signal type SIG_XXX [No] /* or SIG_SSI_GENERIC + code + number /* of data bits float LimitFrequency; /* Limit frequency of the transducer [Hz] */ WORD Sign; /* Invert sign of channel [1/0] */ BYTE NegLimit; /* Range limit - min. [%] */ BYTE PosLimit; /* Range limit - max. [%] */ int Day; /* Date of last change [No] */ int Month; /* Date of last change [No] */ int Year; /* Date of last change [No] */ } DoPESensorAbsData; </pre> Write protected		
.NET <Edc object>.Sensor.WrSensorAbsData(DoPE.CONNECTOR Connector, DoPE.SensorAbsData SenAbsData)		

13.10 DoPECalcSensorAbsGenericSignalType

Get the "SignalType" of a SSI/BISS absolute sensor by its generic data.			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE10.23			
Function declaration		Description	
extern unsigned DLLAPI	DoPESetSsiGenericSignalType(unsigned short *SignalType unsigned Protocol unsigned Code unsigned Bits	Function returns Error constant (DoPERR_XXXX) Pointer to signal type variable Protocol of the absolute sensor Code of the sensor binary or gray Number of data bits of the sensor 8 to 32	
.NET <Edc object>.Sensor.CalcSensorAbsGenericSignalType (ref Int16 SignalType, DoPE.ABS_TYPES Protocol, DoPE.ABS_CODE Code, Int32 Bits)			

13.11 DoPECalcSensorAbsGenericData

Get the generic data of a SSI/BISS absolute sensor by its "SignalType".			
Minimum requirements: EDCi with EDCiApp 9149.001, DoPE10.23			
Function declaration		Description	
extern unsigned DLLAPI	DoPESSiGenericSignalTypeInfo(unsigned short SignalType unsigned *Protocol unsigned *Code unsigned *Bits	Function returns Error constant (DoPERR_XXXX) Signal type of the SSI sensor Pointer to storage for protocol of the absolute sensor Pointer to storage for code of the sensor binary or gray Pointer to storage for number of the sensor 8 to 32	
.NET <Edc object>.Sensor.CalcSensorAbsGenericDataByte (int16 SignalType, ref DoPE.ABS_TYPES Protocol, ref DoPE.ABS_CODE Code, ref Int32 Bits)			

14 Reestablishing a Connection

For long-term experiments, it may be useful to close an open link without interrupting a running movement command. At a later time the connection can be resumed. For this DoPE provides two functions. One enables the so called ReInitialize Mode and retrieves the current state of the connection in the ReInitialize Data. The other one recovers the state of the connection with this data.

14.1 DoPEReInitializeEnable

Activates the ReInitialize Mode and provides ReInitialize Data. This data will be passed to the DoPEReInitialize function to restore the communication link to the current state. DoPESelfSetup and the DoPEInitialize functions set the the ReInitialize Mode to the disabled state.

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPEReInitializeEnable (DoPE_HANDLE DoPEHdl unsigned Enable DoPEReInitializeData* ReInitData)	Function returns Error constant (DoPERR_XXXX) DoPE link handle !=0 enables, =0 disables the ReInitialize mode pointer to storage for ReInitialize data
<pre>#define DoPEREINITIALIZEDATA_LEN 0x8000 typedef BYTE DoPEReInitializeData [DoPEREINITIALIZEDATA_LEN]; /* ReInitialize data array [No]*/</pre>	
.NET <Edc object>.Setup.ReInitializeEnable(bool Enable, ref byte[] ReInitData)	

14.2 DoPEReInitialize

Recover a previously disconnected communication link.

Attention: DoPEReInitialize checks that:

- the EDC is still in ReInitialize Mode.
- the DoPEAPIVERSION of the current DoPE session matches the one used when the ReInitData were retrieved.
- the DeviceID didn't change.
- the ReInitData contains valid data (CRC check).

Minimum requirements: EDCi with EDCiApp 9149.001, DoPE 10.00

Function declaration	Description
extern unsigned DLLAPI DoPEReInitialize (DoPE_HANDLE DoPEHdl DoPEReInitializeData *ReInitData)	Function returns Error constant (DoPERR_XXXX) DoPE link handle pointer to ReInitialize data
.NET <Edc object>.Setup.ReInitialize(ref byte[] ReInitData)	

Here a “C” sample program:

First session of the PC Program:

```
// open the communication link
DoPEErr = DoPEErr = DoPEOpenFunctionID( ... &DoPEHdl );
if( DoPEErr != DoPERR_NOERROR)
    //add errorhandling here
    return 0;

// select a setup
DoPEErr = DoPESelSetup ( DoPEHdl, 1, UserScale, NULL, NULL );
if(DoPEErr != DoPERR_NOERROR)
    //add errorhandling here

// add event handler and other application stuff
...

// switch drive on
DoPEErr = DoPEOn ( DoPEHdl );
if(DoPEErr != DoPERR_NOERROR)
    //add errorhandling here

// start a long term test
DoPEErr = DoPEDynCycle ( DoPEHdl, ... HalfCycles = 10000000, ... );
if(DoPEErr != DoPERR_NOERROR)
    //add errorhandling here

// enable the Reinitialize Mode and keep the ReInitialize Data
DoPEReInitializeData ReInitData;
DoPEErr = DoPEReInitializeMode ( DoPEHdl, true, &ReInitData );
if(DoPEErr != DoPERR_NOERROR)
    //add errorhandling here

...save ReInitDat to Disk...

// Close the communication link
DoPEErr = DoPECloseLink ( &DoPEHdl );
if(DoPEErr != DoPERR_NOERROR)
    //add errorhandling here
```

Recovery session of the PC Program:

```
// open the communication link
DoPEErr = DoPEErr = DoPEOpenFunctionID( ... &DoPEHdl );
if( DoPEErr != DoPERR_NOERROR)
    //add errorhandling here
    return 0;

// recover communication link with previously stored ReInitData
...load ReInitData from disk
DoPEErr = DoPEReInitialize ( DoPEHdl, &ReInitData );
if(DoPEErr != DoPERR_NOERROR)
    //add errorhandling here

// add event handler and other application stuff
...
```

15 Default Measuring Data Record

double	Position	X-Head position
double	Load	Load
double	Extension	Extension
double	Sensor3	Sensor 3 measuring channel
double	Sensor4	Sensor 4 measuring channel
double	Sensor5	Sensor 5 measuring channel
double	Sensor6	Sensor 6 measuring channel
double	Sensor7	Sensor 7 measuring channel
double	Sensor8	Sensor 8 measuring channel
double	Sensor9	Sensor 9 measuring channel
double	Sensor10	Sensor 10 measuring channel
double	Sensor11	Sensor 11 measuring channel
double	Sensor12	Sensor 12 measuring channel
double	Sensor13	Sensor 13 measuring channel
double	Sensor14	Sensor 14 measuring channel
double	Sensor15	Sensor 15 measuring channel
double	Command	Command
double	Feedback	Feedback
double	Output	Output
double	Time	Time from subsystem
unsigned long	Cycles	Cycle counter for cyclic commands
unsigned long	BlockCycles	Cycle counter for block commands
unsigned long	BlockLine	Current block command line
double	CmdFrequency	Frequency for cycling commands
unsigned short	BitIn0	Digital input device 0
unsigned short	BitIn1	Digital input device 1
unsigned short	BitIn2	Digital input device 2
unsigned short	BitIn3	Digital input device 3
unsigned short	BitIn4	Digital input device 4
unsigned short	BitIn5	Digital input device 5
unsigned short	BitIn6	Digital input device 6
unsigned short	BitIn7	Digital input device 7
unsigned short	BitIn8	Digital input device 8
unsigned short	BitIn9	Digital input device 9
unsigned short	BitOut0	Digital output device 0
unsigned short	BitOut1	Digital output device 1
unsigned short	BitOut2	Digital output device 2
unsigned short	BitOut3	Digital output device 3
unsigned short	BitOut4	Digital output device 4
unsigned short	BitOut5	Digital output device 5
unsigned short	BitOut6	Digital output device 6
unsigned short	BitOut7	Digital output device 7
unsigned short	BitOut8	Digital output device 8
unsigned short	BitOut9	Digital output device 9
unsigned short	BitOutErrorHigh	Digital output error high (24V)
unsigned short	BitOutErrorLow	Digital output error low (0V)
unsigned short	InSignals	Logical input signals definition (see below)
unsigned short	OutSignals	Logical output signals definition (see below)
unsigned short	CtrlState1	Controller state WORD 1 definition (see below)
unsigned short	CtrlState2	Controller state WORD 2 definition (see below)
unsigned short	UpperLimits	Upper range limits exceeded (Bit0=Position...Bit15=Sensor15)
unsigned short	LowerLimits	Lower range limits exceeded (Bit0=Position...Bit15=Sensor15)
unsigned short	ActiveCtrl	Active control channel (Bit0=Position...Bit15=Sensor15)
unsigned short	UpperSft	Upper soft limit active (Bit0=Position...Bit15=Sensor15)
unsigned short	LowerSft	Lower soft limit active (Bit0=Position...Bit15=Sensor15)

unsigned short	SensorConnected	Sensor plug connected state (Bit0=Position...Bit15=Sensor15)
unsigned short	SensorKeyPressed	Sensor plug key state (Bit0=Position...Bit15=Sensor15)
unsigned short	DrvIntfStateIn1	Drive interface state input 1 (see below)
unsigned short	DrvIntfStateIn2	Drive interface state input 2 (see below)
unsigned short	DrvIntfStateOut	Drive interface state output (see below)
unsigned short	SafetyState	Safety state (reserved)
unsigned long	ModuleError	Error bits for installed modules (Bit0=Mainboard, Bit1=X21, Bit2=X22...)

15.1 Logical Input Signals

The logical input signals are functional relevant digital inputs. They are configured during initialization and one logical input signal may map to more than one digital input. The state of the logical input signals is maintained in the measuring data record (InSignals).

Bit number	Constant	Description
0	IN_SIG_DRIVE_OFF (IN_SIG_ESTOP)	Drive off or emergency off If this signal is active, the machine control remains in the emergency stop state. Possible sources are limit switches, range limit active, emergency stop button.
1	IN_SIG_E_MOVE_RQ (IN_SIG_ESTOP_DRIVING_FREE)	Emergency off, emergency movement required This signal also forces the machine control into the emergency stop state, but can be masked during a driving free movement.
2	IN_SIG_UPPER_LIMIT_SWITCH	Upper hard-limit switch active This signal force the machine control into the emergency stop state and can be masked during a driving free movement.
3	IN_SIG_LOWER_LIMIT_SWITCH	Lower hard-limit switch active This signal force the machine control into the emergency stop state and can be masked during a driving free movement.
4	IN_SIG_NO_CTRL (IN_SIG_DRIVE_NOT_READY)	Drive not ready (stop) This signal reports the state of the connected drive unit. If set, the drive is not ready.
5	IN_SIG_DF_KEY (for internal use only)	Drive free withdrawn by key
6	IN_SIG_SHALT	Signal S-HALT activated The signal is especially suitable for the connection of a HALT key, the cross-head can be halted with independently of the actual operating state.
7	IN_SIG_REFERENZ (IN_SIG_REFSWITCH_X_HEAD)	X-head reference switch This signal reports the state a reference switch at the X-Head.
8	(IN_SIG_ENABLE_LIMIT_POS)	unused
9	(IN_SIG_ENABLE_LIMIT_SPEED)	unused
10	(IN_SIG_ENABLE_LIMIT_LOAD)	unused
11	IN_SIG_IO_SHALT_UPPER	IO-Signal SHaltUpper
12	IN_SIG_IO_SHALT_LOWER	IO-Signal SHaltLower
13	IN_SIG_CPU_EMERGENCY_OFF	Internal emergency off
14	IN_SIG_ESTOP_UP_DOWN	Upper/Lower limit switch

Former signal names of EDC220/580 in brackets

15.2 Logical Output Signals

The logical output signals are digital outputs to control the drive and associated devices. They are configured during initialization. The state of the logical output signals is maintained in the measuring data record (OutSignals).

Bit number	Constant	Description
0	O_SIG_SOFTWARE_READY (O_SIG_DRIVE_ON)	Drive ON output Being in the inactive state, this output signal should force the drive into its defined emergency stop state. If the signal is active , the drive should change after an arbitrary period of time into the 'operational' state. 'Operational' means that the drive can react upon the output signal 'Drive Free' (see below) without delay.
1	O_SIG_DRIVE_ENABLE (O_SIG_DRIVE_FREE)	Drive enable output With this signal, the external drive may be enabled and disabled.
2	O_SIG_BRAKE_RELEASE (O_SIG_BRAKE)	Brake output The signal 'Brake Release' is linked with the signal 'Drive Free' because of their fault levels, but will be operated with a time shift due to their corresponding change of states. The time shift should enable systems with mechanical brakes to achieve a controlled transition between braking and position control.
3	O_SIG_E_MOVE	Emergency movement output The signal 'Emergency Movement' is only active, as long as the Subsystem executes an emergency movement. It can be used to deactivate the function of the machine limit switches on the drive system.
4	O_SIG_BYPASS	Bypass output
5	O_SIG_MC_LIMIT (O_SIG_LIMIT)	Load limit for grip reached This signal is used by the measuring channel supervision activated by DoPESetCheckLimit command.
7	O_SIG_ON_RELAY	ON Relay output
9	O_SIG_INTERNAL_DRIVE_ENABLE	Internal Drive enable /disable
10	O_SIG_READY_TO_MOVE	Controller is ready for positioning .commands
11	O_SIG_EDC_READY	EDC ready
12	O_SIG_DRIVE_OFF	EDC drive off
13	O_SIG_OUT_LIMIT_POS	IO Limit Position exceeded
14	O_SIG_OUT_LIMIT_SPEED	IO Limit Speed exceeded
15	O_SIG_OUT_LIMIT_LOAD	IO Limit Load exceeded

Former signal names of EDC220/580 in brackets

15.3 Controller State WORD 1

State word 1 of the closed loop controller (CtrlState1)		
Bit number	Constant	Description
4	CTRL_HALT	Command generator halts (controlled halt in S/F/E).
5	CTRL_DOWN	Movement DOWN
6	CTRL_UP	Movement UP
7	CTRL_MOVE	X-Head moves (not halted) This bit is always set when the cross-head is not securely halted. The bit is not set with a switched off machine and with S controlled holding of a position. In all other cases the bit is set.
8	CTRL_READY	Movement command will be accepted
9	CTRL_FREE	Waiting for free signal. The EDC waits for the drive enable through the Host software or from the user.
10	CTRL_INIT_E	Emergency movement has to be activated
11	CTRL_SFTSET	Change of softends allowed
12	CTRL_SYNCINPUT	Sync input: 1 = ok, 0 = not connected
13	CTRL_SYNCHWAIT	Synch State: 1 = wait for Start
14	CTRL_SLAVE	Synch State: 0 = Master 1 = Slave
15	CTRL_E_ACTIVE	Emergency movement active

15.4 Controller State WORD 2

State word 2 of the closed loop controller (CtrlState2)		
Bit number	Constant	Description
6	CTRL_GUARD_CLOSED	Guard closed
7	CTRL_GUARD_LIM_MODE	Guard limited mode active
8	CTRL_GUARD_LIM_MODE_ACK	Guard limited mode acknowledge
9	CTRL_GUARD_LIM_OVERRUN	Guard limit overrun
10	CTRL_CYCLES_ACTIVE	Cycle command active (DoPECycle, DoPEDynCycles)
11	CTRL_HIGH_PRESSURE	High pressure active
12	CTRL_CALIBRATION	Calibration of analogue channels is currently active.
15	CTRL_ERROR	Deviation position controller

15.5 Drive Interface State Input 1

Bit number	Constant	Description
0	DrvIntfErr	1 = Drive Interface Error
1	UpStop	1 = Default; 0 = Upper limit switch triggered
2	DownStop	1 = Default; 0 = Lower limit switch triggered
3	CtrlEnable	Hardware controller enable input; 1 = enabled; 0 = disabled
4	MoveEnable	Hardware movement enable input; 1 = enabled; 0 = disabled If disabled machine stops in position control (SHALT)
5	ManRst	State of the Manual Reset Button
6	EstopExt	1 = Default; 0 = External emergency stop triggered
7	LEDR	1 = Drive ON LED is on
8	DrvIntfOutpErrGND	1 = Drive interface/box output driver error (shortcut, overcurrent at drive IO's)
9	EstopRMC	1 = Default; 0 = RMC emergency stop triggered
10	EDMErr	1 = External device monitoring error
11	DrvIntfWatchdogErr	1 = Default; 0 = Watchdog error
12	IntCtrlEnableR	1 = Internal drive amplifier ready
13	CMDon	1 = Enable relay for external contactors on; 0 = Enable relay off
14	DrvIntfComEr	1 = Drive interface communication error DIC
15	DrvBoxOutpErrGND	1 = Drive box output driver error (shortcut, overcurrent at standard IO's)

15.6 Drive Interface State Input 2

Bit number	Constant	Description
0	EstopUpDown	1 = Default; 0 = Emergency stop or limit switch triggered
1	ManRstAct	Manual Reset Configuration 1 = Activated; the drive on command is possible after a Manual Reset pulse, only 0 = Disable; the drive on command is always possible
2	DrvIntfOutpErr	1 = Drive interface/box output monitoring error (drive IO's)
3	DrvBoxOutpErr	1 = Drive box output monitoring error (standard IO's)
4	SupEstopErr	1 = Emergency stop supply error (shortcut, overcurrent, ...)
5	EstopRMCErr	1 = RMC emergency stop error (shortcut, contact error, cross connection, ...)
6	EstopExtErr	1 = External emergency stop error (shortcut, contact error, cross connection, ...)
7	EstopUpDownErr	1 = External emergency stop or limit switch error (shortcut, contact error, cross connection, ...)
8	EDConErr	1 = Overrun relay error
9	CMDonErr	1 = ON-Relay error 1=Fehler im externen Sicherheitskreis / Relais-/Schütz-Ansteuerungskreis
10	EstopAppR	1 = Default; 0 = Application emergency stop triggered
11	BrakeOpenR	1 = release brake; 0 = close brake
12	BypassR	1 = activate bypass valve; 0 = close bypass valve
13	ManRstReqDrvInt	1 = Manual Reset Required from Drive Interface
14	EDCrdyR	1 = EDC ready, Closed Loop Control possible
15	CMDrdyR	1 = Contoller active

15.7 Drive Interface State Output

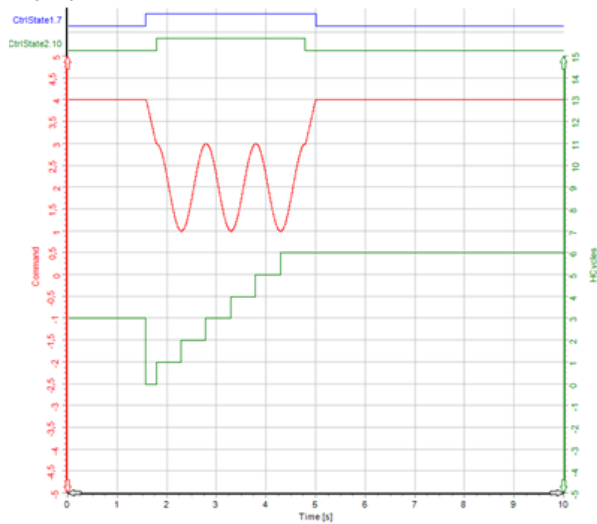
Bit number	Constant	Description
0	EstopApp	1 = Default; 0 = Application emergency stop
1	AVF	1 = Enable drive amplifier
2	BrakeOpen	1 = release brake
3		
4	ClrUpDnStop	1 = Limit switch Up/Down overrun (bypass Up/Down limit switch)
5	DrvIntfOn	1 = start drive on sequence (initiated by RMC or DoPE)
6	DrvIntfOff	1 = turn off drive
7	AVF_INT	1 = Enable internal drive amplifier
8	CtrlBlocked	1 = Controller not ready
9	DrvIntfReset	1 = Reset drive interface/box (clear error)
10	Bypass	1= activate bypass valve
11	EDCrdy	1 = EDC ready (no errors active)
12	IntCtrlEnable	1 = Internal control enable signal
13	ManRstReq	1 = Manual reset requested

15.8 Detecting Cycles

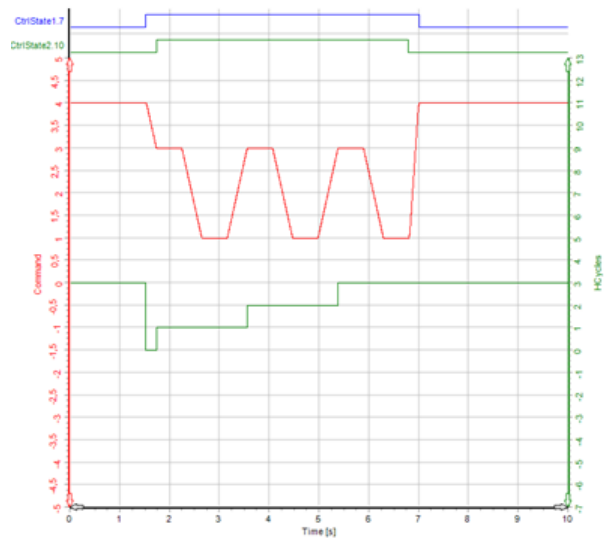
Cycle handling

- Bit7 of CtrlState1 signals 'movement command is active'
- Bit10 of CtrlState2 signals 'cycles active'
- Cycles count resets to zero at the beginning of the movement command (rising edge of CtrlState1.7)
- Cycles count increases at the **start of each (half)cycle**
- 'cycles active' resets to low at the end of the last cycle

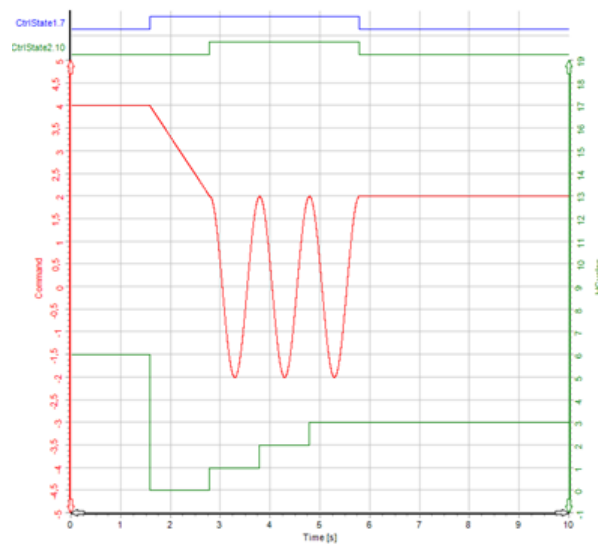
DynCycle Command



Cycle Command



PcCmd



16 DoPE System Message

Number	Message constant	Description
0	SYSTEM_MSG_NOERROR	No error
General runtime errors		
10000	SYSTEM_MSG_UNKNOWN	Unknown runtime error
10001	SYSTEM_MSG_POWERAMP_ERROR	Power amplifier error
10003	SYSTEM_MSG_DRIVE_NOTREADY	Drive not ready
10004	SYSTEM_MSG_DRIVE_RELEASE	Drive release withdrawn
10005	SYSTEM_MSG_UPPER_SENLIMIT	Upper sensor limit exceeded
10006	SYSTEM_MSG_LOWER_SENLIMIT	Lower sensor limit exceeded
10007	SYSTEM_MSG_LIMITSWITCH	Upper or lower hard-limit switch
10008	SYSTEM_MSG_DEVIATION	Controller deviation error
10009	SYSTEM_MSG_NODIGIPOTI	No DigiPoti configured
10010	SYSTEM_MSG_NORMC	No remote control plugged
10011	SYSTEM_MSG_EMERGENCY_OFF	Emergency switch activated
10012	SYSTEM_MSG_NOMEMORY	not enough memory
10013	SYSTEM_MSG_NO_LICENSE	No licence found
10014	SYSTEM_MSG_HARDWARE_VERSION	Wrong hardware version
System EEPROM errors (flash disk)		
10100	SYSTEM_MSG_SYSEEP_CRC	data CRC error
10101	SYSTEM_MSG_SYSEEP_NOBLOCK	block not found
10102	SYSTEM_MSG_SYSEEP_BLOCKLENGTH	wrong block length
10103	SYSTEM_MSG_SYSEEP_NOMEMORY	not enough EEPROM memory
10104	SYSTEM_MSG_SYSEEP_BIOSNOFUNC	BIOS: function not found
10105	SYSTEM_MSG_SYSEEP_BIOSNODEVICE	BIOS: device error
10106	SYSTEM_MSG_SYSEEP_BIOSPARA	BIOS: parameter error
10107	SYSTEM_MSG_SYSEEP_BIOSREAD	BIOS: read error
10108	SYSTEM_MSG_SYSEEP_BIOSWRITE	BIOS: write error
10109	SYSTEM_MSG_SYSEEP_BIOSREENT	BIOS: reentrance error
10110	SYSTEM_MSG_SYSEEP_USB_DISK_MISSING	USB disk not plugged
10111	SYSTEM_MSG_SYSEEP_USB_DISK_FULL	USB disk full
General initialization errors		
10200	SYSTEM_MSG_INIT_ENDDEV	Error at initialization level 1
10201	SYSTEM_MSG_INIT_ENDINI	Error at initialization level 2
10202	SYSTEM_MSG_INIT_SYSTIME	Wrong system time
10204	SYSTEM_MSG_INIT_DATA_TRANSMISSION_RATE	Wrong data transmission rate
10207	SYSTEM_MSG_INIT_MAINBOARD	EDC main board initialization error
10208	SYSTEM_MSG_INIT_MODULE_ERROR	Module error
10209	SYSTEM_MSG_INIT_CTRL	Error closed loop controller
10210	SYSTEM_MSG_INIT_SYSTIME_USB	System time with USB not possible
10211	SYSTEM_MSG_INIT_SYSTIME_SENSORS	Too many sensors for this system time
Sensor initialization errors		
10300	SYSTEM_MSG_SEN_EEP_NOTFOUND	Sensor EEPROM not found
10301	SYSTEM_MSG_SEN_EEP_BCC	Sensor EEPROM data BCC error
10302	SYSTEM_MSG_SEN_EEP_CLASS	Unknown sensor EEPROM class
10303	SYSTEM_MSG_SEN_NOTFOUND	Sensor not found
10304	SYSTEM_MSG_SEN_INITBYTE	Initialization word error
10305	SYSTEM_MSG_SEN_INIT	Sensor initialization error
10306	SYSTEM_MSG_SEN_PARA	Wrong sensor parameter
10307	SYSTEM_MSG_SEN_CORR	Wrong sensor correction value
10308	SYSTEM_MSG_SEN_MCINTEGR	Wrong integration time for measured channel values
10309	SYSTEM_MSG_SEN_CTRLINTEGR	Wrong integration time for closed loop control
10310	SYSTEM_MSG_SEN_LIMIT	Wrong sensor limits

10311	SYSTEM_MSG_SEN_NOMINALACC	Wrong nominal acceleration
10312	SYSTEM_MSG_SEN_NOMINALSPEED	Wrong nominal speed
10313	SYSTEM_MSG_SEN_POS_CTRL	Wrong parameter for position closed loop control
10314	SYSTEM_MSG_SEN_SPEED_CTRL	Wrong parameter for speed closed loop control
10315	SYSTEM_MSG_SEN_BIOS	Wrong BIOS Version
10316	SYSTEM_MSG_SEN_OFFSET	Wrong sensor offset
10317	SYSTEM_MSG_SEN_SCALE	Wrong sensor scale
10318	SYSTEM_MSG_SEN_DITHER_FREQ	Wrong dither frequency
10319	SYSTEM_MSG_SEN_DITHER_AMPL	Wrong dither amplitude
10320	SYSTEM_MSG_SEN_DITHER_INIT	dither initialization error
Output channel initialization errors		
10400	SYSTEM_MSG_OC_EEP_NOTFOUND	Output channel EEPROM not found
10401	SYSTEM_MSG_OC_EEP_BCC	Output channel EEPROM data BCC error
10402	SYSTEM_MSG_OC_EEP_CLASS	Unknown Output channel EEPROM class
10403	SYSTEM_MSG_OC_INIT	Output channel initialization error
10404	SYSTEM_MSG_OC_VOLTAGE	Wrong voltage for DDAXx
10405	SYSTEM_MSG_OC_CURRENT	Wrong current for DDAXx
10406	SYSTEM_MSG_OC_POWER	Wrong power for DDAXx
10407	SYSTEM_MSG_OC_PARA	Wrong output channel parameter
10408	SYSTEM_MSG_OC_MAX_CURR_TIME	Wrong max current time for I²T
10412	SYSTEM_MSG_OC_CURRENT_CTRL	Wrong parameter for current closed loop control
10413	SYSTEM_MSG_OC_MC2OUTPUT	Mc2Output initialization error
10414	SYSTEM_MSG_OC_MC2OUTPUT	Mc2Output mode not 164mplemented
Bit input/output initialization errors		
10500	SYSTEM_MSG_BIN_INIT	Bit input initialization error
10600	SYSTEM_MSG_BOUT_INIT	Bit output initialization error
10601	SYSTEM_MSG_BOUT_ERROR_HIGH	Bit output error high (24V)
10602	SYSTEM_MSG_BOUT_ERROR_LOW	Bit output error low (0V)
IO-Signals initialization errors		
10710	SYSTEM_MSG_IO_GRIP_MODE	IOGrip: Mode not implemented
10711	SYSTEM_MSG_IO_GRIP_BIN	IOGrip: Bit input error
10713	SYSTEM_MSG_IO_GRIP_OC	IOGrip: Output channel error
10714	SYSTEM_MSG_IO_GRIP_LIMIT	IOGrip: Limit error
10730	SYSTEM_MSG_IO_EXT_MODE	IOExtensometer: Mode not implemented
10731	SYSTEM_MSG_IO_EXT_BIN	IOExtensometer: Bit input error
10740	SYSTEM_MSG_IO_FIXED_XHEAD_MODE	IOFixedXHead: Mode not implemented
10741	SYSTEM_MSG_IO_FIXED_XHEAD_BIN	IOFixedXHead: Bit input error
10750	SYSTEM_MSG_IO_HIGH_PRESSURE_MODE	IOHighPressure: Mode not implemented
10751	SYSTEM_MSG_IO_HIGH_PRESSURE_BIN	IOHighPressure: Bit input error
10753	SYSTEM_MSG_IO_HIGH_PRESSURE_OC	IOHighPressure: Output channel error
10780	SYSTEM_MSG_IO_MISC_MODE	IOMisc: Mode not implemented
10781	SYSTEM_MSG_IO_MISC_BIN	IOMisc: Bit input error
10790	SYSTEM_MSG_IO_MISC_TEMPERATURE1	IOMisc: temperature1 (warning)
10791	SYSTEM_MSG_IO_MISC_TEMPERATURE2	IOMisc: temperature2 (emergency off)
10792	SYSTEM_MSG_IO_MISC_OIL_LEVEL	IOMisc: oil level(emergency off)
10793	SYSTEM_MSG_IO_MISC_OIL_FILTER	IOMisc: oil filter (warning)
10794	SYSTEM_MSG_IO_MISC_POWER_FAIL	IOMisc: power fail (emergency off)
10820	SYSTEM_MSG_IO_LIMIT_POS_MODE	IOLimitPos: Mode not implemented
10821	SYSTEM_MSG_IO_LIMIT_POS_BIT	IOLimitPos: Bit device error
10830	SYSTEM_MSG_IO_LIMIT_SPEED_MODE	IOLimitSpeed: Mode not implemented
10831	SYSTEM_MSG_IO_LIMIT_SPEED_BIT	IOLimitSpeed: Bit device error
10840	SYSTEM_MSG_IO_LIMIT_LOAD_MODE	IOLimitLoad: Mode not implemented
10841	SYSTEM_MSG_IO_LIMIT_LOAD_BIT	IOLimitLoad: Bit device error
10850	SYSTEM_MSG_IO_SHALT_MODE	IOSHalt: Mode not implemented
10851	SYSTEM_MSG_IO_SHALT_BIN	IOSHalt: Bit input error
10860	SYSTEM_MSG_IO_GUARD_MODE	IOGuard: Mode not implemented
10861	SYSTEM_MSG_IO_GUARD_BIT	IOGuard: Bit device error

10862	SYSTEM_MSG_IO_GUARD_INIT	IOGuard: initialization error
10900	SYSTEM_MSG_LANGUAGE_READ	Language file: read error
10901	SYSTEM_MSG_LANGUAGE_VERSION	Language file: wrong version
10902	SYSTEM_MSG_LANGUAGE_SYNTAX	Language file: syntax errors
11000	SYSTEM_MSG_SYNC_ERROR_PARAM	Sync error: wrong parameter
11001	SYSTEM_MSG_SYNC_ERROR_VERSION	Sync error: wrong hardware version
11002	SYSTEM_MSG_SYNC_ERROR_NO_MASTER	Sync error: no master present

17 DoPE Error Constants

Error number	Error constant	Description
0	DoPERR_NOERROR	No error
1	DoPERR_NOFLOAT	No float in WIN16 callback
2	DoPERR_SYNC	Synchronization to callback failed
3	DoPERR_TIMEOUT	Timeout at await answer
4	DoPERR_NOFNC	Function not implemented
5	DoPERR_VERSION	No compatible Version EDC-DoPE
6	DoPERR_INIT	Initialization Error Subsystem
7	DoPERR_PARAMETER	Invalid parameter
9	DoPERR_RTE_UNHANDLED	Unhandled runtime error
Command errors		
1001	DoPERR_CMD_PARCORR	Error in parameter (corrected)
1003	DoPERR_CMD_PAR	Error in parameter. not correctable
1004	DoPERR_CMD_XMOVE	X-Head is not halted
1005	DoPERR_CMD_INITSEQ	Sequence in init not observed
1006	DoPERR_CMD_NOTINIT	Controller part not initialized
1007	DoPERR_CMD_DIR	Movement direction not possible
1008	DoPERR_CMD_TMP	Required resource not available
1009	DoPERR_CMD_RUNTIME	Run time error active
1010	DoPERR_CMD_INTERN	Internal error in subsystem
1011	DoPERR_CMD_MEM	Insufficient memory
1012	DoPERR_CMD_CST	Wrong controller Structure
1013	DoPERR_CMD_NIM	Command not implemented
2001	DoPERR_CMD_MSGNO	Unknown message number
2003	DoPERR_CMD_VERSION	Wrong PE interface version
2004	DoPERR_CMD_OPEN	Set-up not opened
2005	DoPERR_CMD_MEMORY	Not enough memory
Machine normalization errors		
0x4001	DoPERR_PARDS	Parameter Error
0x4002	DoPERR_ZERODIV	Division by ZERO
0x4003	DoPERR_OVFLOW	Overflow
0x4004	DoPERR_NIN	Not Initialized
Low level communication errors		
0x8001	DoPERR_NODATA	No receiver data available
0x8002	DoPERR_NOBUFFER	No transmitter buffer available
0x8003	DoPERR_OFFLINE	Connection is offline
0x8004	DoPERR_HANDLE	Invalid DoPE handle
0x8005	DoPERR_MSGSIZE	Message to long
0x8007	DoPERR_NOMEM	Not enough heap memory
0x8008	DoPERR_BADPORT	Invalid device ID
0x800A	DoPERR_OPEN	Device already in use
0x800B	DoPERR_HARDWARE	Device not present
0x800C	DoPERR_NOTOPEN	Connection not open
0x800D	DoPERR_PORTLIMIT	Unused
0x800E	DoPERR_NOTIMER	No timer for timeout check
0x800F	DoPERR_NODRIVER	No driver available
0x8010	DoPERR_NOTHREAD	Win32: Thread creation failed
0x8011	DoPERR_BADOS	Not supported operating system
0x8012	DoPERR_THUNK	Win32: Thread creation failed
-1	DoPERR_INTERNAL	Internal driver error

18 For Experts

18.1 Realtime Version of DoPE Event Handler

For each event, there are two versions of event handlers:

- The “normal” event handler (e.g. DoPESetOnDataHdlr),
- And a real time version (e.g. DoPESetOnData**Rt**Hdlr).

The difference of these two versions is the time priority; the event handler has to be called.

For the normal priority, DoPE uses the Windows message queue, while the real time handler is called by DoPE at a high priority level.

A realtime handler must not call any functions that read data from EDC like DoPERdSetup.

Application programmer using the real time event handler must be aware of the restrictions for functions at a high priority level.

In most cases, the normal version of event handler is sufficient.

19 .NET Wrapper

The DoPE10Net dynamic link library contains a .NET wrapper for the DoPE10 API.

The usage of the wrapper is demonstrated in the C# and Visual Basic .NET demo. Use these demos together with the Starter-Manual-C#-DoPE.NET.pdf and Starter-Manual-VB-DoPE.NET.pdf to write your own application.

The DoPE10Net wrapper is implemented in the Doli.DoPE namespace.

19.1 Establish Communication

The DoPENet wrapper provides the Edc class, which is a representation of a connection to the DOLI EDCi.

To open a connection you can use the following code:

```
private Edc MyEdc;  
MyEdc = new Edc(DoPE.OpenBy.DeviceId, 0);
```

or

```
MyEdc = new Edc(DoPE.OpenBy.FunctionIdDeviceId, 0);
```

This is equivalent to the classic API function DoPEOpenDeviceID and DoPEFunctionID.

The EdcList class represents a list of Edc objects and can be used as a replacement for

```
private EdcList MyEdcList;  
edcList = new EdcList(32);
```

This is equivalent to the classic API function DoPEOpenAll with InfoTableMaxEntries set to 32.

Disposing an Edc object closes the corresponding connection.

Disposing an EdcList object closes all connections, represented by the Edc objects in the list.

This cleanup will be performed automatically by shutting down the application.

19.2 Event Handler

Event handlers can be hooked to an Edc object.

For the OnDataBlock handler you can use the following code:

```
MyEdc.Eh.OnDataBlockHdlr += new DoPE.OnDataBlockHdlr(OnDataBlock);
```

This is equivalent to the classic API function DoPESetOnDataBlockHdlr function.

To remove the event handler you can use

```
MyEdc.Eh.OnDataBlockHdlr -= OnDataBlock;
```

This cleanup will be performed automatically by shutting down the communication.

19.3.NET API function groups

The DoPE10 API contains about 200 functions. In DoPENet they are divided into several groups for a better overview. E.g. the classic API function DoPEPos is part of the Move group. To find the corresponding .NET API function, replace DoPE by the Edc object, find the right group and add the rest of the function name. In our case we end up with

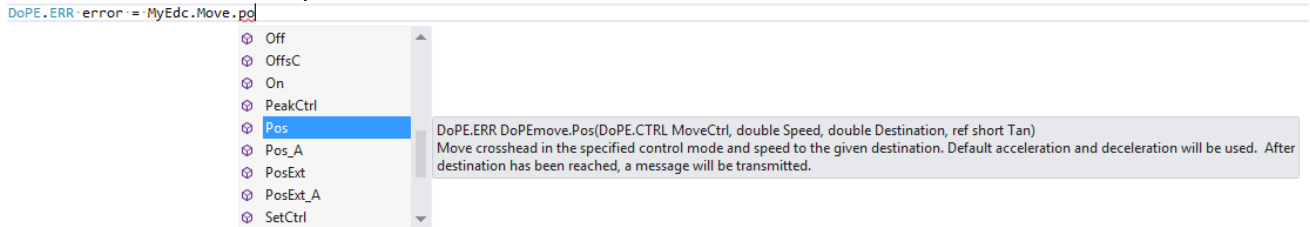
`MyEdc.Move.Pos`

This is the list of all available groups.

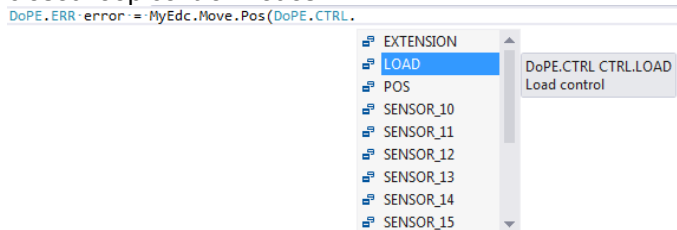
Block	Block Command handling
Check	check functions
Cmc	calculated measuring channels and peak value handling
Corr	sensor correction handling
Ctrl	Closed Loop Controller handling
Data	data acquisition
Debug	debug message handling
Display	display handling
Eh	event handler interface
Info	info handling
Ilc	ILC handling
Io	Input/Output handling
IoSignal	IO signals handling
Key	keyboard handling
Move	setup and movement command handling
Output	Output Channel handling
Rmc	RMC handling
Sensor	sensor EEPROM handling
SerSen	serial sensor handling
Setup	setup handling
Syn	synchronized data acquisition
Tare	tare, basic tare and reference signal handling

19.4 IntelliSense

The Microsoft online help system IntelliSense can be used with DoPENet. Simply type your Edc's object name followed by a period and IntelliSense lists all groups and other information. Selecting the Move group gives you a list of the available function, together with an overview of the argument list and a brief description of the function. Here's an example of the Pos function:



Same can be done with the Enums and Structs of the Doli.DoPE namespace. Start with DoPE followed by a period and the Enum or Struct name, to get a list of their members. Here's an example for the constants for closed loop control modes.



20 Versions

Version	Changes	Date	Name
10.00	First EDCi API Version	12.09.2016	HEG
10.01	New Keyboard Lock functions.	19.07.2017	HEG
10.02	DoPEOpen... error code documentation	19.07.2017	HEG
10.03	Obsolete Calculated Measuring Channel functions removed	21.08.2017	HEG
10.06	Obsolete Shield functions removed	01.02.2018	HEG
10.10	New name DoPE10.dll Linux support DoPEREINITIALIZEDAT_LEN corrected (must be 0x8000)	03.07.2018	HEG
10.11	New DoPEShieldLock function	11.01.2019	HEG
10.12	DoPEwDspMValue limitations for RMCi1	05.02.2019	HEG
10.13	System Message 10112 removed New System Message 10210...10212, 10601, 10602 New RMCi8 DoPEDestWnd description updated Brief .NET wrapper documentation	05.06.2019	HEG
10.14	EDCi10 support / limitations .NET format of DoPE functions	09.12.2019	HEG
10.15	New System Message 11000...11002 System Message 10203 removed	03.06.2020	HEG
10.16	New DoPEIOGuard Commands New Manual Reset status bits New System Message 10013...14 New System Message 10860...62	25.11.2020	HEG
10.17	New ".bmv2" file format for DoPEPcCmdFromFile	21.06.2021	GRS
10.18	New DoPEIOGuard muting commands	15.10.2021	GRS
10.19	New Event Handler for GuardOption. New RMC default handling description added. Fix Some errors in function descriptions corrected.	31.03.2022	GRS
10.20	Fix RMC default handling description corrected. New DoPEGetData for .NET added. New DoPESetPeakCtrlValue added New DoPEGetPeakCtrlValue added	31.03.2023	GRS
10.21	No changes	24.05.2023	GRS
10.22	Fix DoPEMachineData structure added	11.04.2024	GRS

10.23	Fix	DoPEDynCycle description updated	31.07.2024	GRS
	New	DoPESetSsiGenericSignalType replaced by DoPECalcSensorAbsGenericSignalType		
	New	DoPESSiGenericSignalTypeInfo (replaced by DoPECalcSensorAbsGenericData)		